SAL EDUCATION

SAL ENGINEERING AND TECHNICAL INSTITUTE

Opp. Science city, Sola-Bhadaj Road, Ahmedabad, Gujarat 380060
Ph:- 079 67129000 Website: www.sal.edu.in

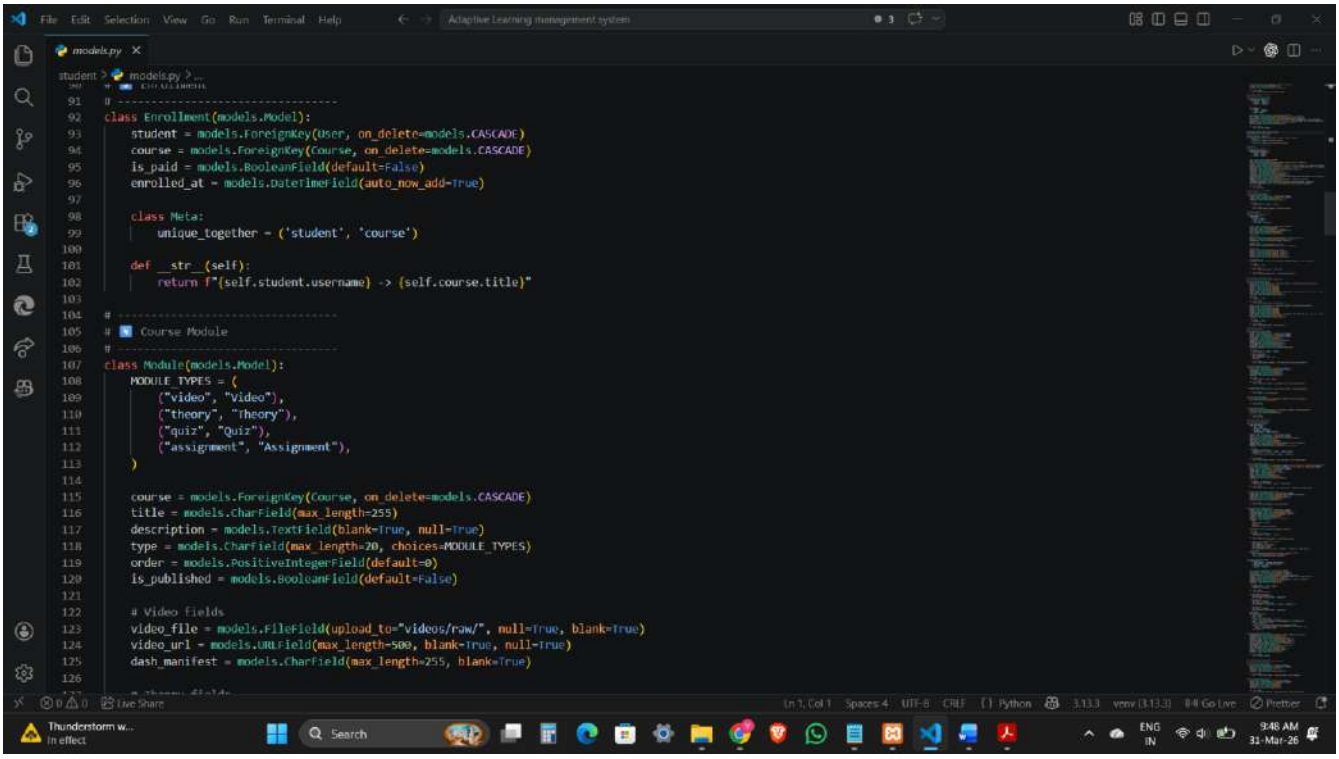
Student's Weekly Report of Internship

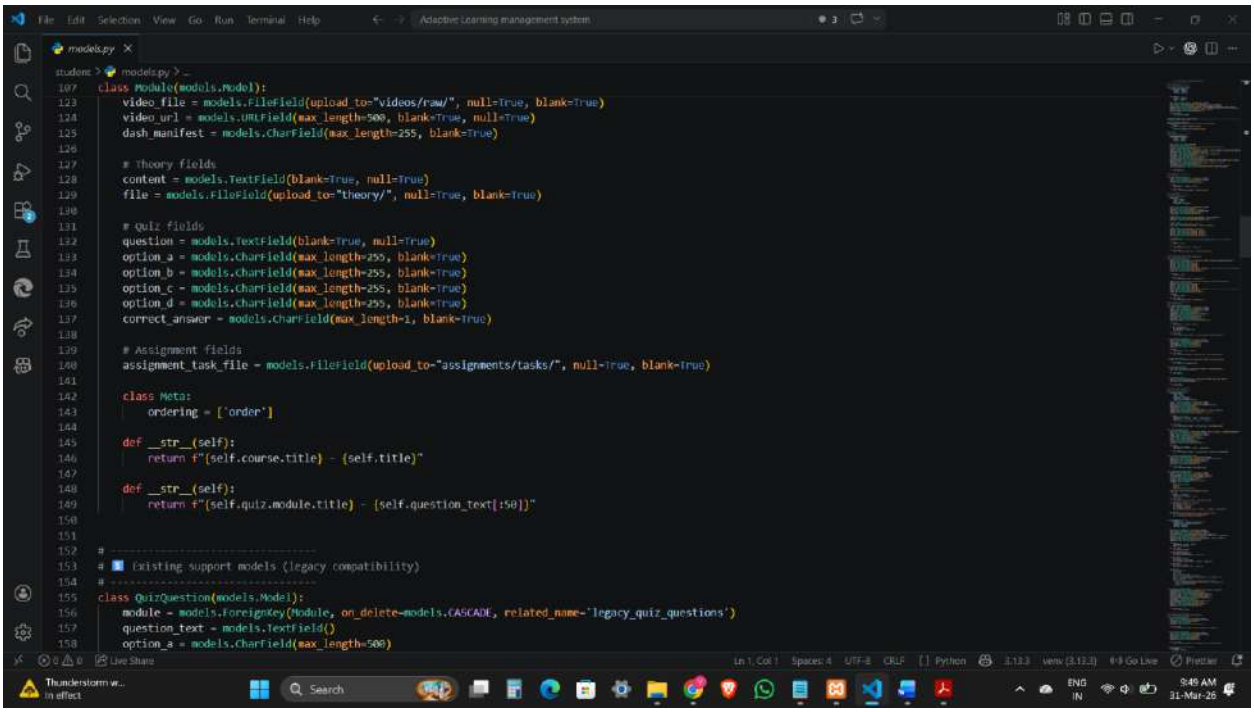
Student Name:	Vekariya Vraj Dipakbhai			
Enrollment Number:	221260116056			
Name Of Organization:	Bisag N			
External Guide Name:	Parth Sutariya			
External Guide Contact details:	**Email:**	Parthsutariya12@gmail.com	**Mob No:**	81558 51038
Internal Faculty Guide Name:	Prof. Amitkumar J. Patel			

Week 7 (16/02/2026 To 22/02/2026)

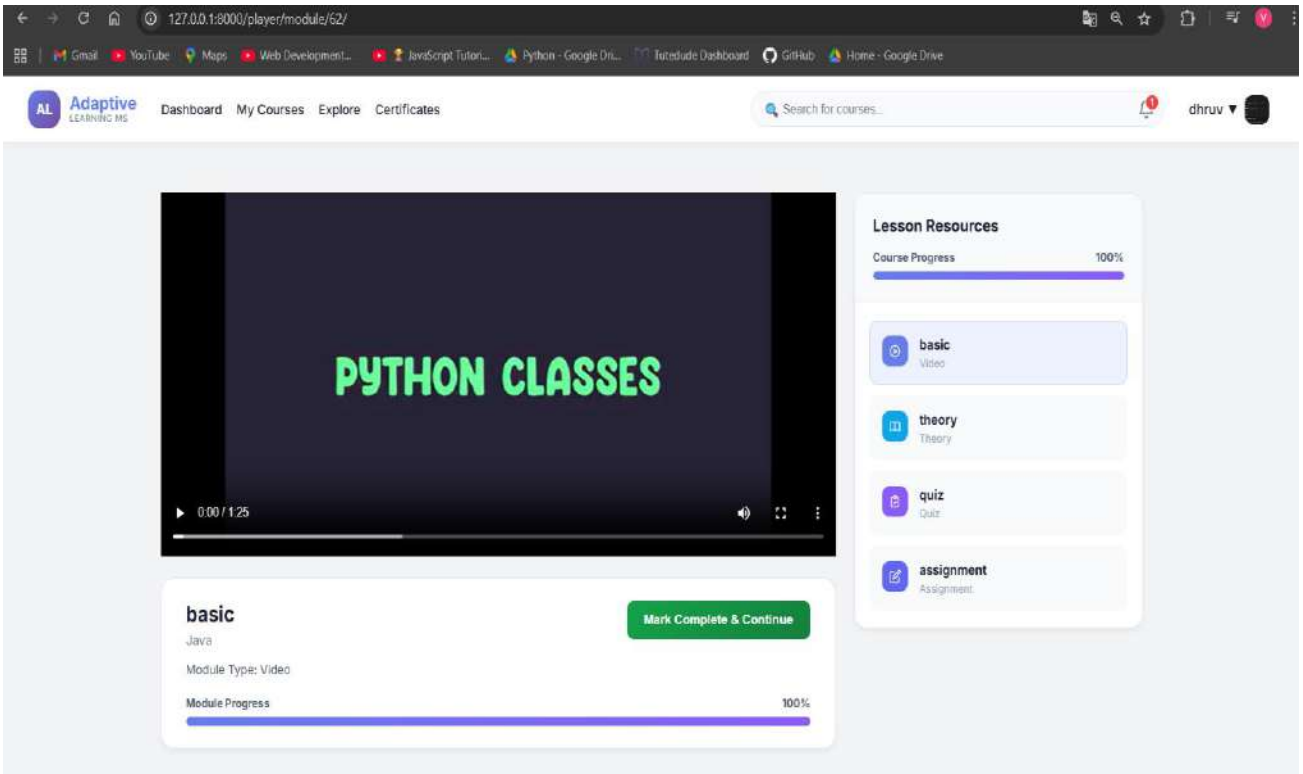
1. Implement course-module structure for student view (Video, Theory, Quiz flow)
2. Design student-related models (Enrollment, Progress tracking)
3. Develop basic student dashboard UI
4. Implement course enrollment functionality
5. Display course progress (percentage + module status)

Day 1 :- Implement course-module structure for student view (Video, Theory, Quiz flow)

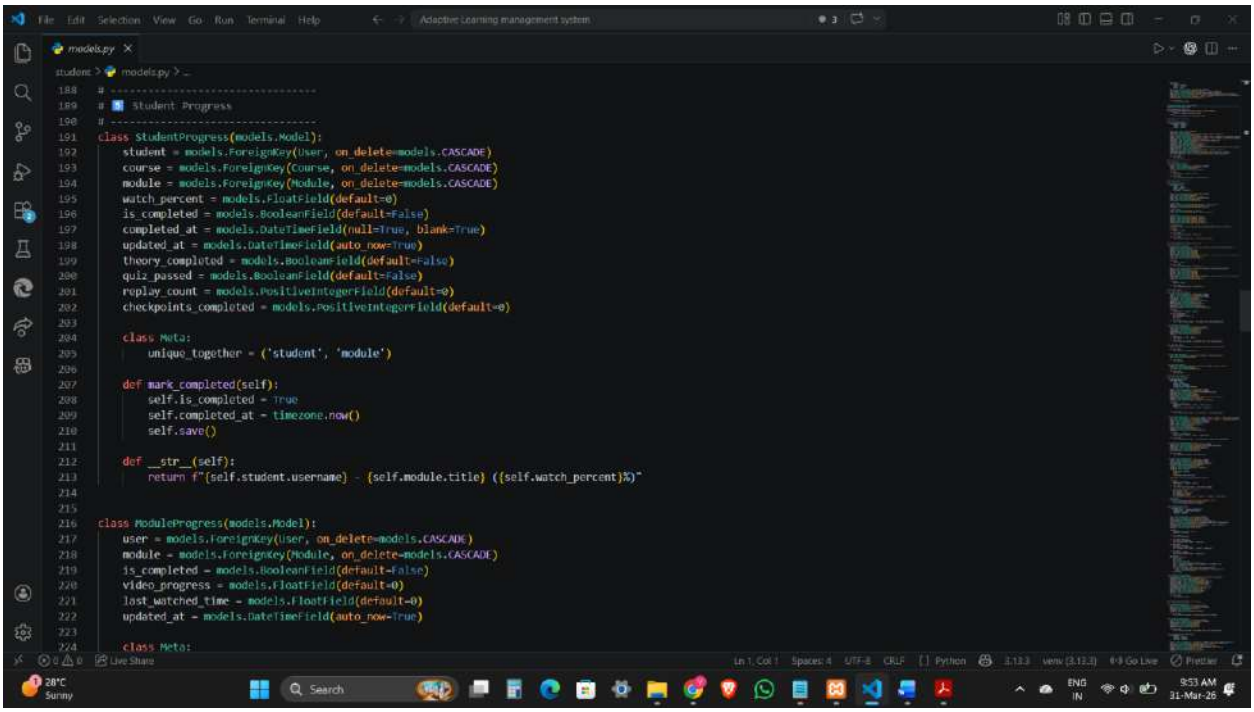




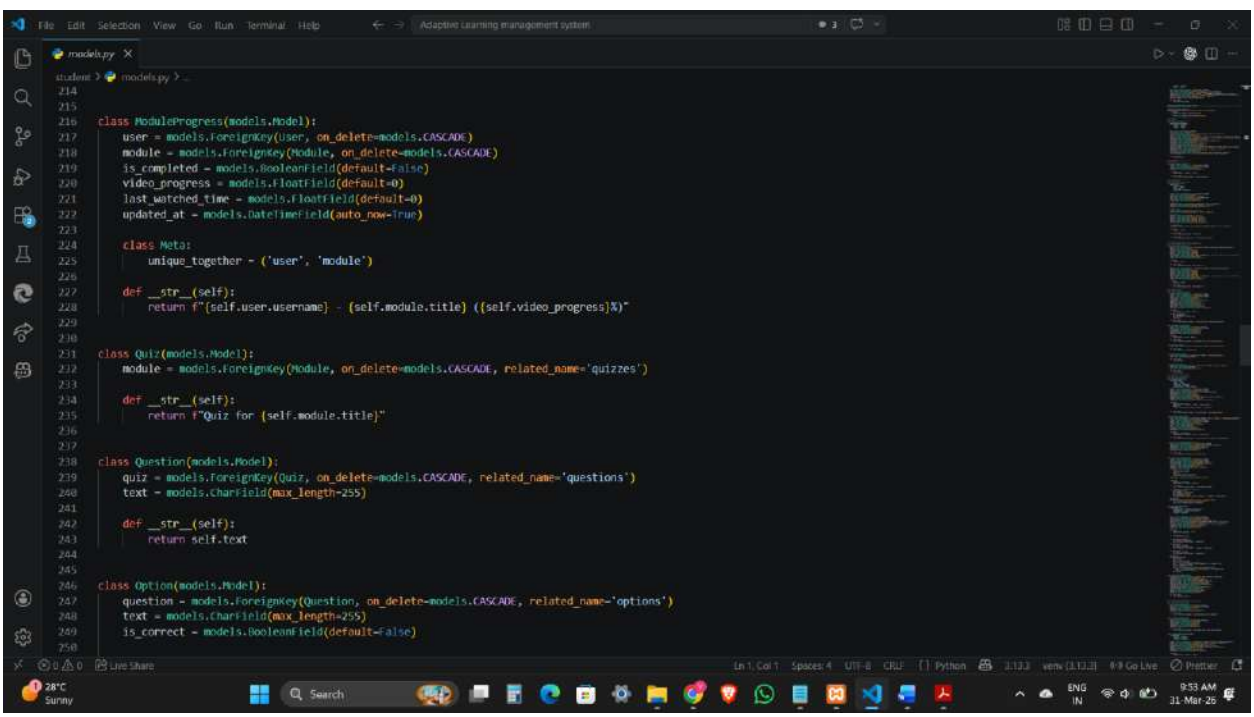
```python
class Module(models.Model):
    video_file = models.FileField(upload_to="videos/raw/", null=True, blank=True)
    video_url = models.URLField(max_length=500, blank=True, null=True)
    dash_manifest = models.CharField(max_length=255, blank=True)

    # Theory fields
    content = models.TextField(blank=True, null=True)
    file = models.FileField(upload_to="theory/", null=True, blank=True)

    # Quiz fields
    question = models.TextField(blank=True, null=True)
    option_a = models.CharField(max_length=255, blank=True)
    option_b = models.CharField(max_length=255, blank=True)
    option_c = models.CharField(max_length=255, blank=True)
    option_d = models.CharField(max_length=255, blank=True)
    correct_answer = models.CharField(max_length=1, blank=True)

    # Assignment fields
    assignment_task_file = models.FileField(upload_to="assignments/tasks/", null=True, blank=True)

    class Meta:
        ordering = ['order']

    def __str__(self):
        return f"{self.course.title} - {self.title}"

    def __str__(self):
        return f"{self.quiz.module.title} - {self.question_text[:50]}"


# ------------------------------
# 🔹 Existing support models (legacy compatibility)
# ------------------------------
class QuizQuestion(models.Model):
    module = models.ForeignKey(Module, on_delete=models.CASCADE, related_name='legacy_quiz_questions')
    question_text = models.TextField()
    option_a = models.CharField(max_length=500)
```



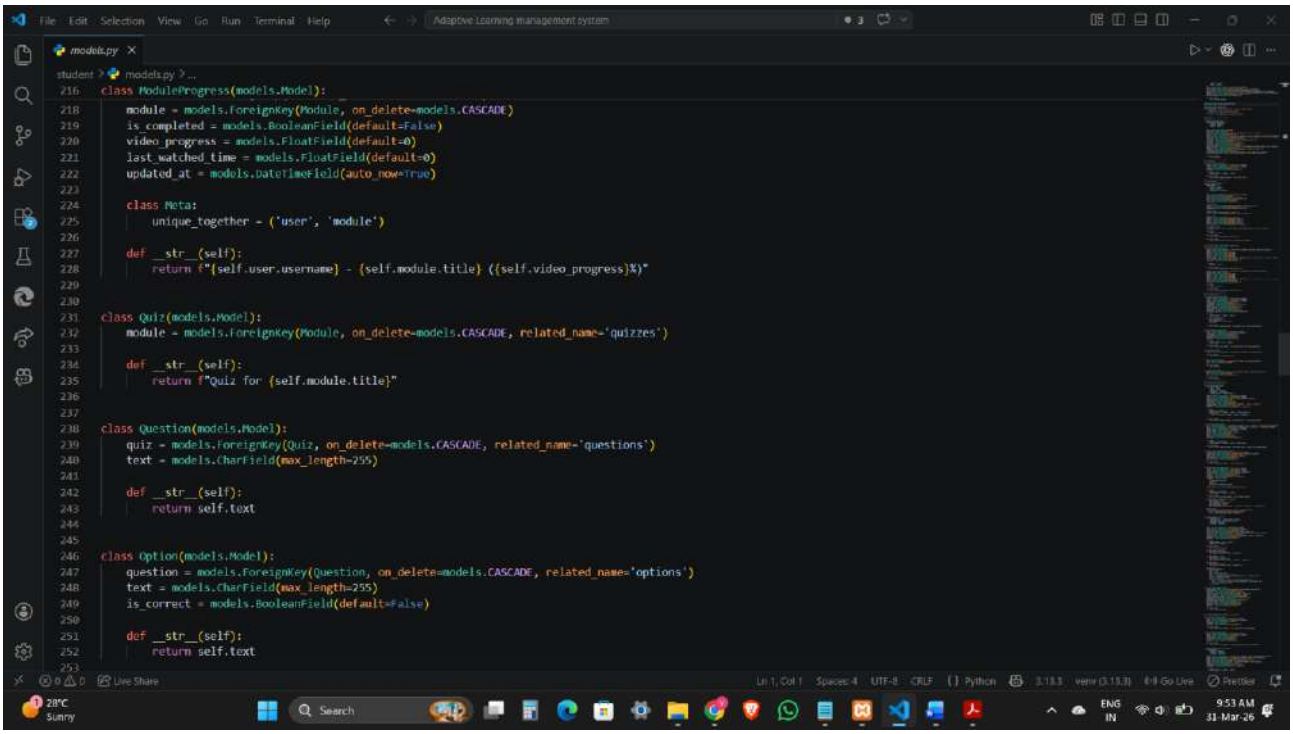
Day 2 :- Design student-related models (Enrollment, Progress tracking)

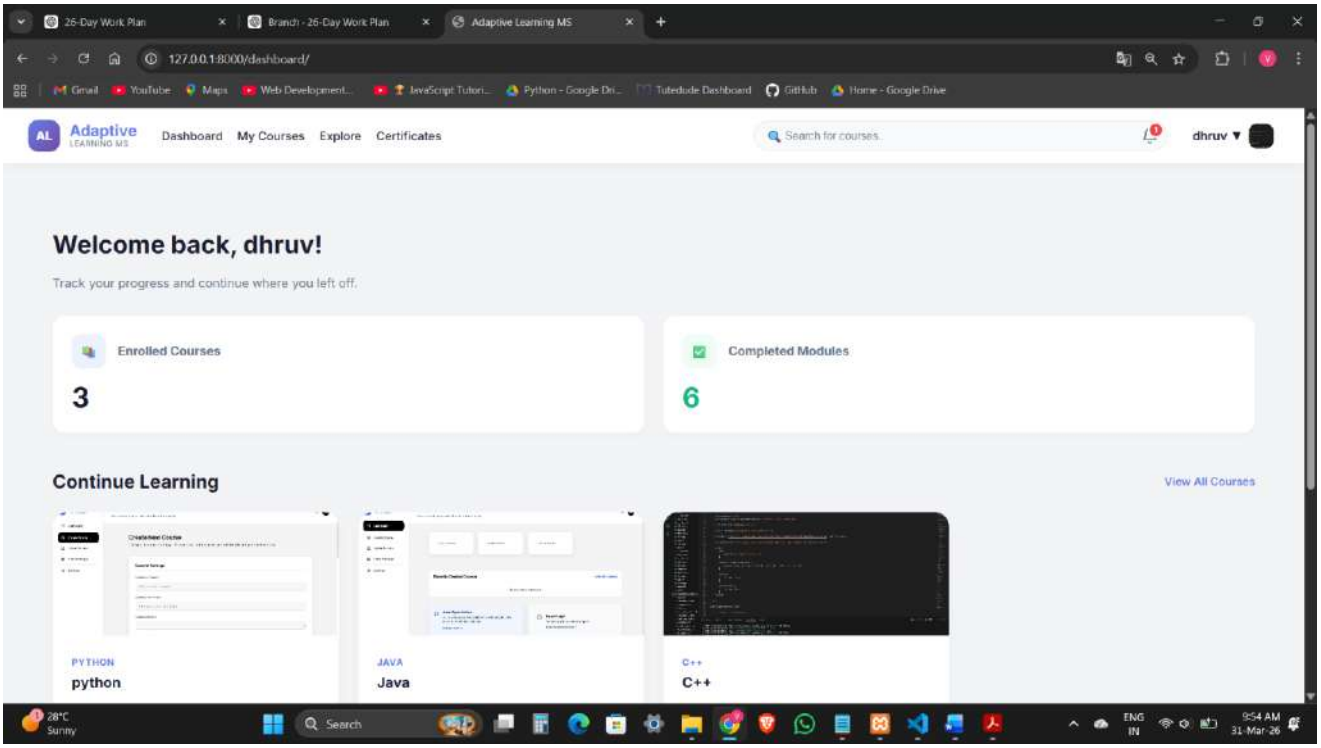


```python
# ---------------------------------
# 📊 Student Progress
# ---------------------------------
class StudentProgress(models.Model):
    student = models.ForeignKey(User, on_delete=models.CASCADE)
    course = models.ForeignKey(Course, on_delete=models.CASCADE)
    module = models.ForeignKey(Module, on_delete=models.CASCADE)
    watch_percent = models.FloatField(default=0)
    is_completed = models.BooleanField(default=False)
    completed_at = models.DateTimeField(null=True, blank=True)
    updated_at = models.DateTimeField(auto_now=True)
    theory_completed = models.BooleanField(default=False)
    quiz_passed = models.BooleanField(default=False)
    replay_count = models.PositiveIntegerField(default=0)
    checkpoints_completed = models.PositiveIntegerField(default=0)

    class Meta:
        unique_together = ('student', 'module')

    def mark_completed(self):
        self.is_completed = True
        self.completed_at = timezone.now()
        self.save()

    def __str__(self):
        return f"{self.student.username} - {self.module.title} ({self.watch_percent}%)"


class ModuleProgress(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    module = models.ForeignKey(Module, on_delete=models.CASCADE)
    is_completed = models.BooleanField(default=False)
    video_progress = models.FloatField(default=0)
    last_watched_time = models.FloatField(default=0)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
```

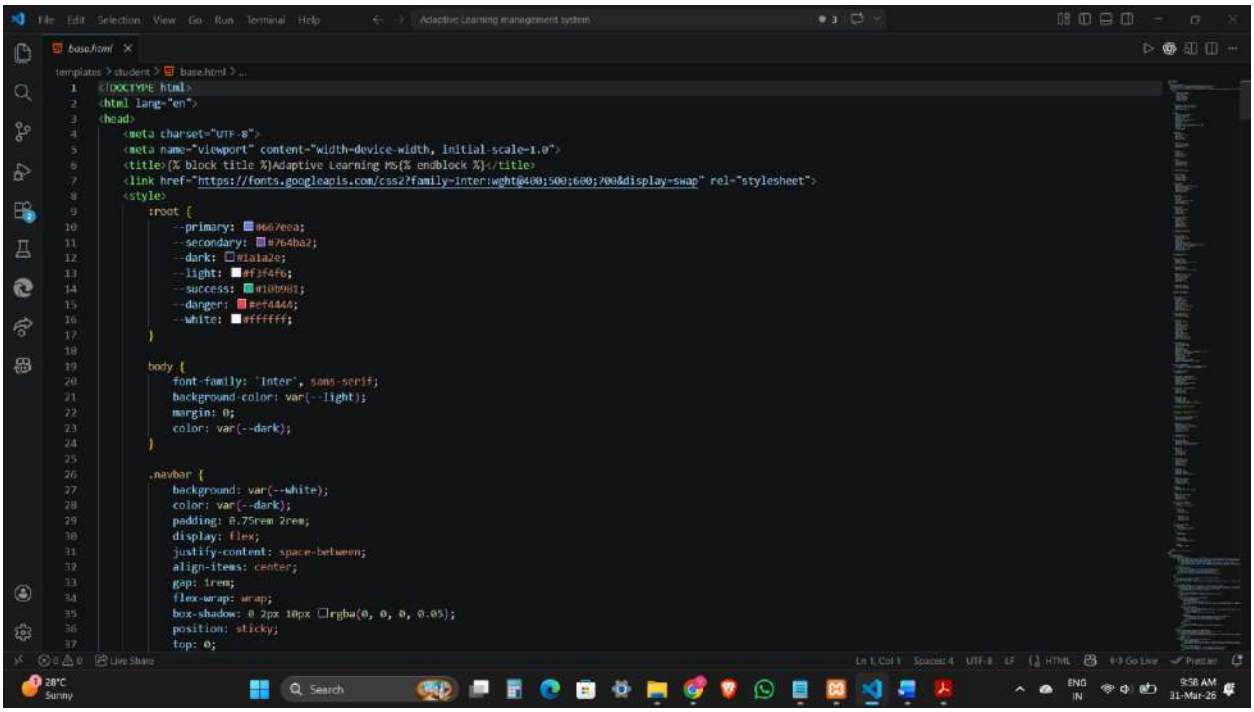


```python


class ModuleProgress(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    module = models.ForeignKey(Module, on_delete=models.CASCADE)
    is_completed = models.BooleanField(default=False)
    video_progress = models.FloatField(default=0)
    last_watched_time = models.FloatField(default=0)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        unique_together = ('user', 'module')

    def __str__(self):
        return f"{self.user.username} - {self.module.title} ({self.video_progress}%)"


class Quiz(models.Model):
    module = models.ForeignKey(Module, on_delete=models.CASCADE, related_name='quizzes')

    def __str__(self):
        return f"Quiz for {self.module.title}"


class Question(models.Model):
    quiz = models.ForeignKey(Quiz, on_delete=models.CASCADE, related_name='questions')
    text = models.CharField(max_length=255)

    def __str__(self):
        return self.text


class Option(models.Model):
    question = models.ForeignKey(Question, on_delete=models.CASCADE, related_name='options')
    text = models.CharField(max_length=255)
    is_correct = models.BooleanField(default=False)
```

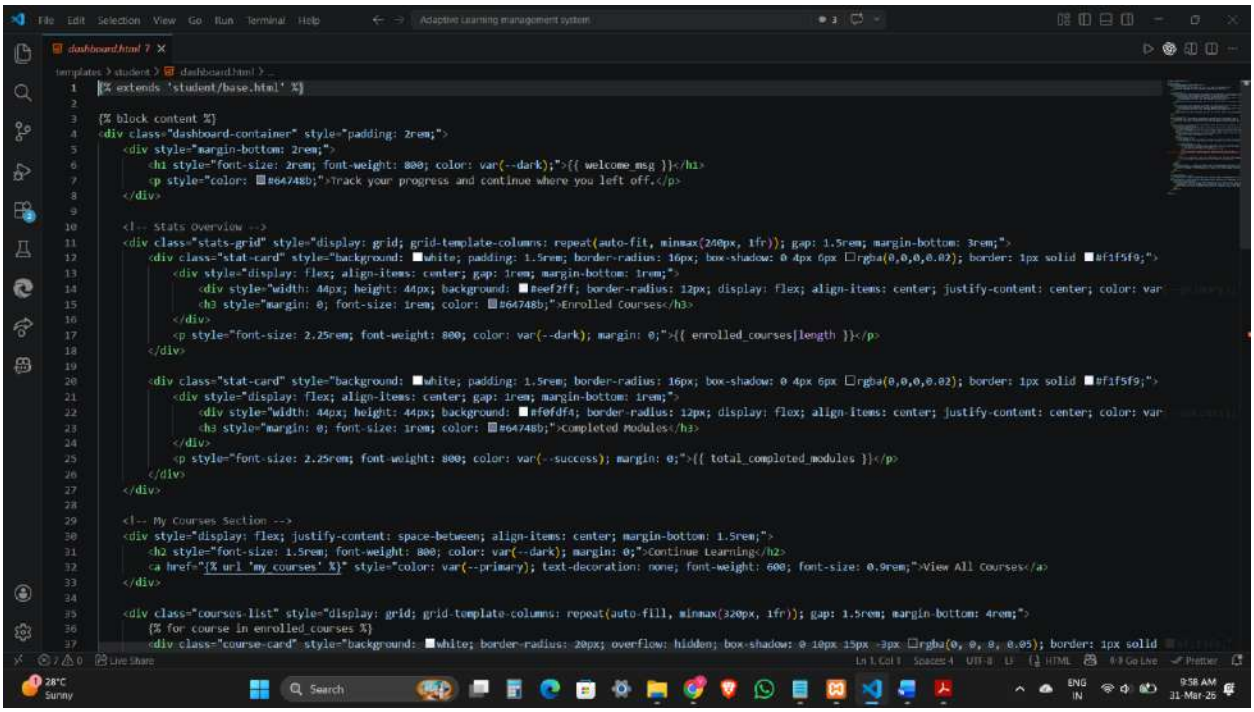

models.py
student > models.py > ...
class ModuleProgress(models.Model):
module = models.ForeignKey(Module, on_delete=models.CASCADE)
is_completed = models.BooleanField(default=False)
video_progress = models.FloatField(default=0)
last_watched_time = models.FloatField(default=0)
updated_at = models.DateTimeField(auto_now=True)
class Meta:
unique_together = ('user', 'module')
def __str__(self):
return f"{self.user.username} - {self.module.title} ({self.video_progress}%)"
class Quiz(models.Model):
module = models.ForeignKey(Module, on_delete=models.CASCADE, related_name='quizzes')
def __str__(self):
return f"Quiz for {self.module.title}"
class Question(models.Model):
quiz = models.ForeignKey(Quiz, on_delete=models.CASCADE, related_name='questions')
text = models.CharField(max_length=255)
def __str__(self):
return self.text
class Option(models.Model):
question = models.ForeignKey(Question, on_delete=models.CASCADE, related_name='options')
text = models.CharField(max_length=255)
is_correct = models.BooleanField(default=False)
def __str__(self):
return self.text


26-Day Work Plan
Branch - 26-Day Work Plan
Adaptive Learning MS
127.0.0.1:8000/dashboard/
Adaptive
LEARNING MS
Dashboard
My Courses
Explore
Certificates
Search for courses...
dhruv
Welcome back, dhruv!
Track your progress and continue where you left off.
Enrolled Courses
3
Completed Modules
6
Continue Learning
View All Courses
PYTHON
python
JAVA
Java
C++
C++

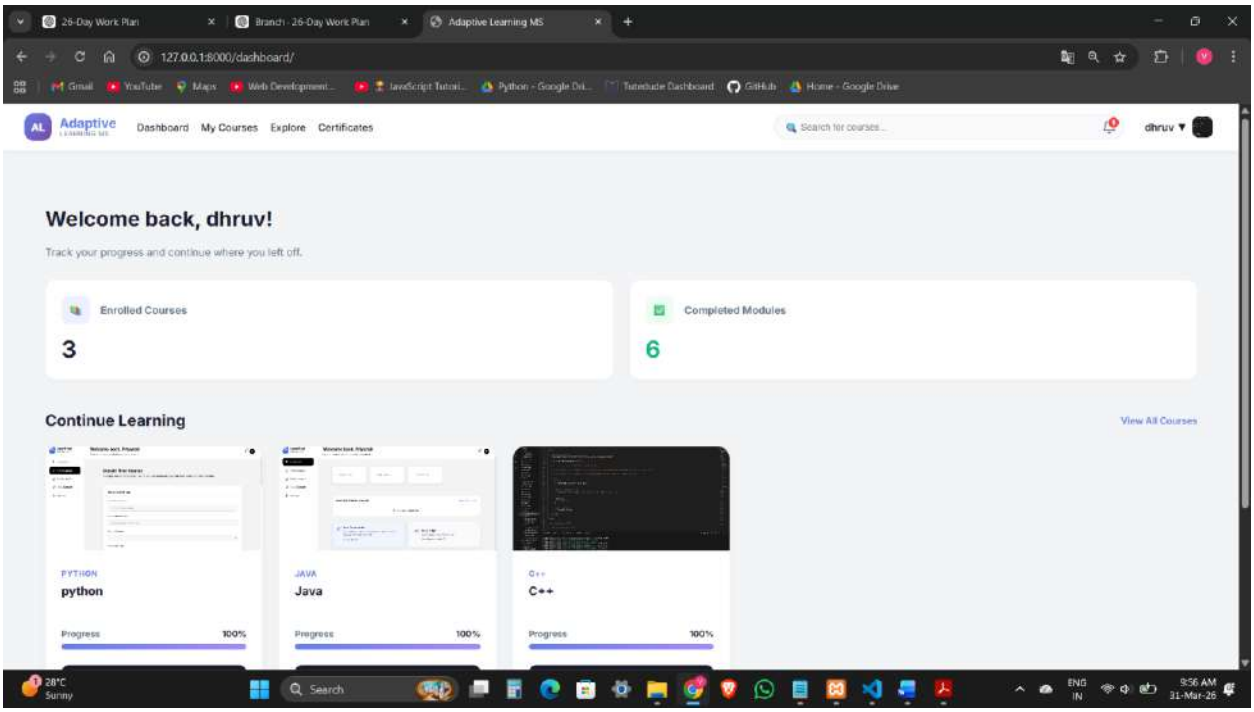
Day 3 :- Develop basic student dashboard UI

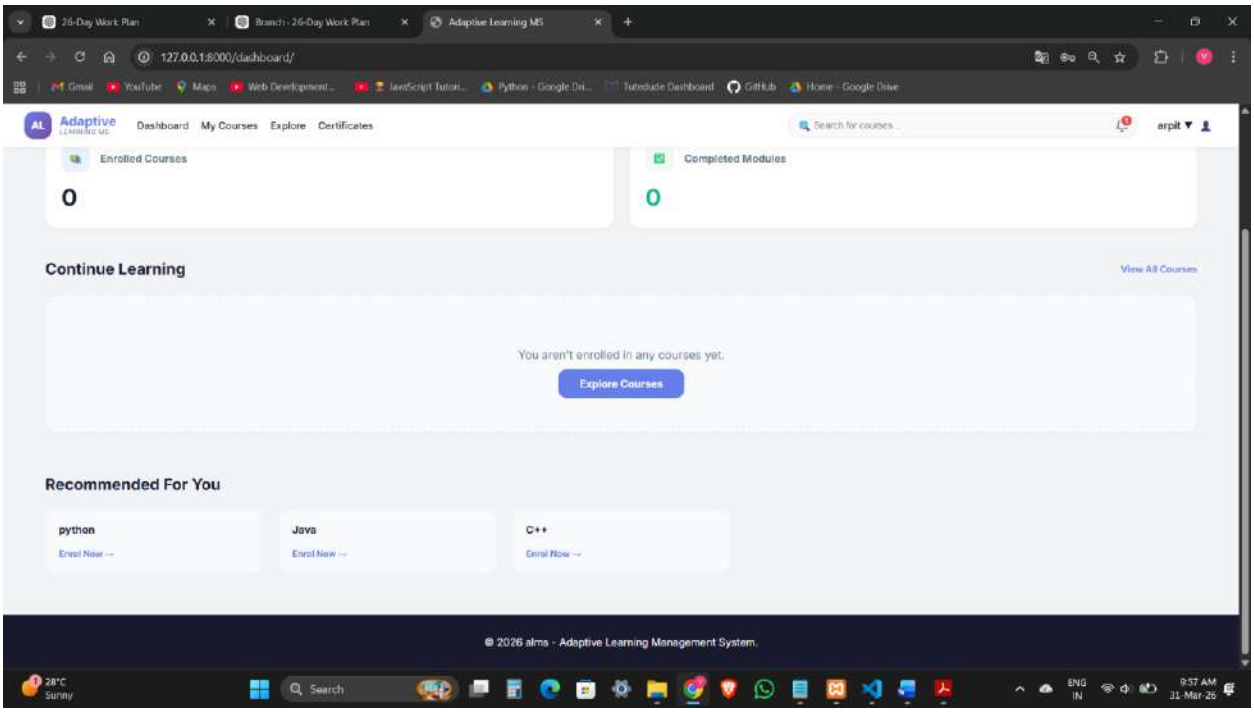


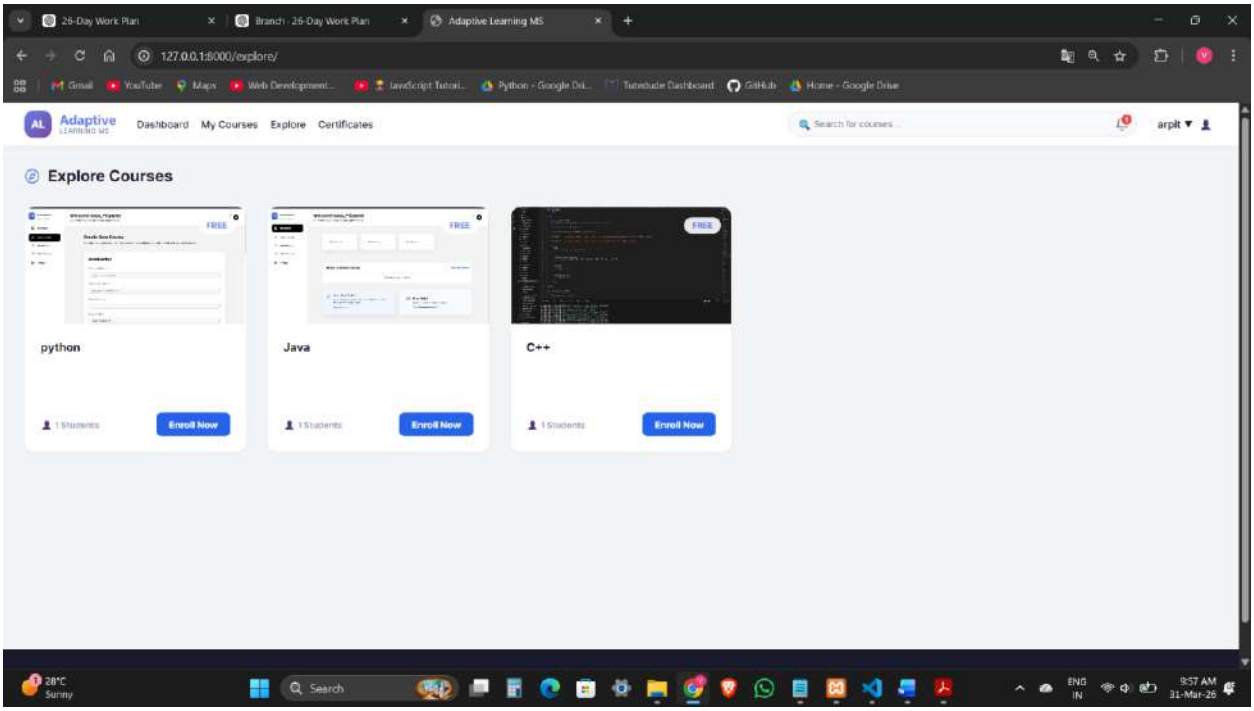
```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Adaptive Learning MS{% endblock %}</title>
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap" rel="stylesheet">
    <style>
        :root {
            --primary: #667eea;
            --secondary: #764ba2;
            --dark: #1a1a2e;
            --light: #f1f4f6;
            --success: #10b981;
            --danger: #ef4444;
            --white: #ffffff;
        }

        body {
            font-family: 'Inter', sans-serif;
            background-color: var(--light);
            margin: 0;
            color: var(--dark);
        }

        .navbar {
            background: var(--white);
            color: var(--dark);
            padding: 0.75rem 2rem;
            display: flex;
            justify-content: space-between;
            align-items: center;
            gap: 1rem;
            flex-wrap: wrap;
            box-shadow: 0 2px 10px rgba(0, 0, 0, 0.05);
            position: sticky;
            top: 0;
```



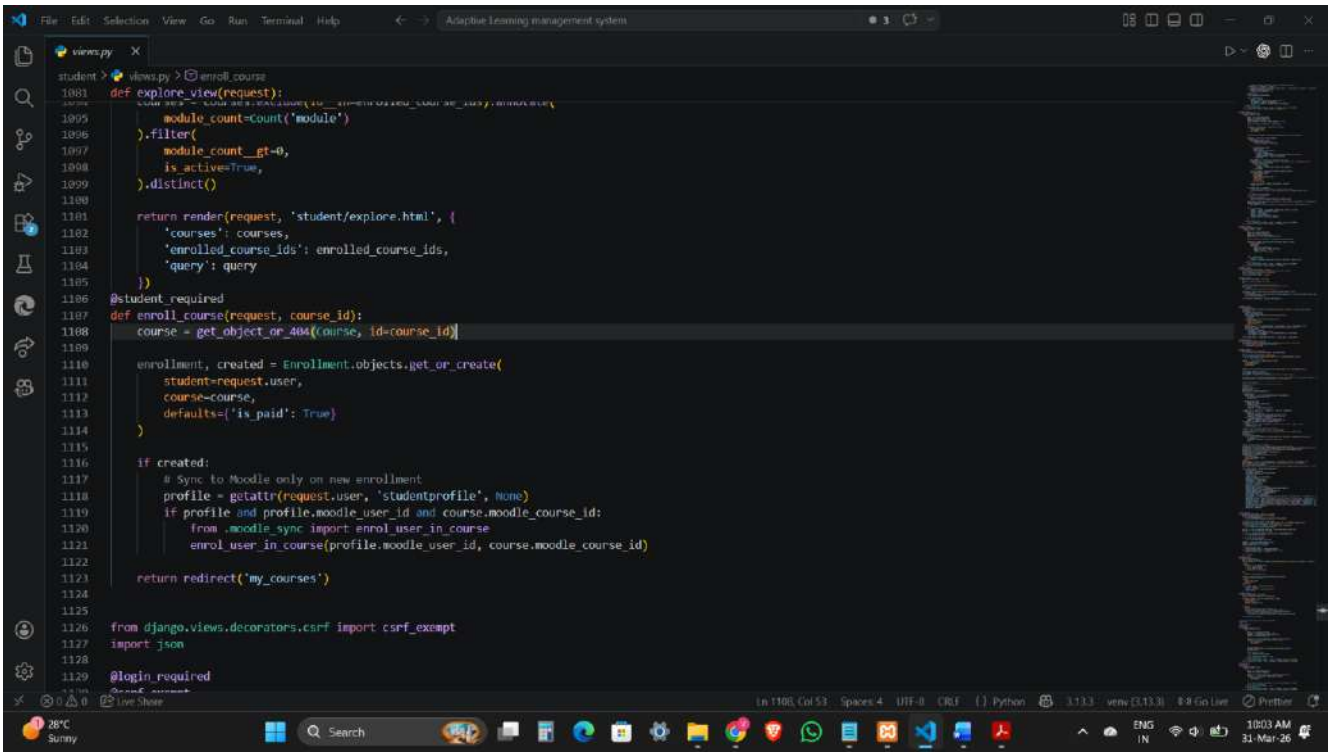
```html
{% extends 'student/base.html' %}

{% block content %}
<div class="dashboard-container" style="padding: 2rem;">
    <div style="margin-bottom: 2rem;">
        <h1 style="font-size: 2rem; font-weight: 800; color: var(--dark);">{{ welcome_msg }}</h1>
        <p style="color: #64748b;">Track your progress and continue where you left off.</p>
    </div>

    <!-- Stats Overview -->
    <div class="stats-grid" style="display: grid; grid-template-columns: repeat(auto-fit, minmax(240px, 1fr)); gap: 1.5rem; margin-bottom: 3rem;">
        <div class="stat-card" style="background: white; padding: 1.5rem; border-radius: 16px; box-shadow: 0 4px 6px rgba(0,0,0,0.02); border: 1px solid #f1f5f9;">
            <div style="display: flex; align-items: center; gap: 1rem; margin-bottom: 1rem;">
                <div style="width: 44px; height: 44px; background: #eef2ff; border-radius: 12px; display: flex; align-items: center; justify-content: center; color: var
                <h3 style="margin: 0; font-size: 1rem; color: #64748b;">Enrolled Courses</h3>
            </div>
            <p style="font-size: 2.25rem; font-weight: 800; color: var(--dark); margin: 0;">{{ enrolled_courses|length }}</p>
        </div>

        <div class="stat-card" style="background: white; padding: 1.5rem; border-radius: 16px; box-shadow: 0 4px 6px rgba(0,0,0,0.02); border: 1px solid #f1f5f9;">
            <div style="display: flex; align-items: center; gap: 1rem; margin-bottom: 1rem;">
                <div style="width: 44px; height: 44px; background: #f0fdf4; border-radius: 12px; display: flex; align-items: center; justify-content: center; color: var
                <h3 style="margin: 0; font-size: 1rem; color: #64748b;">Completed Modules</h3>
            </div>
            <p style="font-size: 2.25rem; font-weight: 800; color: var(--success); margin: 0;">{{ total_completed_modules }}</p>
        </div>
    </div>

    <!-- My Courses Section -->
    <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 1.5rem;">
        <h2 style="font-size: 1.5rem; font-weight: 800; color: var(--dark); margin: 0;">Continue Learning</h2>
        <a href="{% url 'my_courses' %}" style="color: var(--primary); text-decoration: none; font-weight: 600; font-size: 0.9rem;">View All Courses</a>
    </div>

    <div class="courses-list" style="display: grid; grid-template-columns: repeat(auto-fill, minmax(320px, 1fr)); gap: 1.5rem; margin-bottom: 4rem;">
        {% for course in enrolled_courses %}
        <div class="course-card" style="background: white; border-radius: 20px; overflow: hidden; box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.05); border: 1px solid
```


26-Day Work Plan
Branch · 26-Day Work Plan
Adaptive Learning MS
127.0.0.1:8000/dashboard/
Adaptive
Dashboard
My Courses
Explore
Certificates
dhruv
Welcome back, dhruv!
Track your progress and continue where you left off.
Enrolled Courses
3
Completed Modules
6
Continue Learning
View All Courses
PYTHON
python
Progress
100%
JAVA
Java
Progress
100%
C++
C++
Progress
100%


26-Day Work Plan
Branch · 26-Day Work Plan
Adaptive Learning MS
127.0.0.1:8000/dashboard/
Adaptive
Dashboard
My Courses
Explore
Certificates
arpit
Enrolled Courses
0
Completed Modules
0
Continue Learning
View All Courses
You aren't enrolled in any courses yet.
Explore Courses
Recommended For You
python
Enrol Now →
Java
Enrol Now →
C++
Enrol Now →
© 2026 alms - Adaptive Learning Management System.

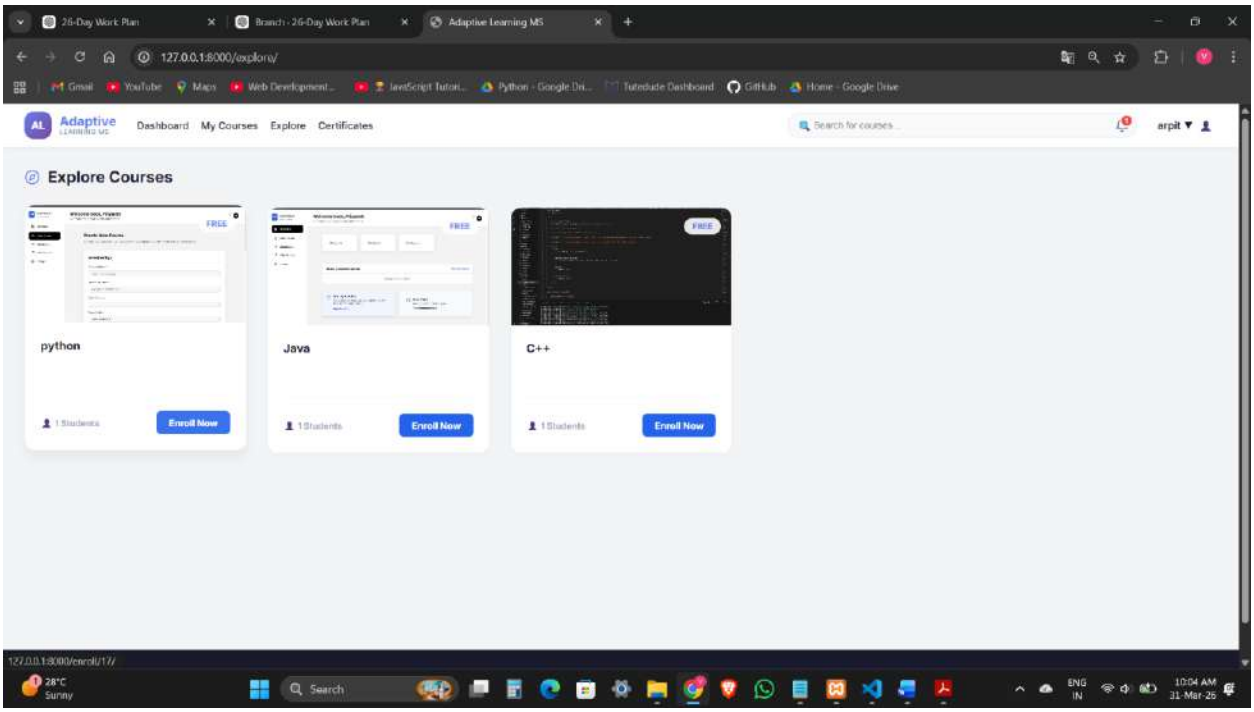


Day 4 :- Implement course enrollment functionality

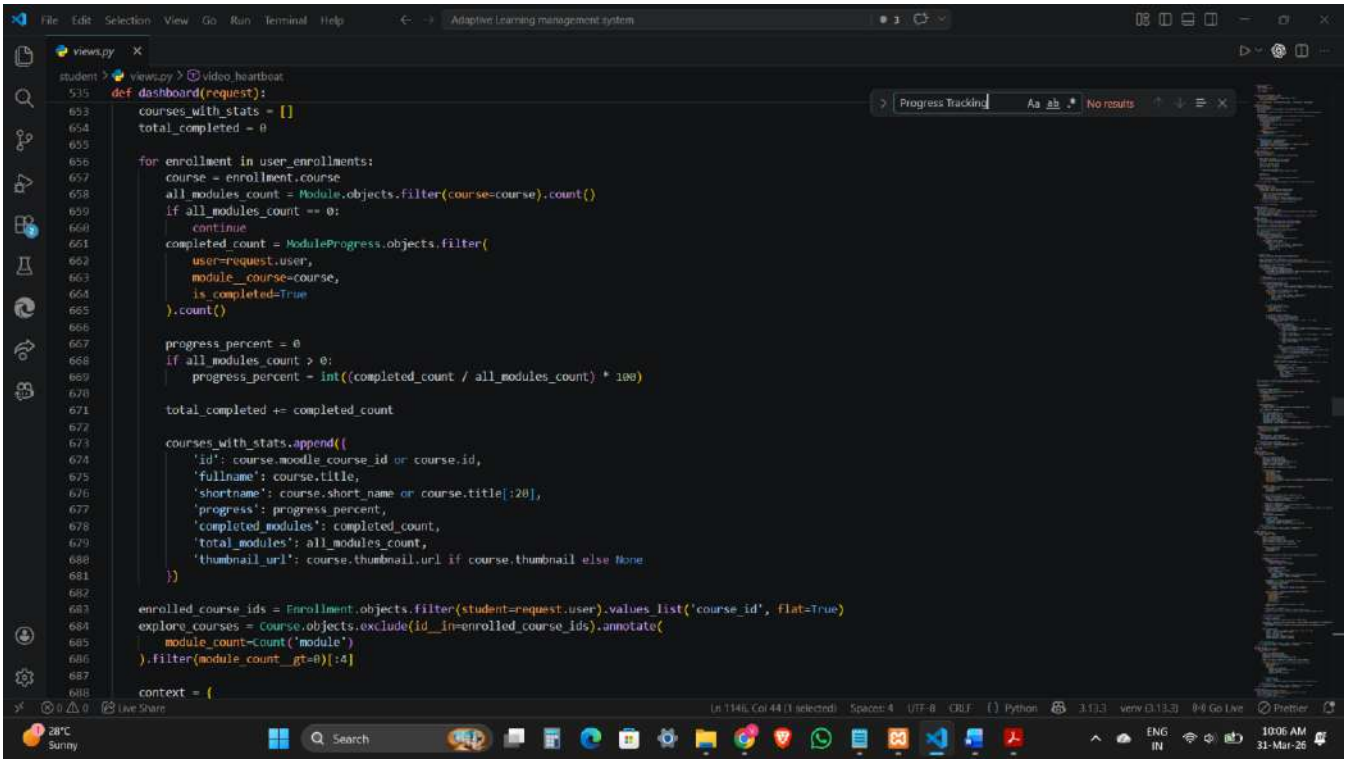


```
def explore_view(request):
        module_count=Count('module')
    ).filter(
        module_count__gt=0,
        is_active=True,
    ).distinct()

    return render(request, 'student/explore.html', {
        'courses': courses,
        'enrolled_course_ids': enrolled_course_ids,
        'query': query
    })
@student_required
def enroll_course(request, course_id):
    course = get_object_or_404(Course, id=course_id)

    enrollment, created = Enrollment.objects.get_or_create(
        student=request.user,
        course=course,
        defaults={'is_paid': True}
    )

    if created:
        # Sync to Moodle only on new enrollment
        profile = getattr(request.user, 'studentprofile', None)
        if profile and profile.moodle_user_id and course.moodle_course_id:
            from .moodle_sync import enrol_user_in_course
            enrol_user_in_course(profile.moodle_user_id, course.moodle_course_id)

    return redirect('my_courses')


from django.views.decorators.csrf import csrf_exempt
import json

@login_required
```

```python
from django.views.decorators.csrf import csrf_exempt
import json

@login_required
@csrf_exempt
def video_heartbeat(request):
    if request.method == 'POST':
        try:
            data = json.loads(request.body)
            percent = float(data.get('percent', 0.0))
            seconds = float(data.get('current_time', 0.0))
            module_id = data.get('module_id')

            module = get_object_or_404(Module, id=module_id, type='video')
            progress, _ = ModuleProgress.objects.get_or_create(
                user=request.user,
                module=module
            )

            # update progress
            progress.video_progress = max(progress.video_progress, percent)
            progress.save()

            # Sync completion state
            _sync_completion_state(progress)

            return JsonResponse({'status': 'ok'})
        except Exception as e:
            return JsonResponse({'status': 'error', 'message': str(e)}, status=400)
    return JsonResponse({'status': 'error', 'message': 'Invalid method'}, status=405)

@login_required
@csrf_exempt
def video_replay(request):
    if request.method == 'POST':
        try:
```

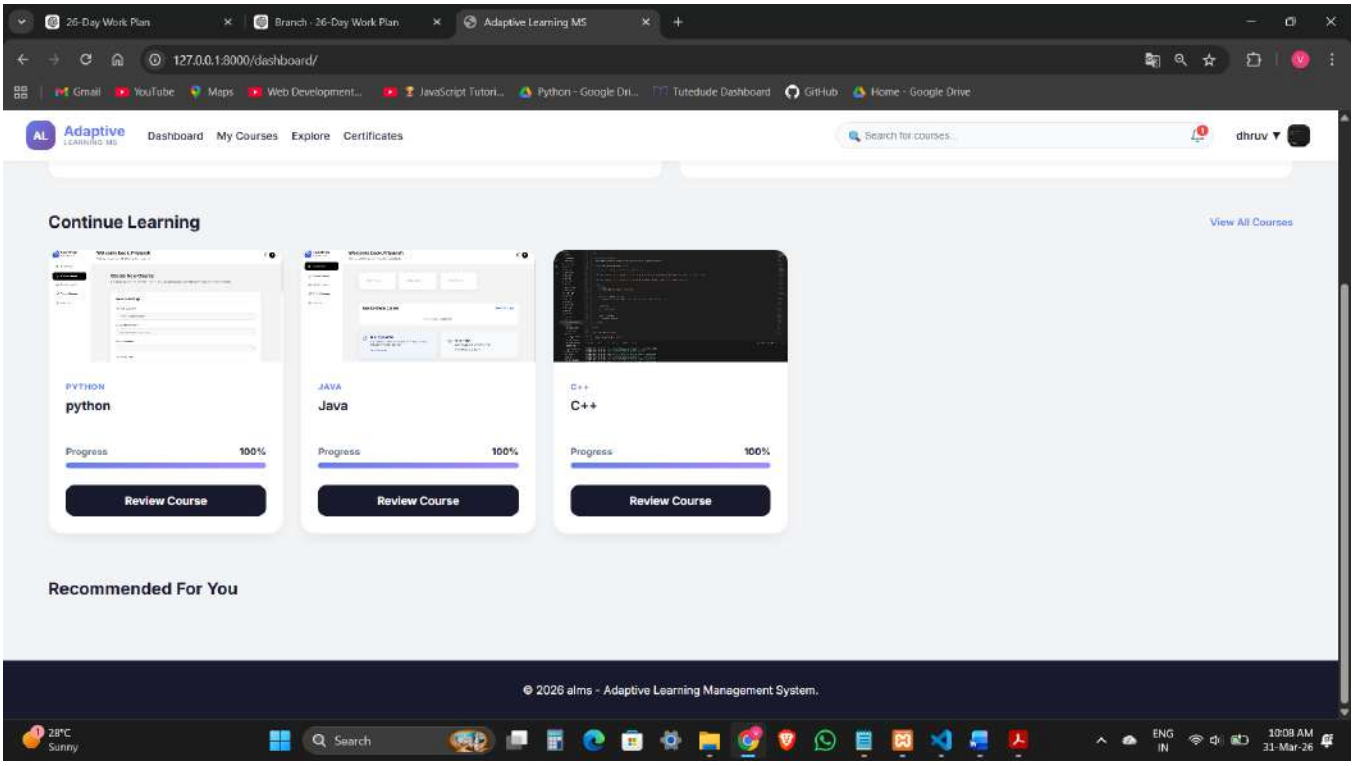


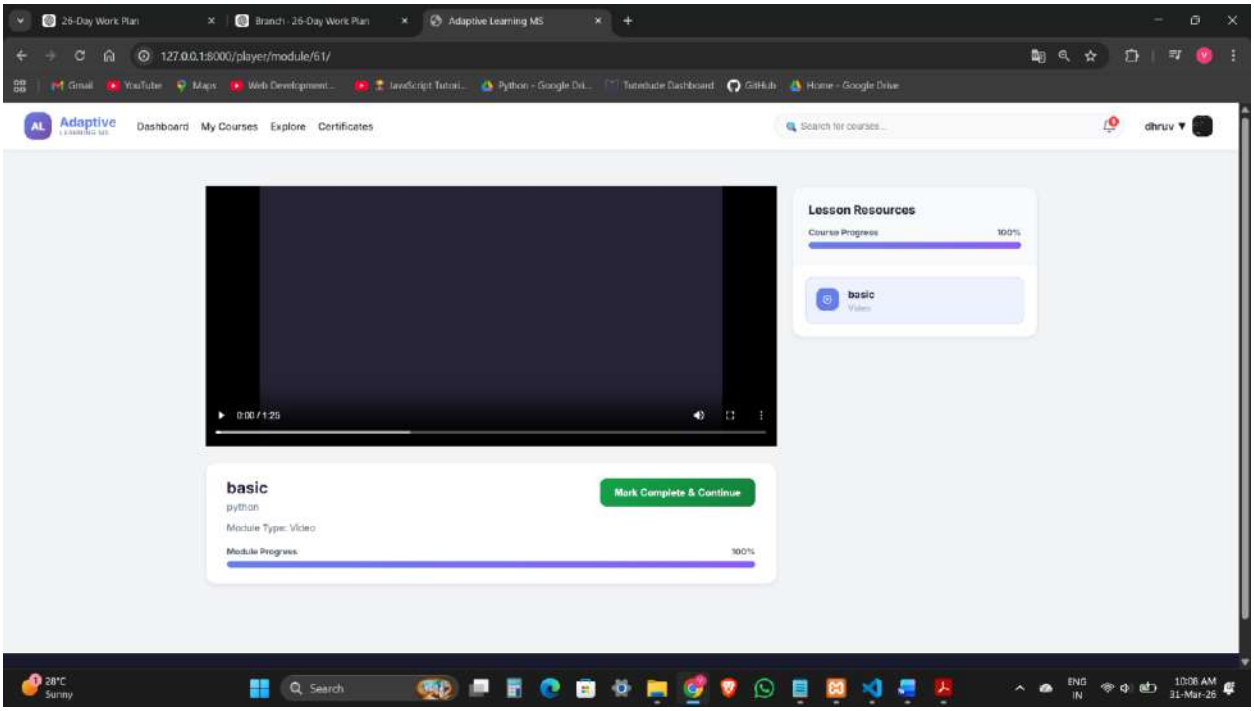
Day 5 :- Display course progress (percentage + module status)



```python
def dashboard(request):
    courses_with_stats = []
    total_completed = 0

    for enrollment in user_enrollments:
        course = enrollment.course
        all_modules_count = Module.objects.filter(course=course).count()
        if all_modules_count == 0:
            continue
        completed_count = ModuleProgress.objects.filter(
            user=request.user,
            module__course=course,
            is_completed=True
        ).count()

        progress_percent = 0
        if all_modules_count > 0:
            progress_percent = int((completed_count / all_modules_count) * 100)

        total_completed += completed_count

        courses_with_stats.append({
            'id': course.moodle_course_id or course.id,
            'fullname': course.title,
            'shortname': course.short_name or course.title[:20],
            'progress': progress_percent,
            'completed_modules': completed_count,
            'total_modules': all_modules_count,
            'thumbnail_url': course.thumbnail.url if course.thumbnail else None
        })

    enrolled_course_ids = Enrollment.objects.filter(student=request.user).values_list('course_id', flat=True)
    explore_courses = Course.objects.exclude(id__in=enrolled_course_ids).annotate(
        module_count=Count('module')
    ).filter(module_count__gt=0)[:4]

    context = {
```

```python
def watch_event_api(request):

            # Update progress
            progress, created = StudentProgress.objects.get_or_create(
                student=request.user,
                course=module.course,
                module=module
            )

            # Calculate watch percentage (simple estimation for now)
            if module.duration_seconds > 0:
                percent = (current_time / module.duration_seconds) * 100
                if percent > progress.watch_percent:
                    progress.watch_percent = min(percent, 100)

            if event_type == 'checkpoint':
                progress.checkpoints_completed = progress.checkpoints_completed + 1
            if event_type in ['complete', 'replay']:
                progress.replay_count = max(progress.replay_count, 1) if event_type == 'complete' else progress.replay_count + 1
            if metadata.get('theory_completed'):
                progress.theory_completed = True

            progress.save()
            _sync_completion_state(progress)

            return JsonResponse({
                'status': 'success',
                'watch_percent': round(progress.watch_percent, 2),
                'is_completed': progress.is_completed,
                'checkpoints_completed': progress.checkpoints_completed,
            })
        except Exception as e:
            return JsonResponse({'status': 'error', 'message': str(e)}, status=400)
    return JsonResponse({'status': 'invalid_method'}, status=405)

@student_required
def quiz_submit_api(request):
```





- **Week 8 Plan :-** Integrate video player with controlled learning features and progress tracking.

Total Working Hrs: 35 hours **Signature of Student:**

The above entries are correct and the grading of work done by Trainee is

Excellent	Very Good	Good	Fair	Below Average	Poor

Signature of External Guide with Company Seal: **Signature of Internal Guide**

Date: **Date:**

SAL EDUCATION

SAL ENGINEERING AND TECHNICAL INSTITUTE

Opp. Science city, Sola-Bhadaj Road, Ahmedabad, Gujarat 380060
Ph:- 079 67129000 Website: www.sal.edu.in

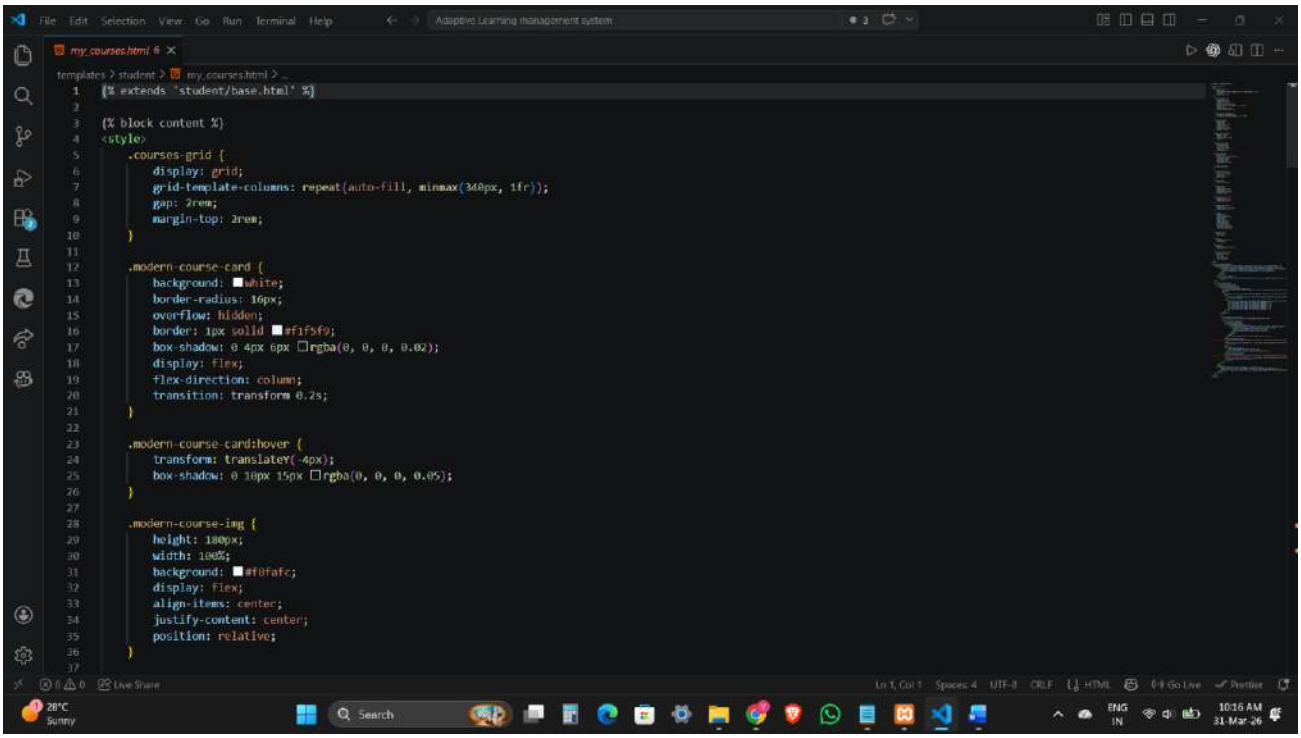
Student's Weekly Report of Internship

Student Name:	Vekariya Vraj Dipakbhai			
Enrollment Number:	221260116056			
Name Of Organization:	Bisag N			
External Guide Name:	Parth Sutariya			
External Guide Contact details:	**Email:**	Parthsutariya12@gmail.com	**Mob No:**	81558 51038
Internal Faculty Guide Name:	Prof. Amitkumar J. Patel			

Week 8 (23/02/2026 – 01/03/2026)

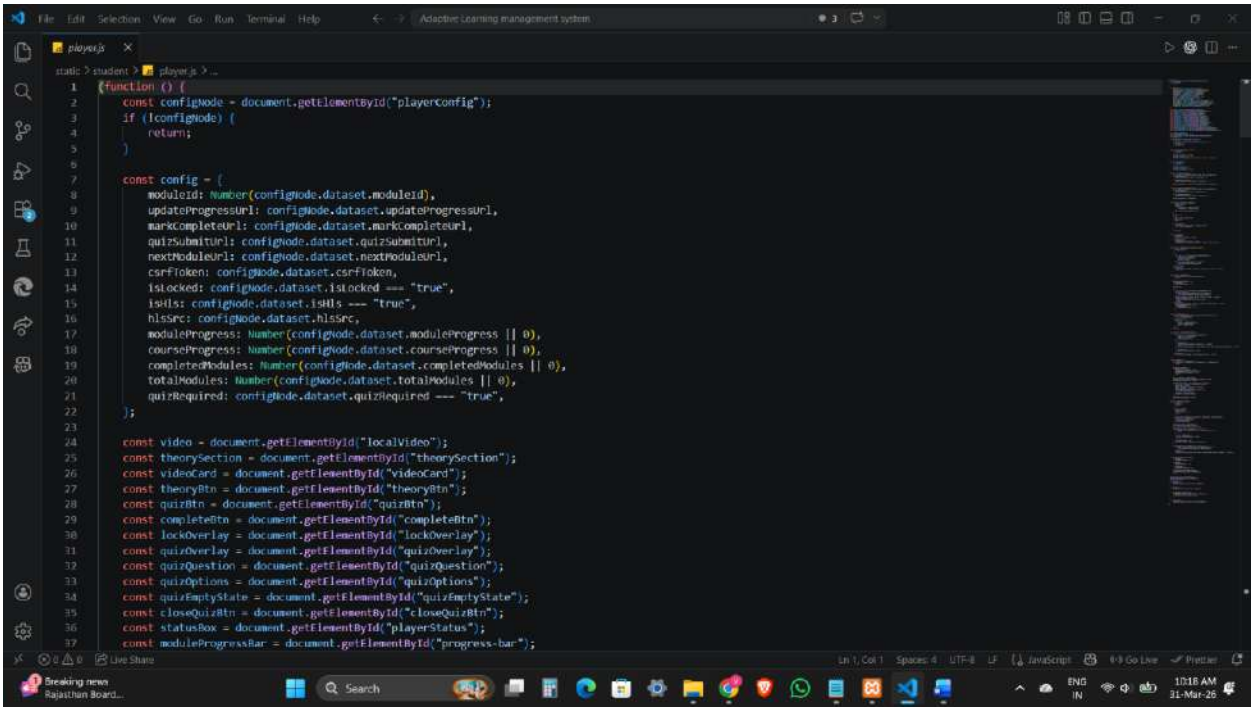
1. Integrate video player in student panel
2. Implement forward-seek restriction (anti-skip learning control)
3. Add checkpoint validation during video playback
4. Store video watch history and progress in backend

Day 1 :- Integrate video player in student panel



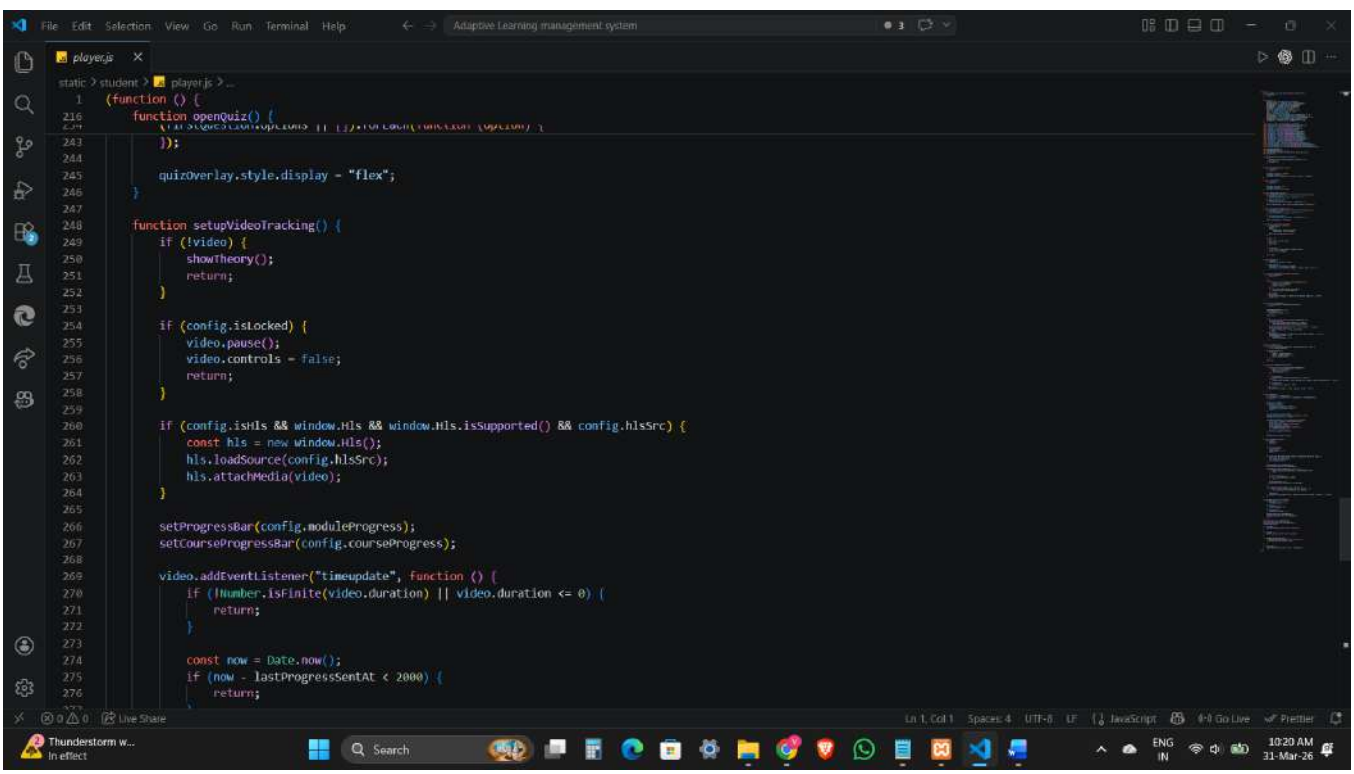
```html
{% extends 'student/base.html' %}
{% load static %}

{% block extra_css %}
<style>
    .player-main-wrapper {
        margin: 1.5rem auto 0;
        max-width: 1280px;
        padding: 0 1.5rem;
    }

    .player-grid {
        display: grid;
        grid-template-columns: 2.8fr 1.2fr;
        gap: 1.5rem;
        align-items: start;
    }

    .viewport-column {
        display: flex;
        flex-direction: column;
        gap: 1.25rem;
    }

    .video-card,
    .course-info-card,
    .sidebar-card,
    #theorySection {
        background: #ffffff;
        border: 1px solid #e2e8f0;
        border-radius: 16px;
        box-shadow: 0 4px 6px rgba(15, 23, 42, 0.05);
    }

    .video-card {
        background: #000;
        overflow: hidden;
```

```html
{% if dash_manifest %}
<script src="https://cdn.dashjs.org/latest/dash.all.min.js"></script>
{% endif %}
{% if is_hls %}
<script src="https://cdn.jsdelivr.net/npm/hls.js@latest"></script>
{% endif %}
<script src="{% static 'student/quiz_player.js' %}"></script>

<script>
    function toggleFeedback() {
        const section = document.getElementById('feedbackSection');
        if (section) {
            section.style.display = section.style.display === 'none' ? 'block' : 'none';
        }
    }

    async function submitAssignment() {
        const moduleId = "{{ module.id }}";
        const githubLink = document.getElementById('githubLink').value;
        const googleDriveLink = document.getElementById('googleDriveLink').value;
        const btn = document.getElementById('submitAssignmentBtn');
        const statusDiv = document.getElementById('playerStatus');

        if (!githubLink && !googleDriveLink) {
            alert('Please provide at least one link (GitHub or Google Drive).');
            return;
        }

        if (btn) { btn.disabled = true; btn.innerText = 'Submitting...'; }

        try {
            const response = await fetch("{% url 'submit_assignment_api' %}", {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json',
                    'X-CSRFToken': "{{ csrf_token }}"
```

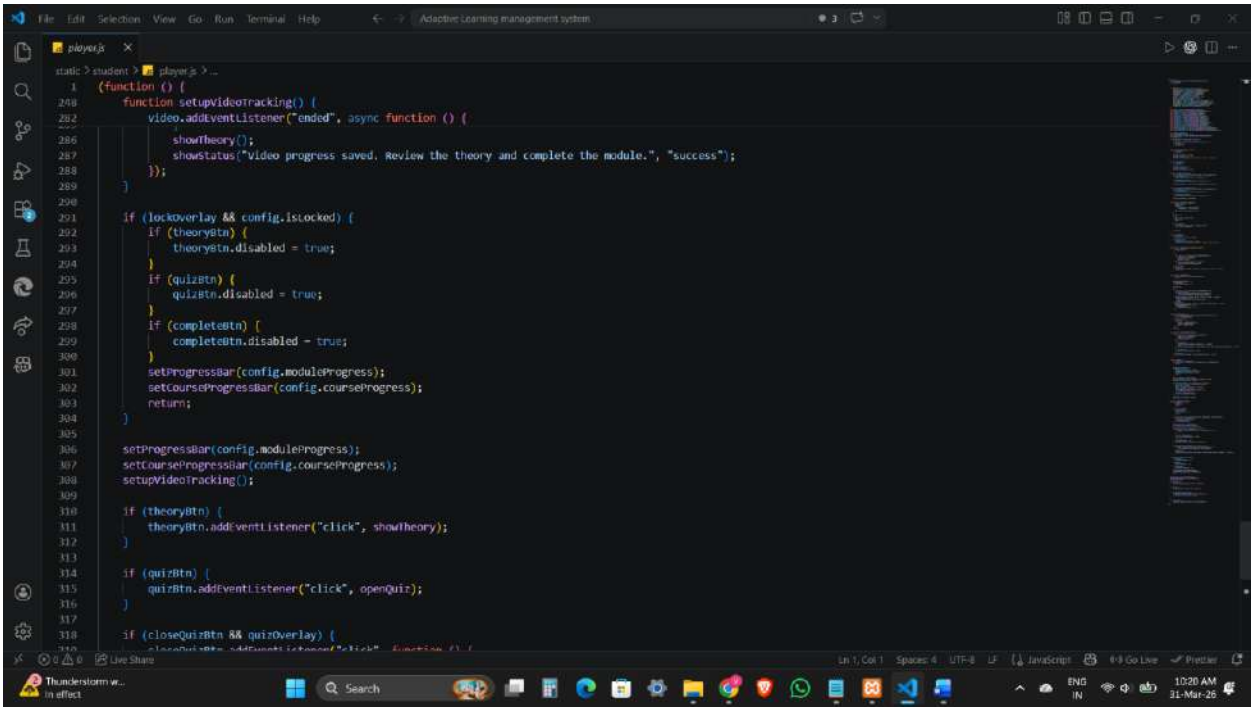


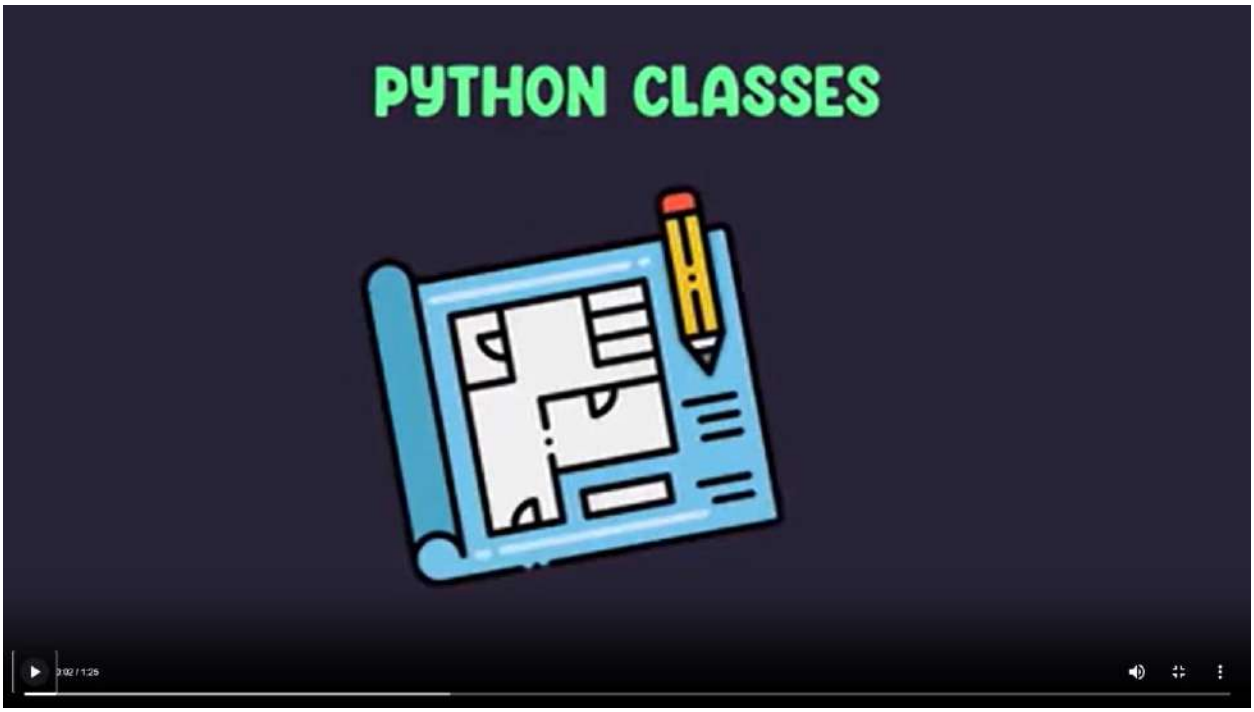
```javascript
(function () {
    const configNode = document.getElementById("playerConfig");
    if (!configNode) {
        return;
    }

    const config = {
        moduleId: Number(configNode.dataset.moduleId),
        updateProgressUrl: configNode.dataset.updateProgressUrl,
        markCompleteUrl: configNode.dataset.markCompleteUrl,
        quizSubmitUrl: configNode.dataset.quizSubmitUrl,
        nextModuleUrl: configNode.dataset.nextModuleUrl,
        csrfToken: configNode.dataset.csrfToken,
        isLocked: configNode.dataset.isLocked === "true",
        isHls: configNode.dataset.isHls === "true",
        hlsSrc: configNode.dataset.hlsSrc,
        moduleProgress: Number(configNode.dataset.moduleProgress || 0),
        courseProgress: Number(configNode.dataset.courseProgress || 0),
        completedModules: Number(configNode.dataset.completedModules || 0),
        totalModules: Number(configNode.dataset.totalModules || 0),
        quizRequired: configNode.dataset.quizRequired === "true",
    };

    const video = document.getElementById("localVideo");
    const theorySection = document.getElementById("theorySection");
    const videoCard = document.getElementById("videoCard");
    const theoryBtn = document.getElementById("theoryBtn");
    const quizBtn = document.getElementById("quizBtn");
    const completeBtn = document.getElementById("completeBtn");
    const lockOverlay = document.getElementById("lockOverlay");
    const quizOverlay = document.getElementById("quizOverlay");
    const quizQuestion = document.getElementById("quizQuestion");
    const quizOptions = document.getElementById("quizOptions");
    const quizEmptyState = document.getElementById("quizEmptyState");
    const closeQuizBtn = document.getElementById("closeQuizBtn");
    const statusBox = document.getElementById("playerStatus");
    const moduleProgressBar = document.getElementById("progress-bar");
```

Day 2 :- Implement forward-seek restriction (anti-skip learning control)

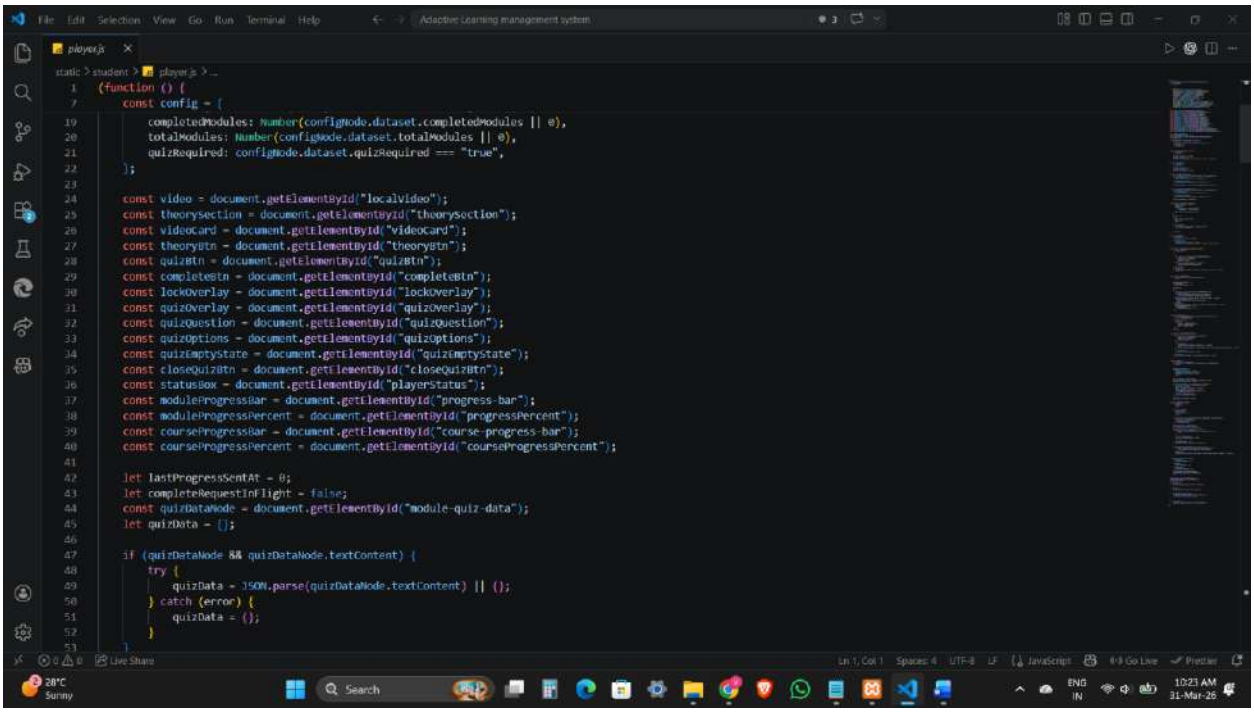


```javascript
(function () {
    function openQuiz() {
        });

        quizOverlay.style.display = "flex";
    }

    function setupVideoTracking() {
        if (!video) {
            showTheory();
            return;
        }

        if (config.isLocked) {
            video.pause();
            video.controls = false;
            return;
        }

        if (config.isHls && window.Hls && window.Hls.isSupported() && config.hlsSrc) {
            const hls = new window.Hls();
            hls.loadSource(config.hlsSrc);
            hls.attachMedia(video);
        }

        setProgressBar(config.moduleProgress);
        setCourseProgressBar(config.courseProgress);

        video.addEventListener("timeupdate", function () {
            if (!Number.isFinite(video.duration) || video.duration <= 0) {
                return;
            }

            const now = Date.now();
            if (now - lastProgressSentAt < 2000) {
                return;
```

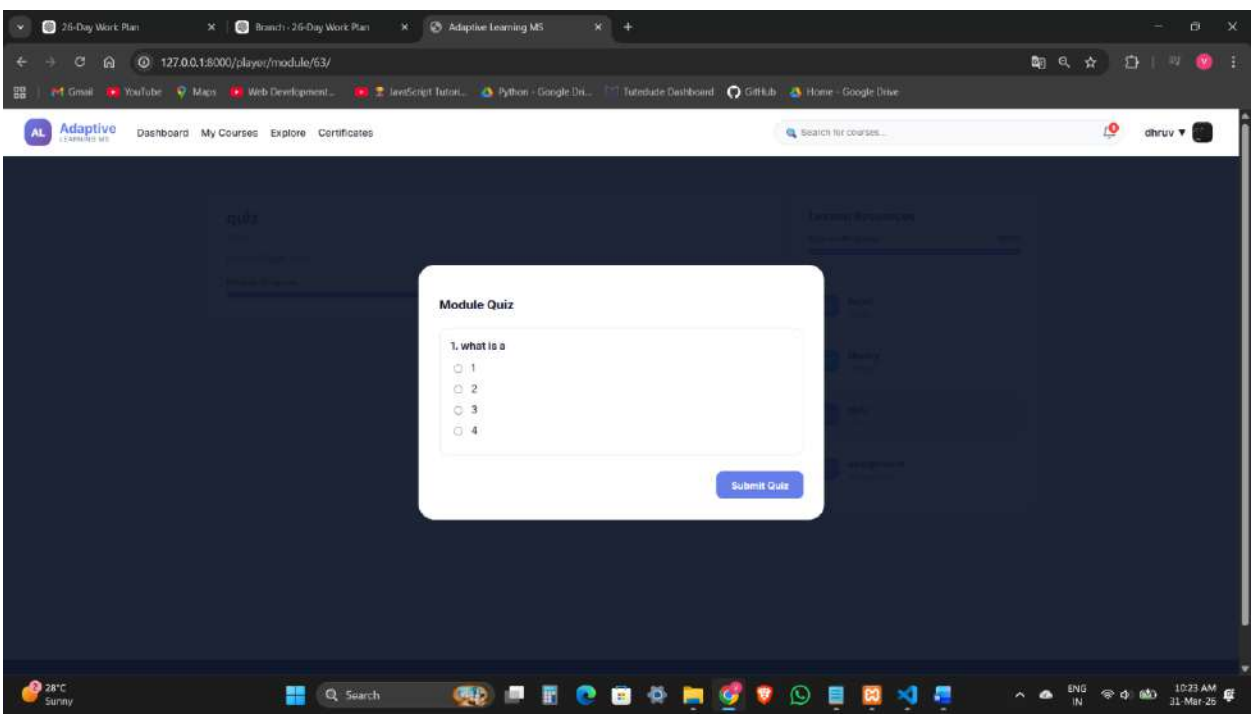

File Edit Selection View Go Run Terminal Help
Adaptive Learning management system
player.js
static > student > player.js > ...
(function () {
function setupVideoTracking() {
video.addEventListener("ended", async function () {
showTheory();
showStatus("Video progress saved. Review the theory and complete the module.", "success");
});
}
if (lockOverlay && config.isLocked) {
if (theoryBtn) {
theoryBtn.disabled = true;
}
if (quizBtn) {
quizBtn.disabled = true;
}
if (completeBtn) {
completeBtn.disabled = true;
}
setProgressBar(config.moduleProgress);
setCourseProgressBar(config.courseProgress);
return;
}
setProgressBar(config.moduleProgress);
setCourseProgressBar(config.courseProgress);
setupVideoTracking();
if (theoryBtn) {
theoryBtn.addEventListener("click", showTheory);
}
if (quizBtn) {
quizBtn.addEventListener("click", openQuiz);
}
if (closeQuizBtn && quizOverlay) {
Live Share
Ln 1, Col 1
Spaces: 4
UTF-8
LF
JavaScript
Go Live
Prettier
Thunderstorm w...
In effect
Search
ENG
IN
10:20 AM
31-Mar-26


PYTHON CLASSES
0:02 / 1:25

Day 3 :- Add checkpoint validation during video playback



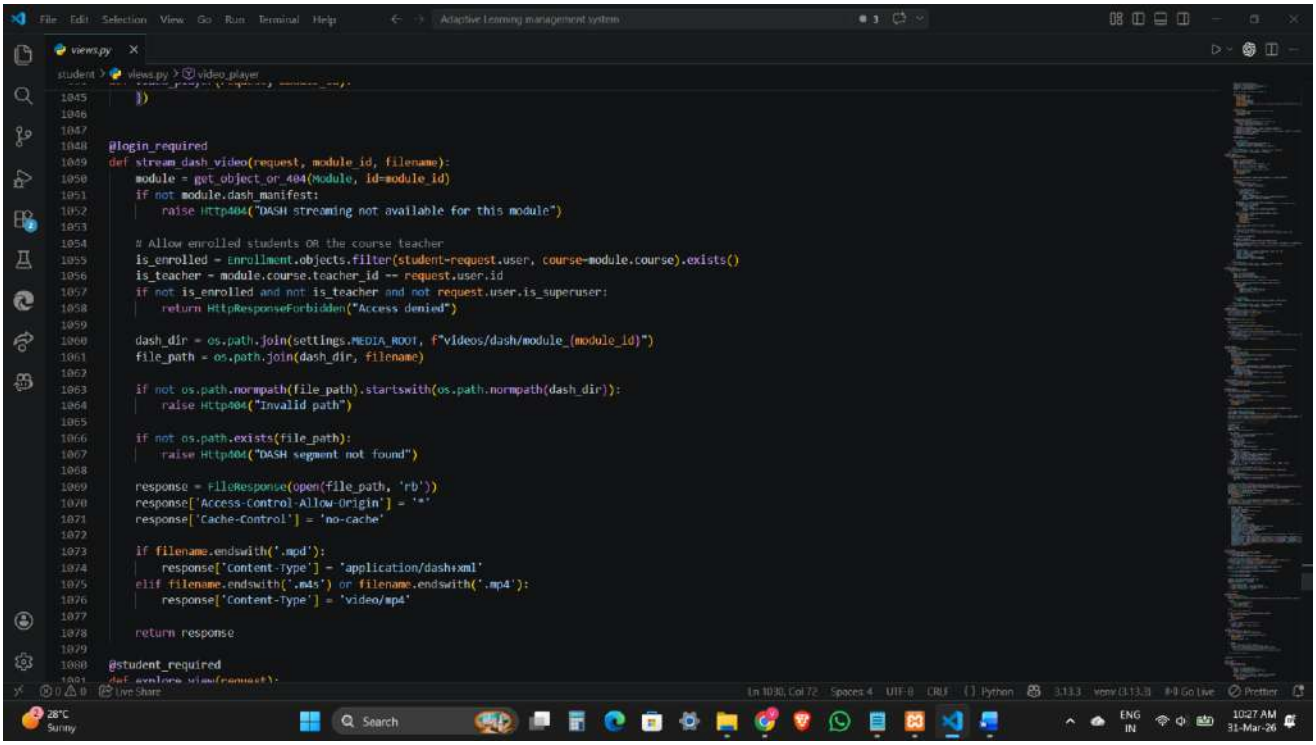
```
(function () {
    const config = {
        completedModules: Number(configNode.dataset.completedModules || 0),
        totalModules: Number(configNode.dataset.totalModules || 0),
        quizRequired: configNode.dataset.quizRequired === "true",
    };

    const video = document.getElementById("localVideo");
    const theorySection = document.getElementById("theorySection");
    const videoCard = document.getElementById("videoCard");
    const theoryBtn = document.getElementById("theoryBtn");
    const quizBtn = document.getElementById("quizBtn");
    const completeBtn = document.getElementById("completeBtn");
    const lockOverlay = document.getElementById("lockOverlay");
    const quizOverlay = document.getElementById("quizOverlay");
    const quizQuestion = document.getElementById("quizQuestion");
    const quizOptions = document.getElementById("quizOptions");
    const quizEmptyState = document.getElementById("quizEmptyState");
    const closeQuizBtn = document.getElementById("closeQuizBtn");
    const statusBox = document.getElementById("playerStatus");
    const moduleProgressBar = document.getElementById("progress-bar");
    const moduleProgressPercent = document.getElementById("progressPercent");
    const courseProgressBar = document.getElementById("course-progress-bar");
    const courseProgressPercent = document.getElementById("courseProgressPercent");

    let lastProgressSentAt = 0;
    let completeRequestInFlight = false;
    const quizDataNode = document.getElementById("module-quiz-data");
    let quizData = [];

    if (quizDataNode && quizDataNode.textContent) {
        try {
            quizData = JSON.parse(quizDataNode.textContent) || [];
        } catch (error) {
            quizData = [];
        }
    }
```



Day 4 :- Store video watch history and progress in backend

```python
def video_player(request, module_id):
    next_module = Module.objects.filter(course=module.course, order__gt=module.order).order_by('order').first()

    return render(request, 'student/video_player.html', {
        'module': module,
        'all_modules': all_modules,
        'last_sequence_number': last_sequence_number,
        'is_youtube': is_youtube,
        'is_hls': is_hls,
        'is_external_page': is_external_page,
        'embed_url': embed_url,
        'moodle_module': moodle_module,
        'course_modules': course_modules,
        'is_locked': is_locked,
        'lock_reason': lock_reason,
        'progress': progress,
        'module_progress': module_progress,
        'module_progress_percent': int(module_progress.video_progress),
        'attempts_remaining': attempts_remaining,
        'course_progress_percent': course_progress_percent,
        'completed_modules_count': completed_modules,
        'total_modules_count': total_modules,
        'quiz_payload': _quiz_payload(quiz),
        'next_module_url': reverse('video_player', kwargs={'module_id': next_module.id}) if next_module else reverse('dashboard'),
        'allow_seek': True,  # Default for new model
        'disable_fast_forward': module.video_details.disable_fast_forward if hasattr(module, 'video_details') and module.video_details else True,
        'enable_checkpoints': module.video_details.enable_checkpoints if hasattr(module, 'video_details') and module.video_details else False,
        'checkpoint_interval': module.video_details.checkpoint_interval if hasattr(module, 'video_details') and module.video_details else 30,
        'checkpoint_interval': module.video_details.checkpoint_interval if hasattr(module, 'video_details') and module.video_details else 30,
        'min_watch_percent': module.video_details.min_watch_percent if hasattr(module, 'video_details') and module.video_details else 80,
        'dash_manifest': dash_manifest,
        'submission': submission,
    })


@login_required
def stream_dash_video(request, module_id, filename):
```

```python
from django.views.decorators.csrf import csrf_exempt
import json

@login_required
@csrf_exempt
def video_heartbeat(request):
    if request.method == 'POST':
        try:
            data = json.loads(request.body)
            percent = float(data.get('percent', 0.0))
            seconds = float(data.get('current_time', 0.0))
            module_id = data.get('module_id')

            module = get_object_or_404(Module, id=module_id, type='video')
            progress, _ = ModuleProgress.objects.get_or_create(
                user=request.user,
                module=module
            )

            # Update progress
            progress.video_progress = max(progress.video_progress, percent)
            progress.save()

            # Sync completion state
            _sync_completion_state(progress)

            return JsonResponse({'status': 'ok'})
        except Exception as e:
            return JsonResponse({'status': 'error', 'message': str(e)}, status=400)
    return JsonResponse({'status': 'error', 'message': 'Invalid method'}, status=405)

@login_required
@csrf_exempt
def video_replay(request):
```



```python
    })


@login_required
def stream_dash_video(request, module_id, filename):
    module = get_object_or_404(Module, id=module_id)
    if not module.dash_manifest:
        raise Http404("DASH streaming not available for this module")

    # Allow enrolled students OR the course teacher
    is_enrolled = Enrollment.objects.filter(student=request.user, course=module.course).exists()
    is_teacher = module.course.teacher_id == request.user.id
    if not is_enrolled and not is_teacher and not request.user.is_superuser:
        return HttpResponseForbidden("Access denied")

    dash_dir = os.path.join(settings.MEDIA_ROOT, f"videos/dash/module_{module_id}")
    file_path = os.path.join(dash_dir, filename)

    if not os.path.normpath(file_path).startswith(os.path.normpath(dash_dir)):
        raise Http404("Invalid path")

    if not os.path.exists(file_path):
        raise Http404("DASH segment not found")

    response = FileResponse(open(file_path, 'rb'))
    response['Access-Control-Allow-Origin'] = '*'
    response['Cache-Control'] = 'no-cache'

    if filename.endswith('.mpd'):
        response['Content-Type'] = 'application/dash+xml'
    elif filename.endswith('.m4s') or filename.endswith('.mp4'):
        response['Content-Type'] = 'video/mp4'

    return response

@student_required
```

- **Week 9 Plan :-** Implement quiz module system with backend validation and result tracking.

Total Working Hrs: 28 hours **Signature of Student:**

The above entries are correct and the grading of work done by Trainee is

Excellent	Very Good	Good	Fair	Below Average	Poor

Signature of External Guide with Company Seal: **Signature of Internal Guide**

Date: **Date:**

SAL EDUCATION

SAL ENGINEERING AND TECHNICAL INSTITUTE

Opp. Science city, Sola-Bhadaj Road, Ahmedabad, Gujarat 380060
Ph:- 079 67129000 Website: www.sal.edu.in

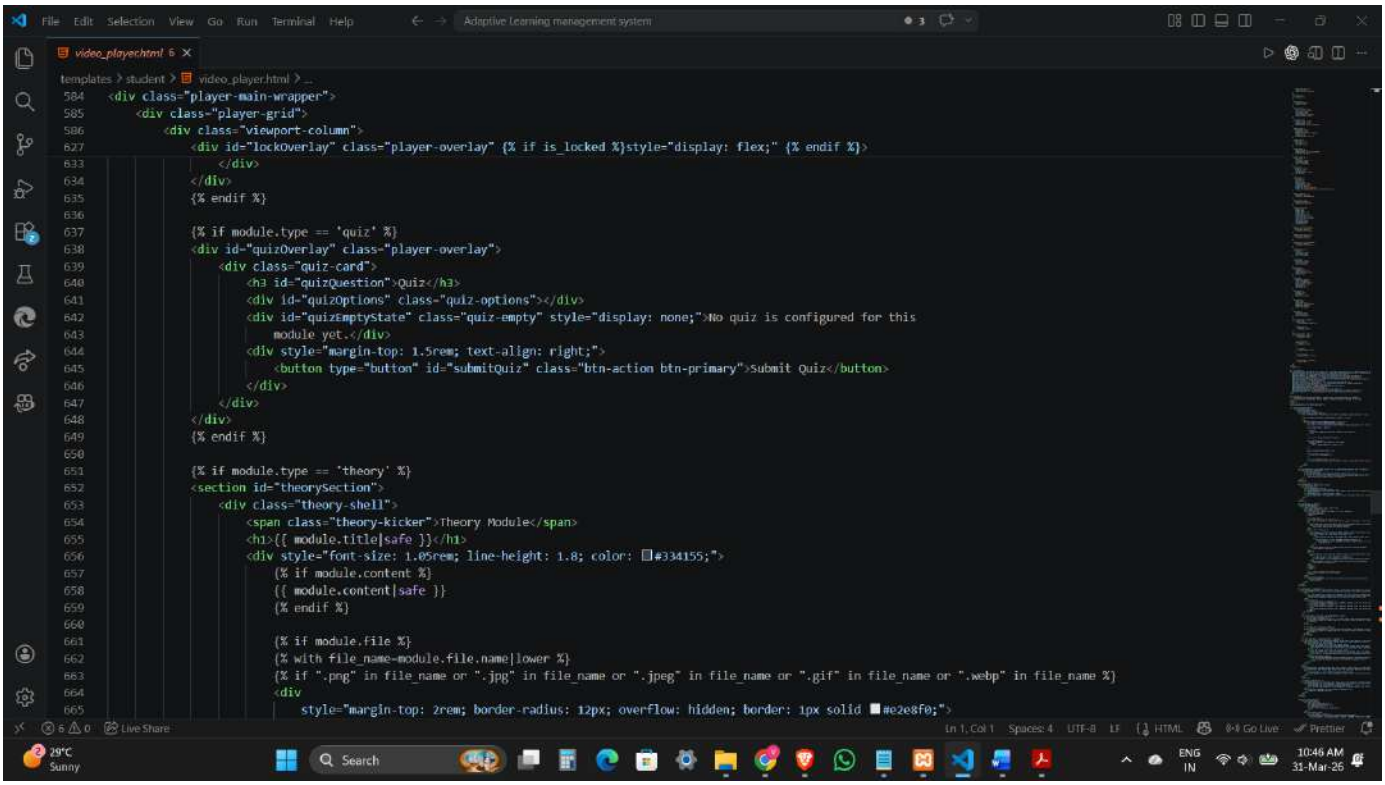
Student's Weekly Report of Internship

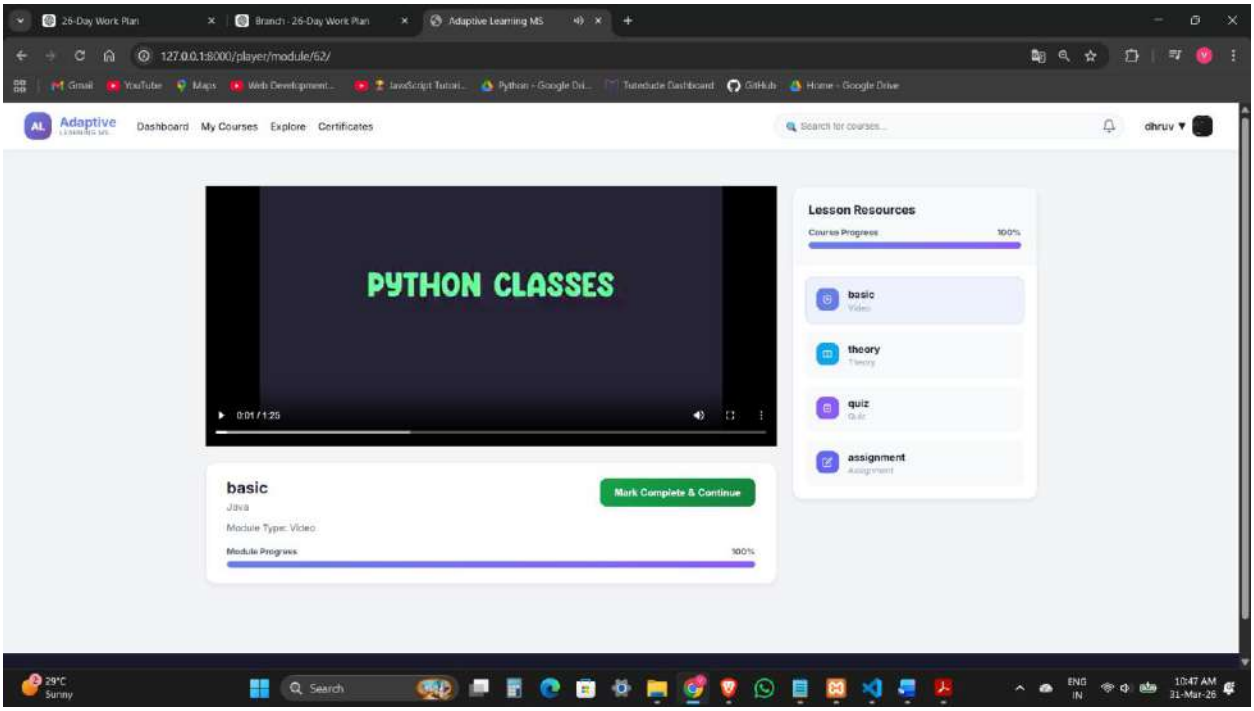
Student Name:	Vekariya Vraj Dipakbhai			
Enrollment Number:	221260116056			
Name Of Organization:	Bisag N			
External Guide Name:	Parth Sutariya			
External Guide Contact details:	**Email:**	Parthsutariya12@gmail.com	**Mob No:**	81558 51038
Internal Faculty Guide Name:	Prof. Amitkumar J. Patel			

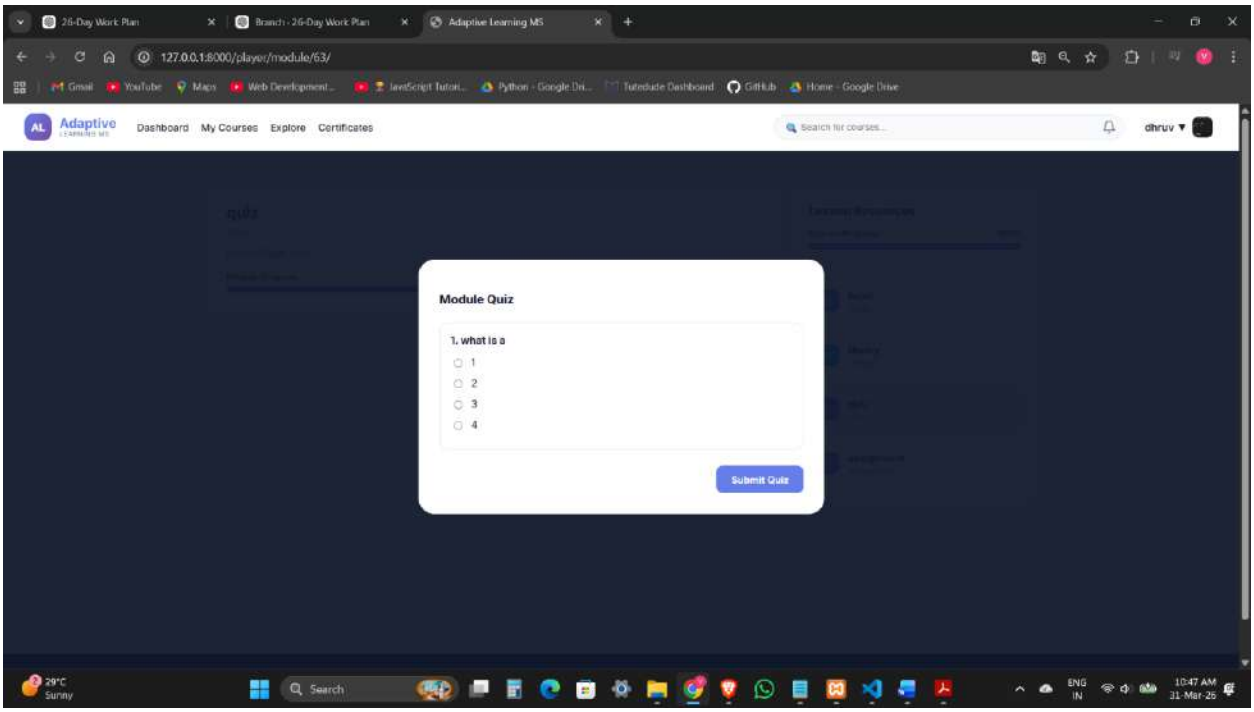
Week 9 (02/03/2026 – 08/03/2026)

1. Implement quiz module interface (separate from video player)
2. Create backend API for quiz answer validation
3. Restrict module completion based on quiz performance
4. Store quiz attempts and results for progress tracking

Day 1 :- Implement quiz module interface (separate from video player)




26-Day Work Plan
Branch · 26-Day Work Plan
Adaptive Learning MS
127.0.0.1:8000/player/module/62/
Adaptive
Dashboard
My Courses
Explore
Certificates
dhruv
PYTHON CLASSES
0:01 / 1:25
Lesson Resources
Course Progress
100%
basic
theory
quiz
assignment
basic
Java
Module Type: Video
Mark Complete & Continue
Module Progress
100%

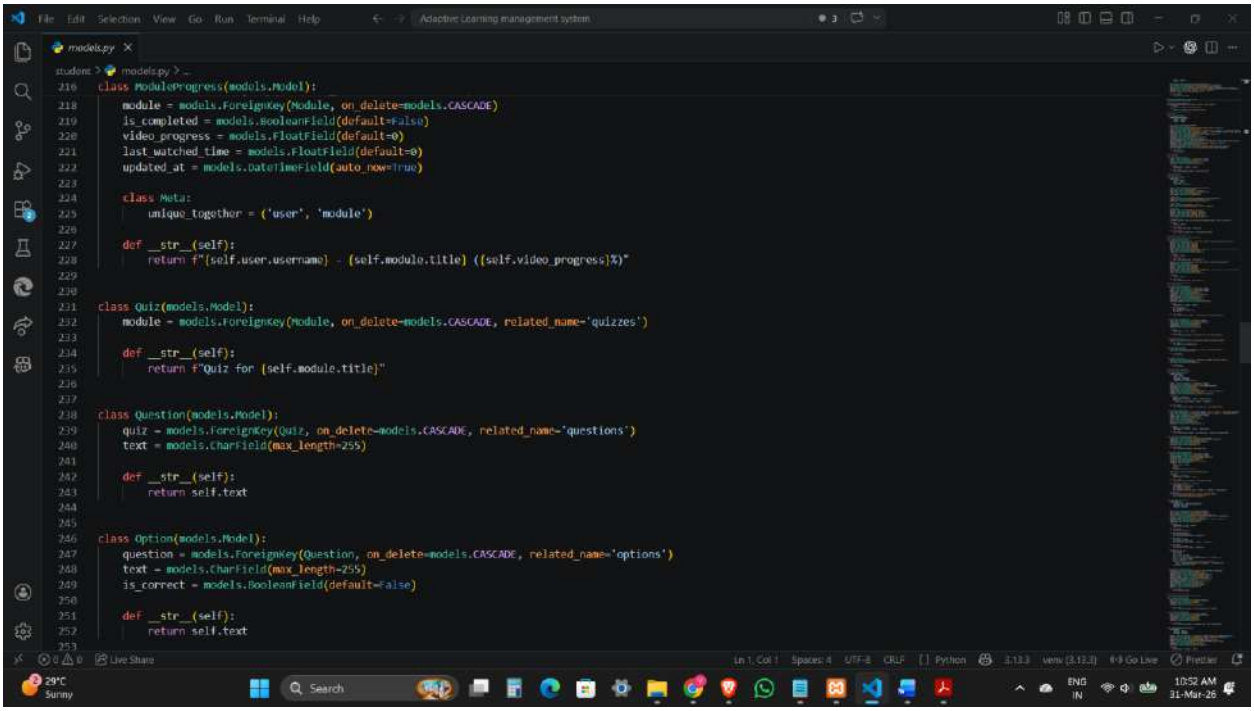

26-Day Work Plan
Branch · 26-Day Work Plan
Adaptive Learning MS
127.0.0.1:8000/player/module/63/
Adaptive
Dashboard
My Courses
Explore
Certificates
dhruv
Module Quiz
1. what is a
1
2
3
4
Submit Quiz

Day 2 :- Create backend API for quiz answer validation

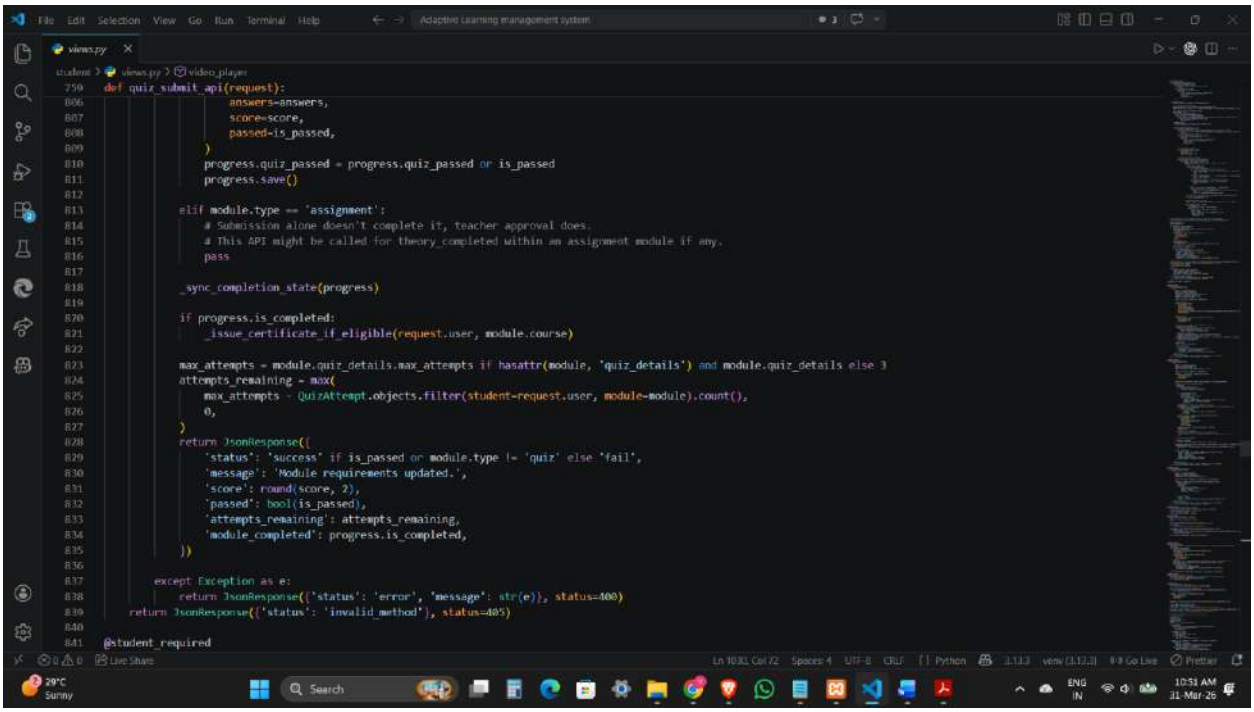
```python
def watch_event_api(request):
    return JsonResponse({'status': 'invalid_method'}, status=405)

@student_required
def quiz_submit_api(request):
    if request.method == 'POST':
        try:
            data = json.loads(request.body)
            module_id = data.get('module_id')
            answers = data.get('answers', {})
            theory_completed = data.get('theory_completed', False)

            module = get_object_or_404(Module, id=module_id)

            progress, _ = StudentProgress.objects.get_or_create(
                student=request.user,
                course=module.course,
                module=module
            )

            progress.theory_completed = progress.theory_completed or bool(theory_completed)

            questions = _module_quiz_questions(module)
            if questions:
                has_submitted_answers = any(
                    _normalize_quiz_answer(value) != ''
                    for value in (answers or {}).values()
                )

                if not has_submitted_answers:
                    progress.save()
                    _sync_completion_state(progress)
                    return JsonResponse({
                        'status': 'pending_quiz',
                        'message': 'Please answer the quiz before completing this module.',
                        'module_completed': progress.is_completed,
                    }, status=400)
```

```python
def quiz_submit_api(request):
                    answers=answers,
                    score=score,
                    passed=is_passed,
                )
                progress.quiz_passed = progress.quiz_passed or is_passed
                progress.save()

            elif module.type == 'assignment':
                # Submission alone doesn't complete it, teacher approval does.
                # This API might be called for theory_completed within an assignment module if any.
                pass

            _sync_completion_state(progress)

            if progress.is_completed:
                _issue_certificate_if_eligible(request.user, module.course)

            max_attempts = module.quiz_details.max_attempts if hasattr(module, 'quiz_details') and module.quiz_details else 3
            attempts_remaining = max(
                max_attempts - QuizAttempt.objects.filter(student=request.user, module=module).count(),
                0,
            )
            return JsonResponse({
                'status': 'success' if is_passed or module.type != 'quiz' else 'fail',
                'message': 'Module requirements updated.',
                'score': round(score, 2),
                'passed': bool(is_passed),
                'attempts_remaining': attempts_remaining,
                'module_completed': progress.is_completed,
            })

        except Exception as e:
            return JsonResponse({'status': 'error', 'message': str(e)}, status=400)
    return JsonResponse({'status': 'invalid_method'}, status=405)

@student_required
```

Day 3 :- Restrict module completion based on quiz performance



```python
class ModuleProgress(models.Model):
    module = models.ForeignKey(Module, on_delete=models.CASCADE)
    is_completed = models.BooleanField(default=False)
    video_progress = models.FloatField(default=0)
    last_watched_time = models.FloatField(default=0)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        unique_together = ('user', 'module')

    def __str__(self):
        return f"{self.user.username} - {self.module.title} ({self.video_progress}%)"


class Quiz(models.Model):
    module = models.ForeignKey(Module, on_delete=models.CASCADE, related_name='quizzes')

    def __str__(self):
        return f"Quiz for {self.module.title}"


class Question(models.Model):
    quiz = models.ForeignKey(Quiz, on_delete=models.CASCADE, related_name='questions')
    text = models.CharField(max_length=255)

    def __str__(self):
        return self.text


class Option(models.Model):
    question = models.ForeignKey(Question, on_delete=models.CASCADE, related_name='options')
    text = models.CharField(max_length=255)
    is_correct = models.BooleanField(default=False)

    def __str__(self):
        return self.text
```

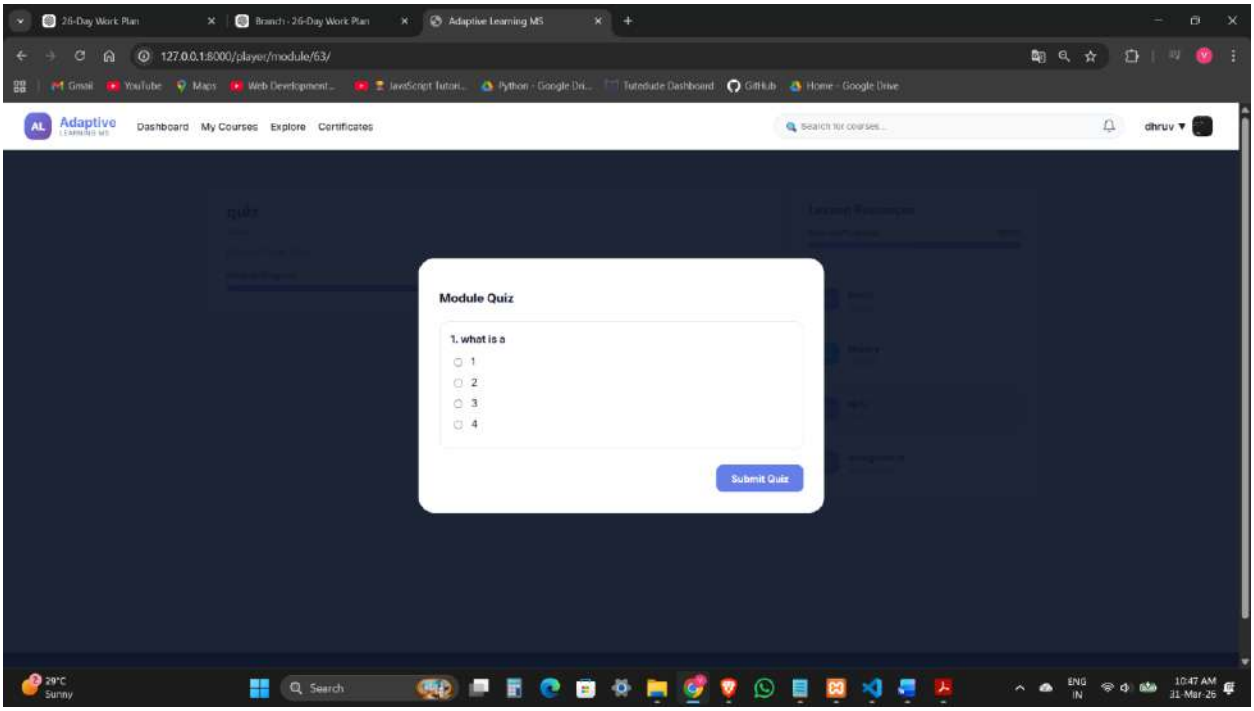


```python
def quiz_submit_api(request):
                    answers=answers,
                    score=score,
                    passed=is_passed,
                )
                progress.quiz_passed = progress.quiz_passed or is_passed
                progress.save()

            elif module.type == 'assignment':
                # Submission alone doesn't complete it, teacher approval does.
                # This API might be called for theory_completed within an assignment module if any.
                pass

            _sync_completion_state(progress)

            if progress.is_completed:
                _issue_certificate_if_eligible(request.user, module.course)

            max_attempts = module.quiz_details.max_attempts if hasattr(module, 'quiz_details') and module.quiz_details else 3
            attempts_remaining = max(
                max_attempts - QuizAttempt.objects.filter(student=request.user, module=module).count(),
                0,
            )
            return JsonResponse({
                'status': 'success' if is_passed or module.type != 'quiz' else 'fail',
                'message': 'Module requirements updated.',
                'score': round(score, 2),
                'passed': bool(is_passed),
                'attempts_remaining': attempts_remaining,
                'module_completed': progress.is_completed,
            })

        except Exception as e:
            return JsonResponse({'status': 'error', 'message': str(e)}, status=400)
    return JsonResponse({'status': 'invalid method'}, status=405)

@student_required
```

Day 4 :- Store quiz attempts and results for progress tracking

```python
# ----------------------------------
class QuizQuestion(models.Model):
    module = models.ForeignKey(Module, on_delete=models.CASCADE, related_name='legacy_quiz_questions')
    question_text = models.TextField()
    option_a = models.CharField(max_length=500)
    option_b = models.CharField(max_length=500)
    option_c = models.CharField(max_length=500)
    option_d = models.CharField(max_length=500)
    correct_answer = models.CharField(max_length=1, choices=[('A', 'A'), ('B', 'B'), ('C', 'C'), ('D', 'D')])
    order = models.PositiveIntegerField(default=0)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['order']

    def __str__(self):
        return f"{self.module.title} - Q{self.order + 1}"

    module = models.ForeignKey(Module, on_delete=models.CASCADE, related_name='quiz_questions')
    question_text = models.TextField()
    option_a = models.CharField(max_length=500)
    option_b = models.CharField(max_length=500)
    option_c = models.CharField(max_length=500)
    option_d = models.CharField(max_length=500)
    correct_answer = models.CharField(max_length=1, choices=[('A', 'A'), ('B', 'B'), ('C', 'C'), ('D', 'D')])
    order = models.PositiveIntegerField(default=0)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['order']

    def __str__(self):
        return f"{self.module.title} - Q{self.order + 1}"

# ----------------------------------
# 📊 Student Progress
# ----------------------------------
```

```python
class ModuleProgress(models.Model):
    module = models.ForeignKey(Module, on_delete=models.CASCADE)
    is_completed = models.BooleanField(default=False)
    video_progress = models.FloatField(default=0)
    last_watched_time = models.FloatField(default=0)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        unique_together = ('user', 'module')

    def __str__(self):
        return f"{self.user.username} - {self.module.title} ({self.video_progress}%)"


class Quiz(models.Model):
    module = models.ForeignKey(Module, on_delete=models.CASCADE, related_name='quizzes')

    def __str__(self):
        return f"Quiz for {self.module.title}"


class Question(models.Model):
    quiz = models.ForeignKey(Quiz, on_delete=models.CASCADE, related_name='questions')
    text = models.CharField(max_length=255)

    def __str__(self):
        return self.text


class Option(models.Model):
    question = models.ForeignKey(Question, on_delete=models.CASCADE, related_name='options')
    text = models.CharField(max_length=255)
    is_correct = models.BooleanField(default=False)

    def __str__(self):
        return self.text
```



- **Week 10 Plan :-** Enhance learning flow with rules, module completion logic, and certificate request system (student side).

Total Working Hrs: 28 hours **Signature of Student:**

The above entries are correct and the grading of work done by Trainee is

Excellent	Very Good	Good	Fair	Below Average	Poor

Signature of External Guide with Company Seal: **Signature of Internal Guide**

Date: **Date:**

SAL EDUCATION

SAL ENGINEERING AND TECHNICAL INSTITUTE

Opp. Science city, Sola-Bhadaj Road, Ahmedabad, Gujarat 380060
Ph:- 079 67129000 Website: www.sal.edu.in

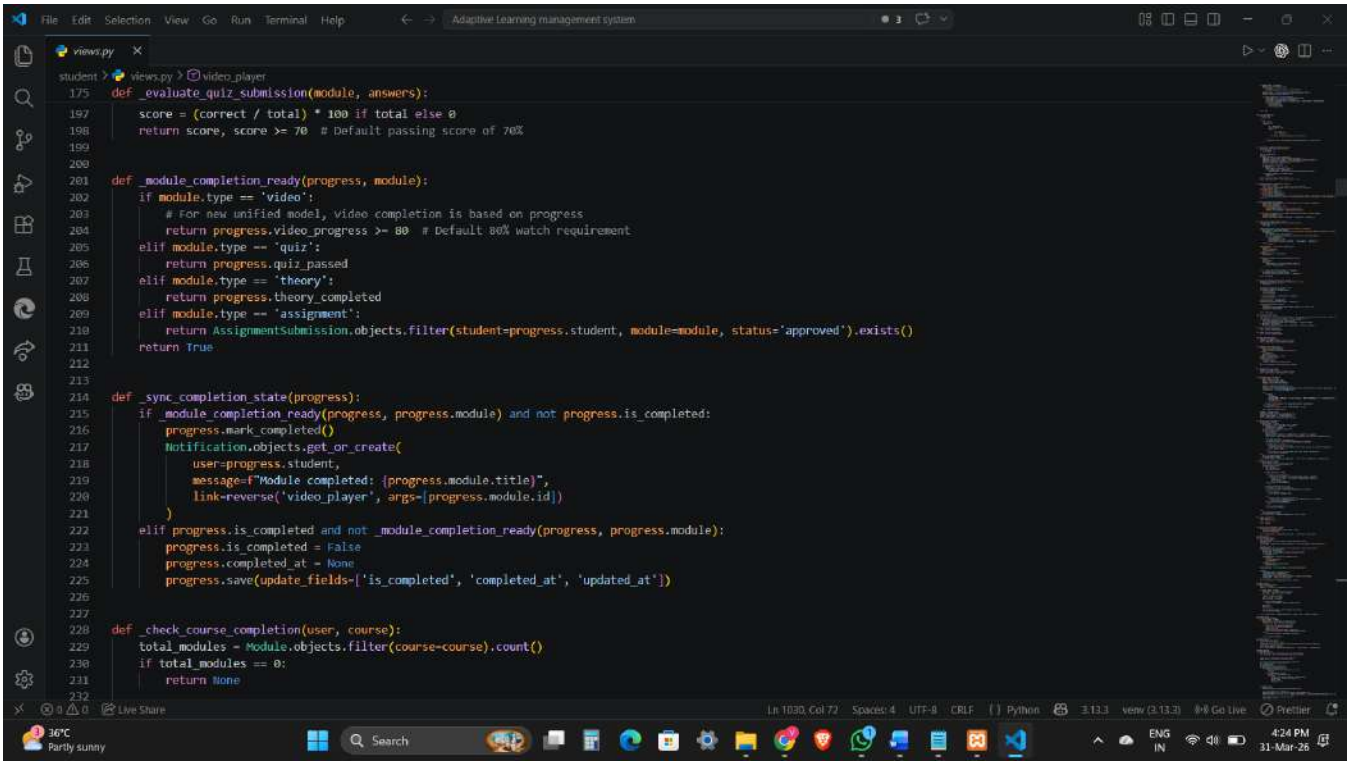
Student's Weekly Report of Internship

Student Name:	Vekariya Vraj Dipakbhai			
Enrollment Number:	221260116056			
Name Of Organization:	Bisag N			
External Guide Name:	Parth Sutariya			
External Guide Contact details:	**Email:**	Parthsutariya12@gmail.com	**Mob No:**	81558 51038
Internal Faculty Guide Name:	Prof. Amitkumar J. Patel			

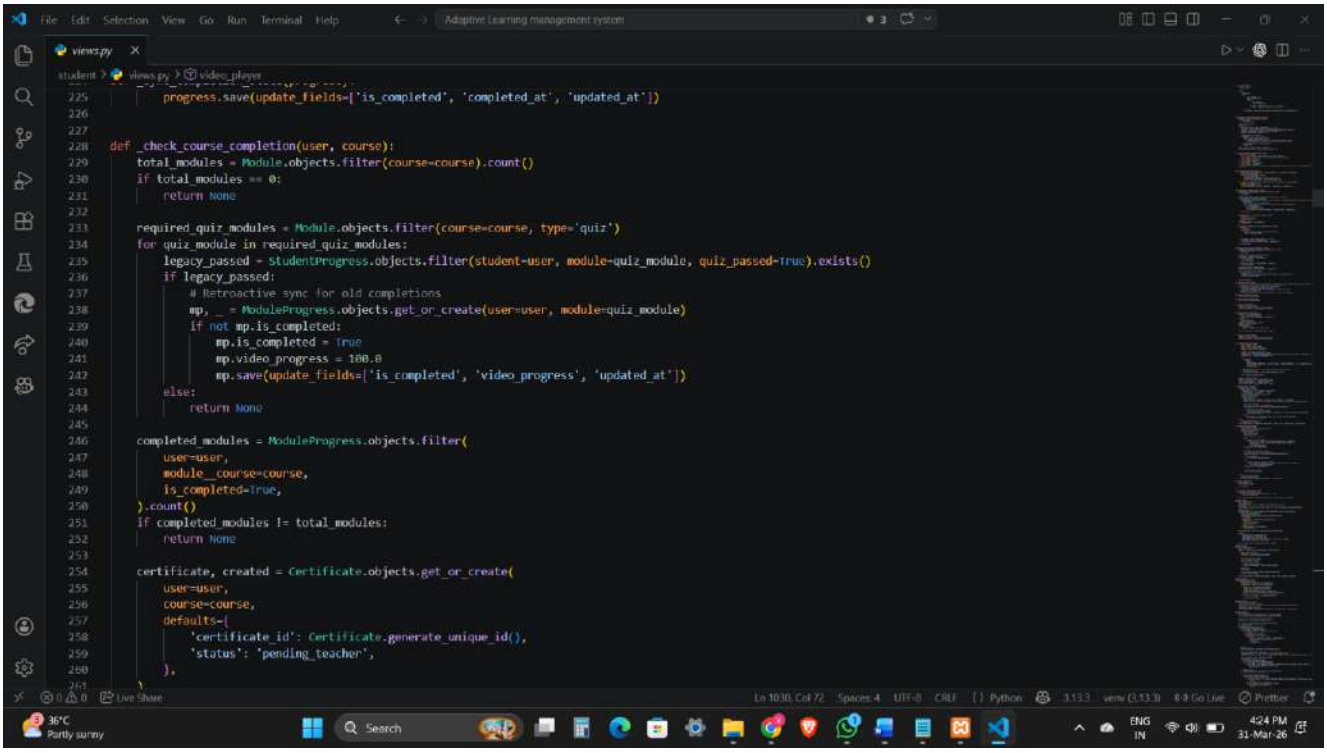
Week 10 (09/03/2026 – 15/03/2026)

1. Implement module completion rules (video watch % + quiz pass condition)
2. Restrict next module access until conditions are satisfied
3. Display course completion status on student dashboard
4. Create certificate request feature (student side)
5. Show certificate eligibility status to student

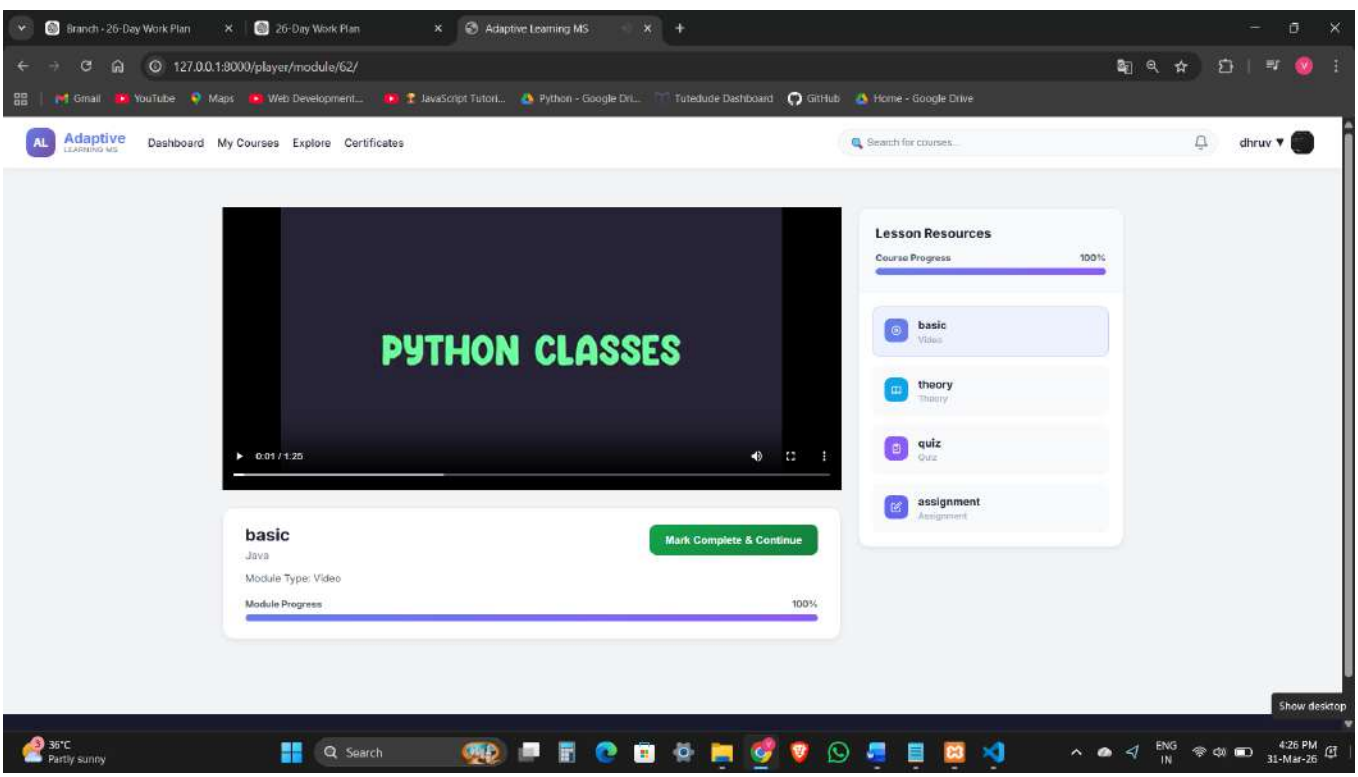
Day 1 :- Implement module completion rules (video watch % + quiz pass condition)



```python
def _evaluate_quiz_submission(module, answers):
    score = (correct / total) * 100 if total else 0
    return score, score >= 70  # Default passing score of 70%


def _module_completion_ready(progress, module):
    if module.type == 'video':
        # For new unified model, video completion is based on progress
        return progress.video_progress >= 80  # Default 80% watch requirement
    elif module.type == 'quiz':
        return progress.quiz_passed
    elif module.type == 'theory':
        return progress.theory_completed
    elif module.type == 'assignment':
        return AssignmentSubmission.objects.filter(student=progress.student, module=module, status='approved').exists()
    return True


def _sync_completion_state(progress):
    if _module_completion_ready(progress, progress.module) and not progress.is_completed:
        progress.mark_completed()
        Notification.objects.get_or_create(
            user=progress.student,
            message=f"Module completed: {progress.module.title}",
            link=reverse('video_player', args=[progress.module.id])
        )
    elif progress.is_completed and not _module_completion_ready(progress, progress.module):
        progress.is_completed = False
        progress.completed_at = None
        progress.save(update_fields=['is_completed', 'completed_at', 'updated_at'])


def _check_course_completion(user, course):
    total_modules = Module.objects.filter(course=course).count()
    if total_modules == 0:
        return None
```



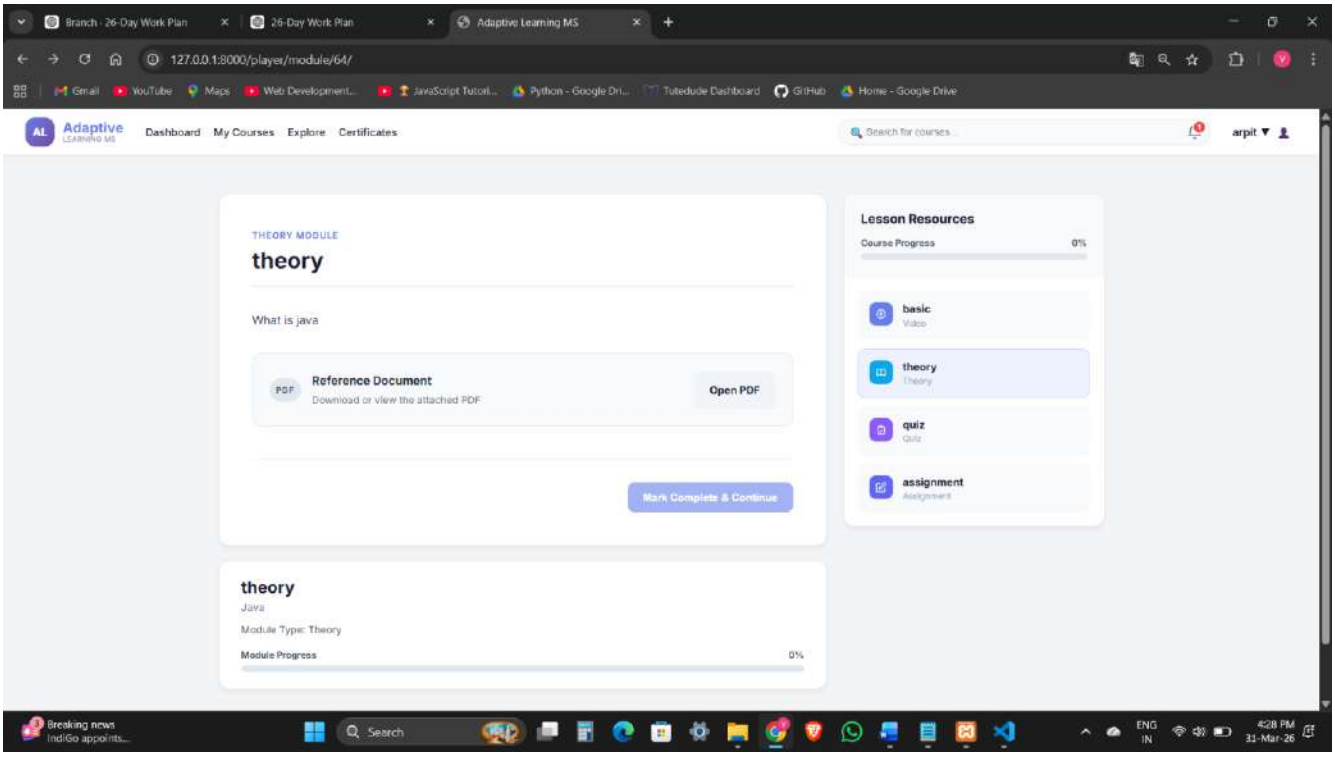
```python
        progress.save(update_fields=['is_completed', 'completed_at', 'updated_at'])


def _check_course_completion(user, course):
    total_modules = Module.objects.filter(course=course).count()
    if total_modules == 0:
        return None

    required_quiz_modules = Module.objects.filter(course=course, type='quiz')
    for quiz_module in required_quiz_modules:
        legacy_passed = StudentProgress.objects.filter(student=user, module=quiz_module, quiz_passed=True).exists()
        if legacy_passed:
            # Retroactive sync for old completions
            mp, _ = ModuleProgress.objects.get_or_create(user=user, module=quiz_module)
            if not mp.is_completed:
                mp.is_completed = True
                mp.video_progress = 100.0
                mp.save(update_fields=['is_completed', 'video_progress', 'updated_at'])
        else:
            return None

    completed_modules = ModuleProgress.objects.filter(
        user=user,
        module__course=course,
        is_completed=True,
    ).count()
    if completed_modules != total_modules:
        return None

    certificate, created = Certificate.objects.get_or_create(
        user=user,
        course=course,
        defaults={
            'certificate_id': Certificate.generate_unique_id(),
            'status': 'pending_teacher',
        },
    )
```



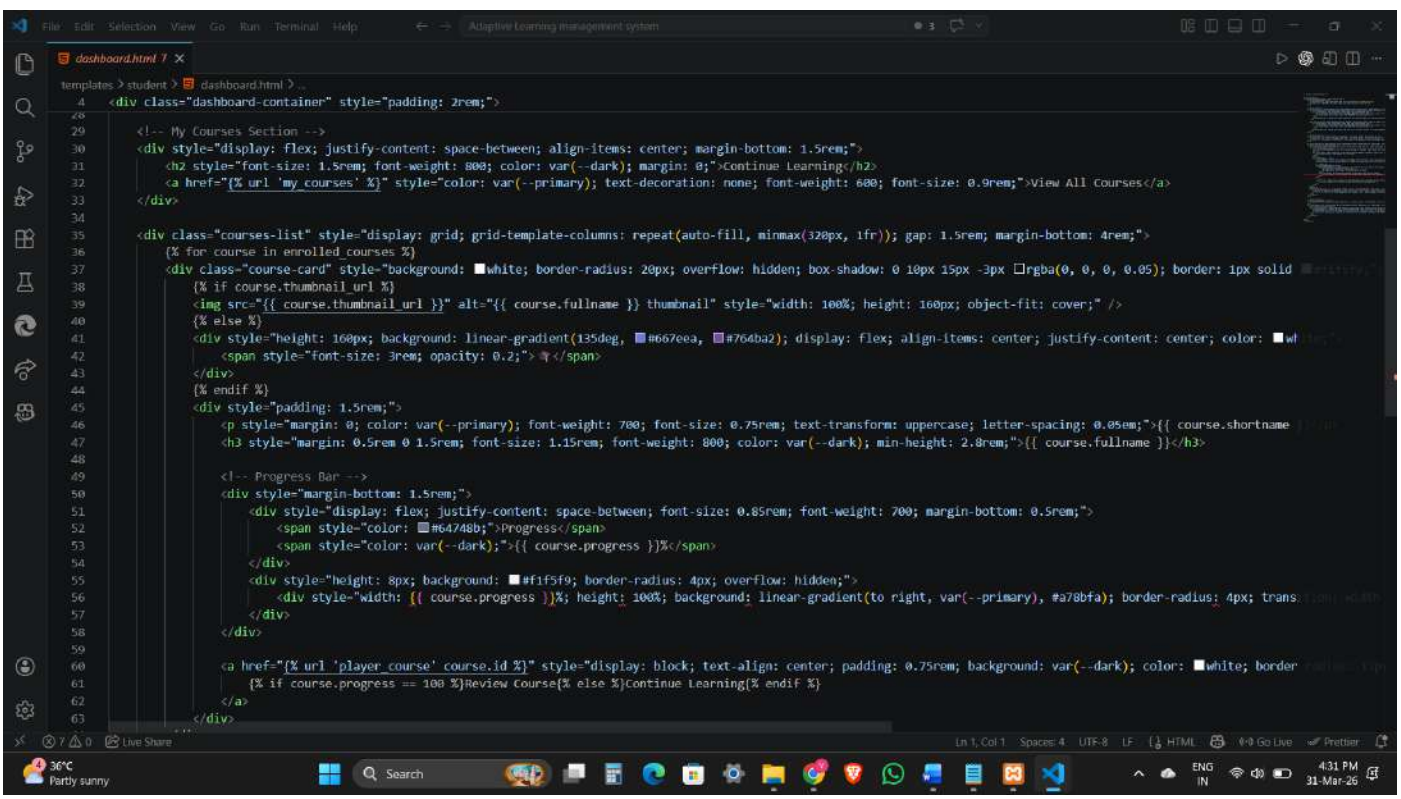
Day 2 :- Restrict next module access until conditions are satisfied

```python
def _evaluate_quiz_submission(module, answers):
    score = (correct / total) * 100 if total else 0
    return score, score >= 70  # Default passing score of 70%


def _module_completion_ready(progress, module):
    if module.type == 'video':
        # For now unified model, video completion is based on progress
        return progress.video_progress >= 80  # Default 80% watch requirement
    elif module.type == 'quiz':
        return progress.quiz_passed
    elif module.type == 'theory':
        return progress.theory_completed
    elif module.type == 'assignment':
        return AssignmentSubmission.objects.filter(student=progress.student, module=module, status='approved').exists()
    return True


def _sync_completion_state(progress):
    if _module_completion_ready(progress, progress.module) and not progress.is_completed:
        progress.mark_completed()
        Notification.objects.get_or_create(
            user=progress.student,
            message=f"Module completed: {progress.module.title}",
            link=reverse('video_player', args=[progress.module.id])
        )
    elif progress.is_completed and not _module_completion_ready(progress, progress.module):
        progress.is_completed = False
        progress.completed_at = None
        progress.save(update_fields=['is_completed', 'completed_at', 'updated_at'])


def _check_course_completion(user, course):
    total_modules = Module.objects.filter(course=course).count()
    if total_modules == 0:
        return None
```

```python
        progress.save(update_fields=['is_completed', 'completed_at', 'updated_at'])


def _check_course_completion(user, course):
    total_modules = Module.objects.filter(course=course).count()
    if total_modules == 0:
        return None

    required_quiz_modules = Module.objects.filter(course=course, type='quiz')
    for quiz_module in required_quiz_modules:
        legacy_passed = StudentProgress.objects.filter(student=user, module=quiz_module, quiz_passed=True).exists()
        if legacy_passed:
            # Retroactive sync for old completions
            mp, _ = ModuleProgress.objects.get_or_create(user=user, module=quiz_module)
            if not mp.is_completed:
                mp.is_completed = True
                mp.video_progress = 100.0
                mp.save(update_fields=['is_completed', 'video_progress', 'updated_at'])
        else:
            return None

    completed_modules = ModuleProgress.objects.filter(
        user=user,
        module__course=course,
        is_completed=True,
    ).count()
    if completed_modules != total_modules:
        return None

    certificate, created = Certificate.objects.get_or_create(
        user=user,
        course=course,
        defaults={
            'certificate_id': Certificate.generate_unique_id(),
            'status': 'pending_teacher',
        },
    )
```

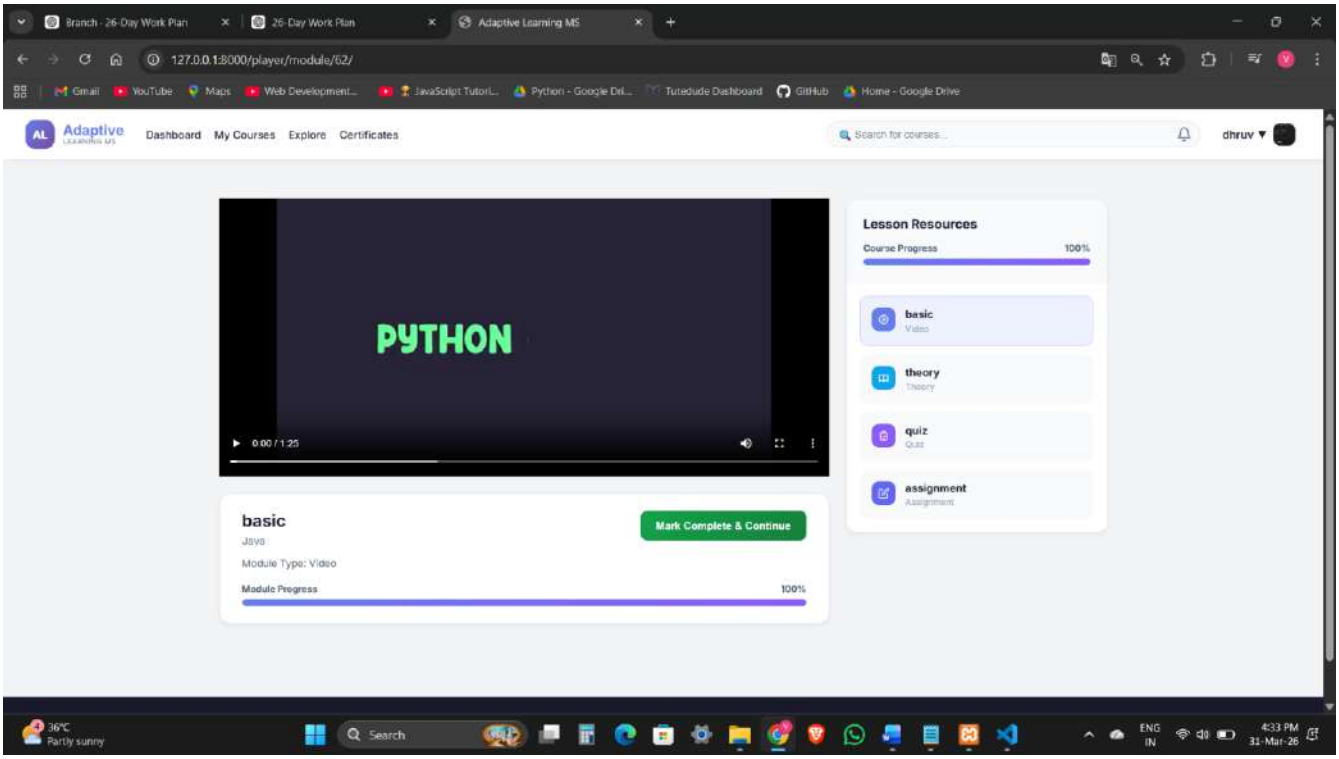


Day 3 :- Display course completion status on student dashboard

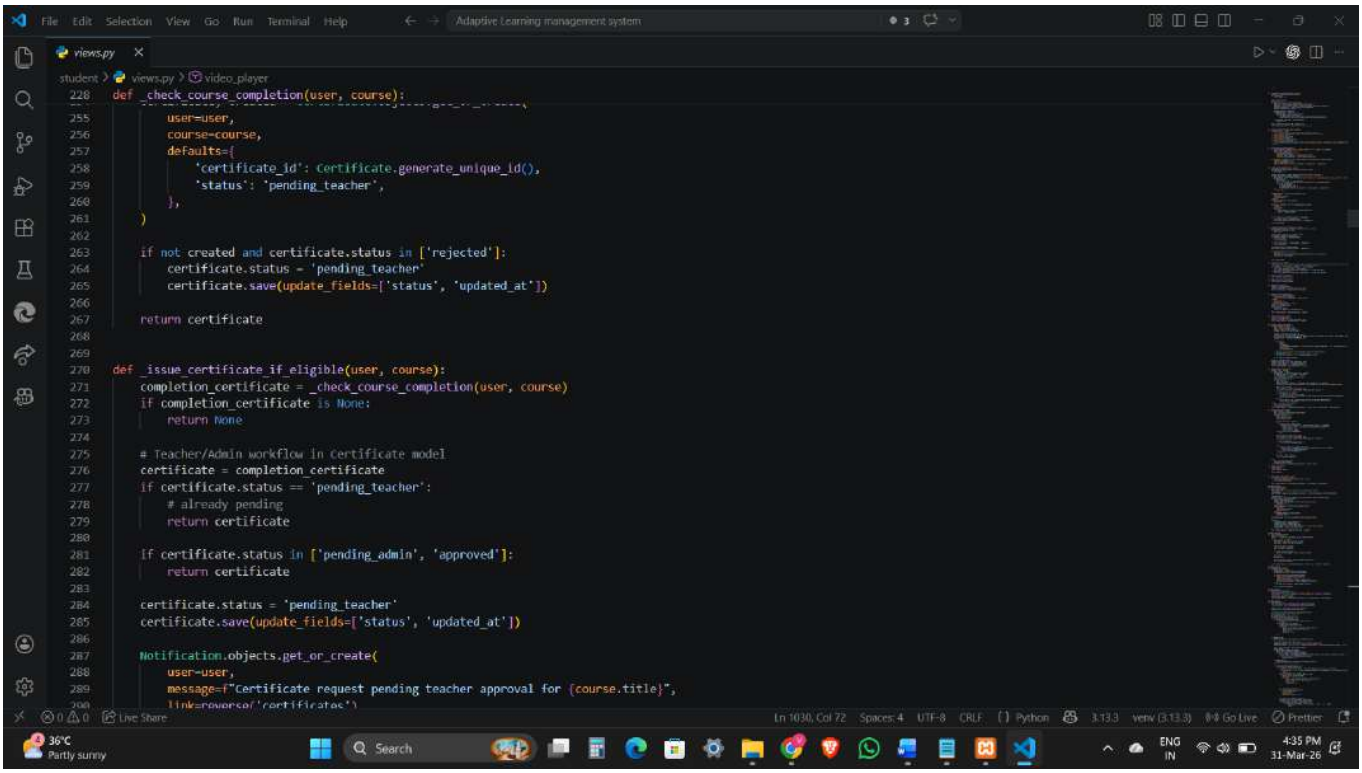


```html
<div class="dashboard-container" style="padding: 2rem;">
    <div class="courses-list" style="display: grid; grid-template-columns: repeat(auto-fill, minmax(320px, 1fr)); gap: 1.5rem; margin-bottom: 4rem;">
        <div class="course-card" style="background: white; border-radius: 20px; overflow: hidden; box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.05); border: 1px solid
            <div style="padding: 1.5rem;">
            </div>
        </div>
        {% empty %}
        <div style="grid-column: 1 / -1; background: #f8fafc; border: 2px dashed #e2e8f0; border-radius: 16px; padding: 4rem; text-align: center;">
            <p style="color: #64748b; margin-bottom: 1.5rem; font-size: 1.1rem;">You aren't enrolled in any courses yet.</p>
            <a href="{% url 'explore' %}" style="background: var(--primary); color: white; padding: 0.75rem 2rem; border-radius: 12px; text-decoration: none; font-weigh
        </div>
        {% endfor %}
    </div>

    <!-- Explore Section -->
    <div style="margin-bottom: 1.5rem;">
        <h2 style="font-size: 1.5rem; font-weight: 800; color: var(--dark); margin: 0;">Recommended For You</h2>
    </div>
    <div class="explore-list" style="display: grid; grid-template-columns: repeat(auto-fill, minmax(280px, 1fr)); gap: 1.5rem;">
        {% for course in explore_courses %}
        <div class="explore-card" style="background: #f8fafc; border-radius: 16px; padding: 1.25rem; border: 1px solid #f1f5f9;">
            <h4 style="margin: 0 0 0.5rem; font-size: 1rem; font-weight: 700; color: var(--dark);">{{ course.title }}</h4>
            <p style="margin: 0 0 1rem; color: #64748b; font-size: 0.85rem; line-height: 1.5;">{{ course.description|striptags|truncatewords:12 }}</p>
            <a href="{% url 'enroll_course' course.id %}" style="color: var(--primary); font-weight: 700; text-decoration: none; font-size: 0.85rem;">Enrol Now →</a>
        </div>
        {% endfor %}
    </div>

</div>
{% endblock %}
```

```python
def _sync_completion_state(progress):
    if _module_completion_ready(progress, progress.module) and not progress.is_completed:
        progress.mark_completed()
        Notification.objects.get_or_create(
            user=progress.student,
            message=f"Module completed: {progress.module.title}",
            link=reverse('video_player', args=[progress.module.id])
        )
    elif progress.is_completed and not _module_completion_ready(progress, progress.module):
        progress.is_completed = False
        progress.completed_at = None
        progress.save(update_fields=['is_completed', 'completed_at', 'updated_at'])


def _check_course_completion(user, course):
    total_modules = Module.objects.filter(course=course).count()
    if total_modules == 0:
        return None

    required_quiz_modules = Module.objects.filter(course=course, type='quiz')
    for quiz_module in required_quiz_modules:
        legacy_passed = StudentProgress.objects.filter(student=user, module=quiz_module, quiz_passed=True).exists()
        if legacy_passed:
            # Retroactive sync for old completions
            mp, _ = ModuleProgress.objects.get_or_create(user=user, module=quiz_module)
            if not mp.is_completed:
                mp.is_completed = True
                mp.video_progress = 100.0
                mp.save(update_fields=['is_completed', 'video_progress', 'updated_at'])
        else:
            return None

    completed_modules = ModuleProgress.objects.filter(
        user=user,
        module__course=course,
        is_completed=True,
    ).count()
```

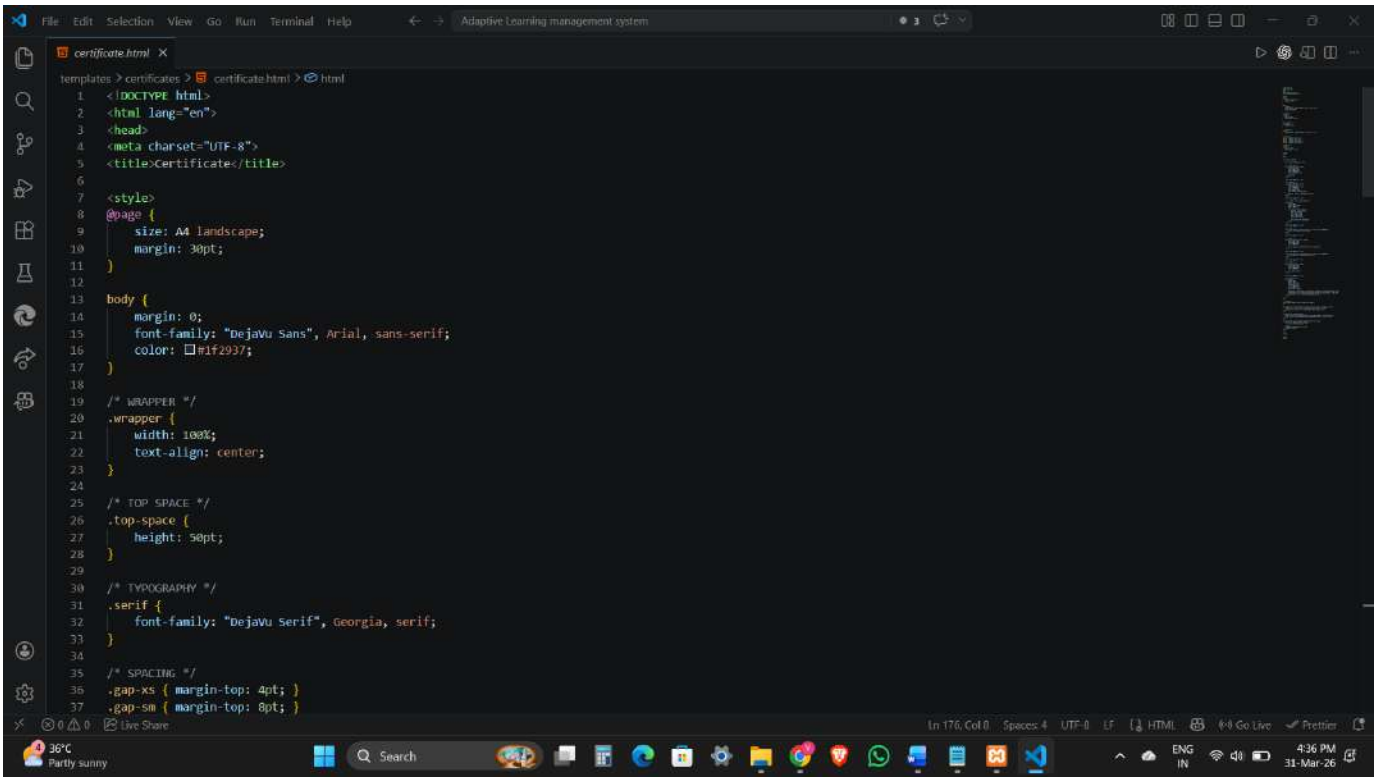


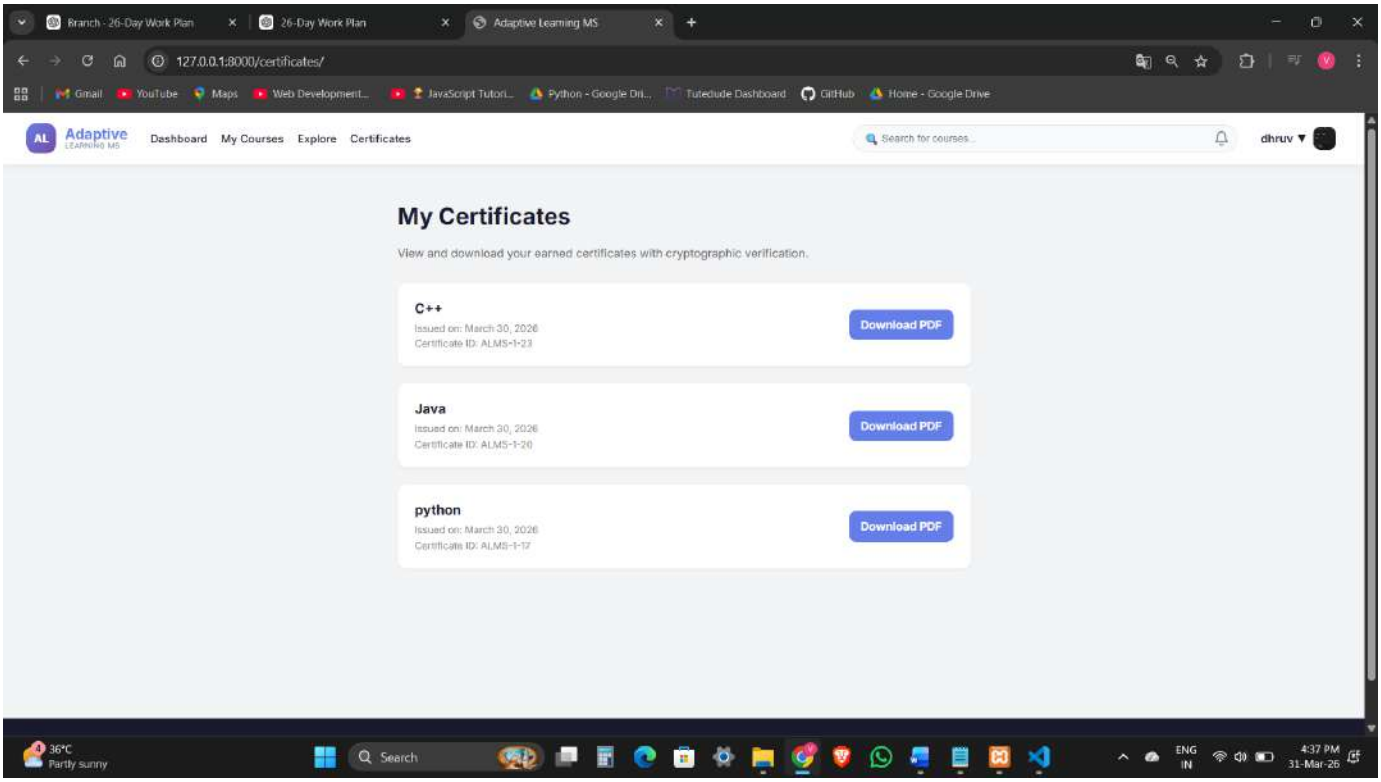
Day 4 :- Create certificate request feature (student side)



```python
def _check_course_completion(user, course):
        user=user,
        course=course,
        defaults={
            'certificate_id': Certificate.generate_unique_id(),
            'status': 'pending_teacher',
        },
    )

    if not created and certificate.status in ['rejected']:
        certificate.status = 'pending_teacher'
        certificate.save(update_fields=['status', 'updated_at'])

    return certificate


def _issue_certificate_if_eligible(user, course):
    completion_certificate = _check_course_completion(user, course)
    if completion_certificate is None:
        return None

    # Teacher/Admin workflow in Certificate model
    certificate = completion_certificate
    if certificate.status == 'pending_teacher':
        # already pending
        return certificate

    if certificate.status in ['pending_admin', 'approved']:
        return certificate

    certificate.status = 'pending_teacher'
    certificate.save(update_fields=['status', 'updated_at'])

    Notification.objects.get_or_create(
        user=user,
        message=f"Certificate request pending teacher approval for {course.title}",
        link=reverse('certificates')
```

```python
    return certificate

def customize_auth_form(form):
    # We only want to transform the username field to 'Email Address' for the login form (which has no email field).
    # For the signup form, it will naturally have both username and email.
    if 'username' in form.fields and 'email' not in form.fields:
        form.fields['username'].label = 'Email Address'
        form.fields['username'].widget.attrs['placeholder'] = 'Enter your email'
    if 'password' in form.fields:
        form.fields['password'].widget.attrs['placeholder'] = 'Enter your password'

def login_role_selection(request):
    return redirect('student_login')

def signup_role_selection(request):
    return redirect('student_signup')


def home_redirect(request):
    context = _landing_context()
    context['user_role'] = get_user_role(request.user)
    return render(request, 'main/home.html', context)


def landing_courses_view(request):
    courses = Course.objects.annotate(
        module_count=Count('module'),
        learner_count=Count('enrollment', distinct=True),
    ).filter(
        is_active=True,
        module_count__gt=0,
    ).order_by('-learner_count', 'title')
    context = _landing_context()
    context.update({
        'courses': courses,
        'user_role': get_user_role(request.user),
    })
```

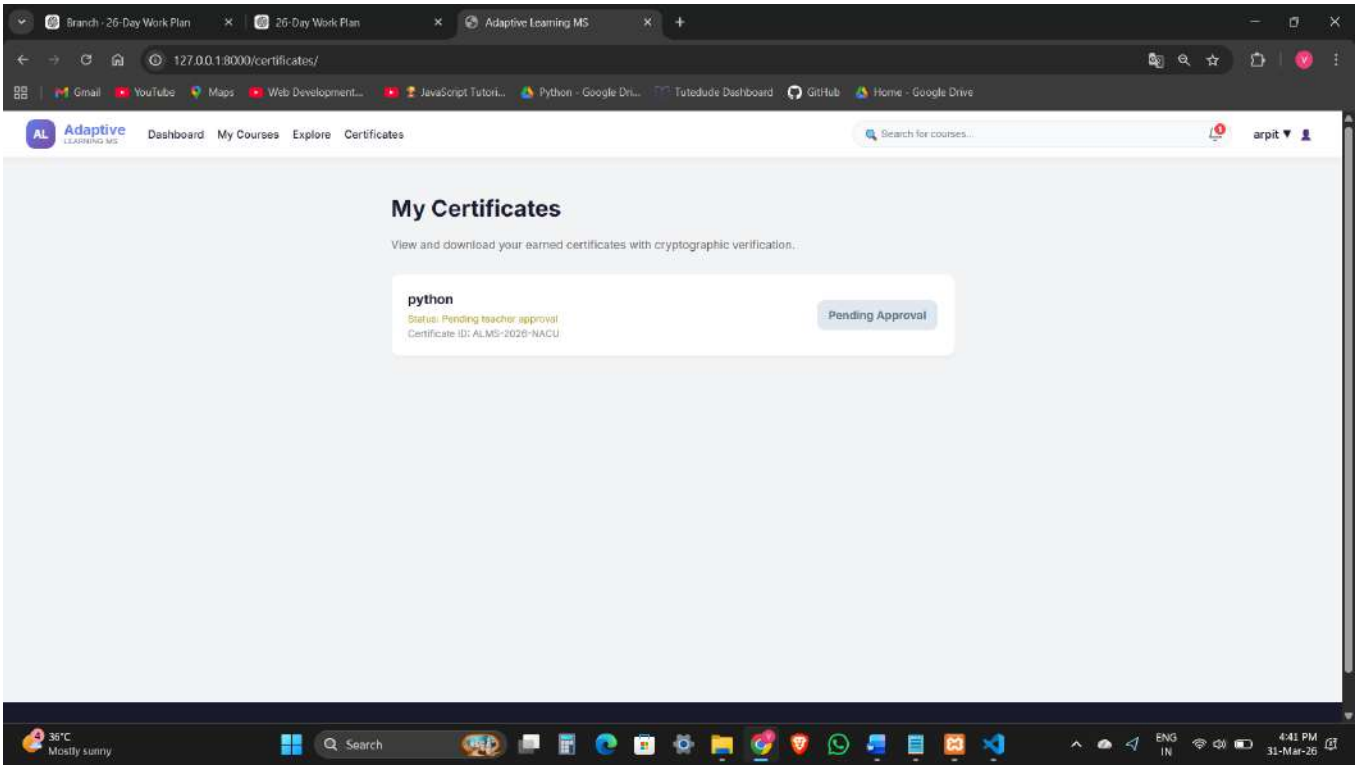

File Edit Selection View Go Run Terminal Help
Adaptive Learning management system
certificate.html
templates > certificates > certificate.html > html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Certificate</title>
<style>
@page {
size: A4 landscape;
margin: 30pt;
}
body {
margin: 0;
font-family: "DejaVu Sans", Arial, sans-serif;
color: #1f2937;
}
/* WRAPPER */
.wrapper {
width: 100%;
text-align: center;
}
/* TOP SPACE */
.top-space {
height: 50pt;
}
/* TYPOGRAPHY */
.serif {
font-family: "DejaVu Serif", Georgia, serif;
}
/* SPACING */
.gap-xs { margin-top: 4pt; }
.gap-sm { margin-top: 8pt; }
Live Share
Ln 176, Col 0 Spaces: 4 UTF-8 LF HTML Go Live Prettier
36°C Partly sunny
Search
ENG IN
4:36 PM 31-Mar-26


Branch - 26-Day Work Plan
26-Day Work Plan
Adaptive Learning MS
127.0.0.1:8000/certificates/
Gmail YouTube Maps Web Development... JavaScript Tutori... Python - Google Dri... Tutedude Dashboard GitHub Home - Google Drive
AL Adaptive LEARNING MS
Dashboard My Courses Explore Certificates
Search for courses...
dhruv
My Certificates
View and download your earned certificates with cryptographic verification.
C++
Issued on: March 30, 2026
Certificate ID: ALMS-1-23
Download PDF
Java
Issued on: March 30, 2026
Certificate ID: ALMS-1-20
Download PDF
python
Issued on: March 30, 2026
Certificate ID: ALMS-1-17
Download PDF
36°C Partly sunny
Search
ENG IN
4:37 PM 31-Mar-26

Day 5 :- Show certificate eligibility status to student

```html
{% extends 'student/base.html' %}

{% block content %}
<div class="certificates-container" style="max-width: 800px; margin: 0 auto;">
    <h1>My Certificates</h1>
    <p style="color: #666;">View and download your earned certificates with cryptographic verification.</p>

    <div class="certificates-list" style="margin-top: 2rem; display: grid; gap: 1.5rem;">
        {% for cert in certificates %}
        <div class="cert-card"
            style="background: white; padding: 1.5rem; border-radius: 12px; display: flex; justify-content: space-between; align-items: center; box-shadow: 0 2px 4px
            <div>
                <h3 style="margin: 0;">{{ cert.course.title }}</h3>
                {% if cert.status == 'approved' %}
                <p style="margin: 0.5rem 0 0; font-size: 0.85rem; color: #888;">Issued on: {{ cert.issued_at|date:"F j, Y" }}</p>
                {% elif cert.status == 'pending_teacher' %}
                <p style="margin: 0.5rem 0 0; font-size: 0.85rem; color: #b59f10;">Status: Pending teacher approval</p>
                {% elif cert.status == 'pending_admin' %}
                <p style="margin: 0.5rem 0 0; font-size: 0.85rem; color: #1f6feb;">Status: Pending admin approval</p>
                {% elif cert.status == 'rejected' %}
                <p style="margin: 0.5rem 0 0; font-size: 0.85rem; color: #991b1b;">Status: Rejected</p>
                {% endif %}
                <p style="margin: 0.35rem 0 0; font-size: 0.85rem; color: #888;">Certificate ID: {{ cert.certificate_id }}</p>
            </div>
            <div style="display: flex; gap: 1rem; align-items: center;">
                {% if cert.status == 'approved' %}
                <a href="{% url 'download_certificate' cert.certificate_id %}"
                   style="display: inline-flex; align-items: center; justify-content: center; padding: 0.7rem 1rem; border-radius: 10px; background: var(--primary); color
                   Download PDF
                </a>
                {% else %}
                <span
                   style="display: inline-flex; align-items: center; justify-content: center; padding: 0.7rem 1rem; border-radius: 10px; background: #e2e8f0; color:
                   Pending Approval
                </span>
                {% endif %}
            </div>
```

```python
def _check_course_completion(user, course):
        user=user,
        course=course,
        defaults={
            'certificate_id': certificate.generate_unique_id(),
            'status': 'pending_teacher',
        },
    )

    if not created and certificate.status in ['rejected']:
        certificate.status = 'pending_teacher'
        certificate.save(update_fields=['status', 'updated_at'])

    return certificate


def _issue_certificate_if_eligible(user, course):
    completion_certificate = _check_course_completion(user, course)
    if completion_certificate is None:
        return None

    # Teacher/Admin workflow in Certificate model
    certificate = completion_certificate
    if certificate.status == 'pending_teacher':
        # already pending
        return certificate

    if certificate.status in ['pending_admin', 'approved']:
        return certificate

    certificate.status = 'pending_teacher'
    certificate.save(update_fields=['status', 'updated_at'])

    Notification.objects.get_or_create(
        user=user,
        message=f"Certificate request pending teacher approval for {course.title}",
        link=reverse('certificates')
```



- **Week 11 Plan :-** Design certificate interface and implement certificate viewing & downloading functionality with verification details.

Total Working Hrs: 35 hours **Signature of Student:**

The above entries are correct and the grading of work done by Trainee is

Excellent	Very Good	Good	Fair	Below Average	Poor

Signature of External Guide with Company Seal: **Signature of Internal Guide**

Date: **Date:**

SAL EDUCATION

SAL ENGINEERING AND TECHNICAL INSTITUTE

Opp. Science city, Sola-Bhadaj Road, Ahmedabad, Gujarat 380060
Ph:- 079 67129000 Website: www.sal.edu.in

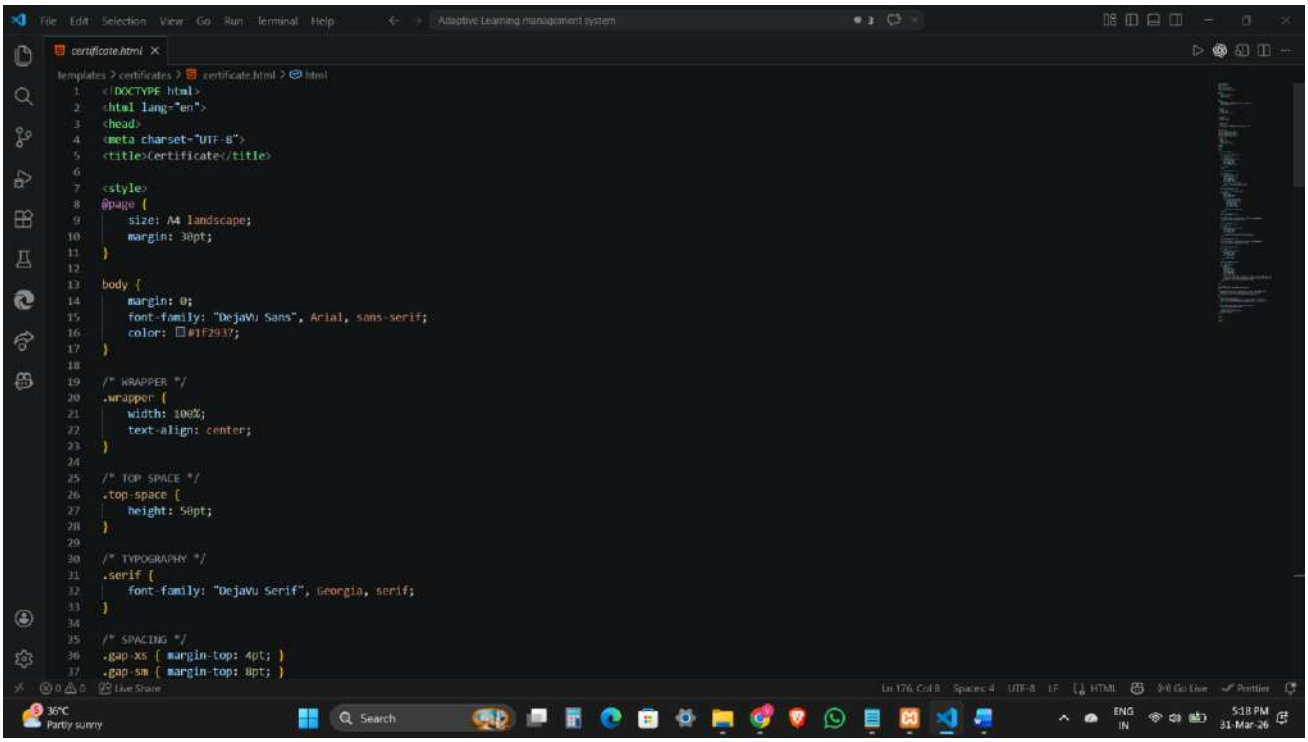
Student's Weekly Report of Internship

Student Name:	Vekariya Vraj Dipakbhai			
Enrollment Number:	221260116056			
Name Of Organization:	Bisag N			
External Guide Name:	Parth Sutariya			
External Guide Contact details:	**Email:**	Parthsutariya12@gmail.com	**Mob No:**	81558 51038
Internal Faculty Guide Name:	Prof. Amitkumar J. Patel			

Week 11 (16/03/2026 – 22/03/2026)

1. Design professional certificate UI (frontend template)
2. Display certificate with student details and course information
3. Add unique certificate ID for verification
4. Implement certificate download (PDF generation for student)

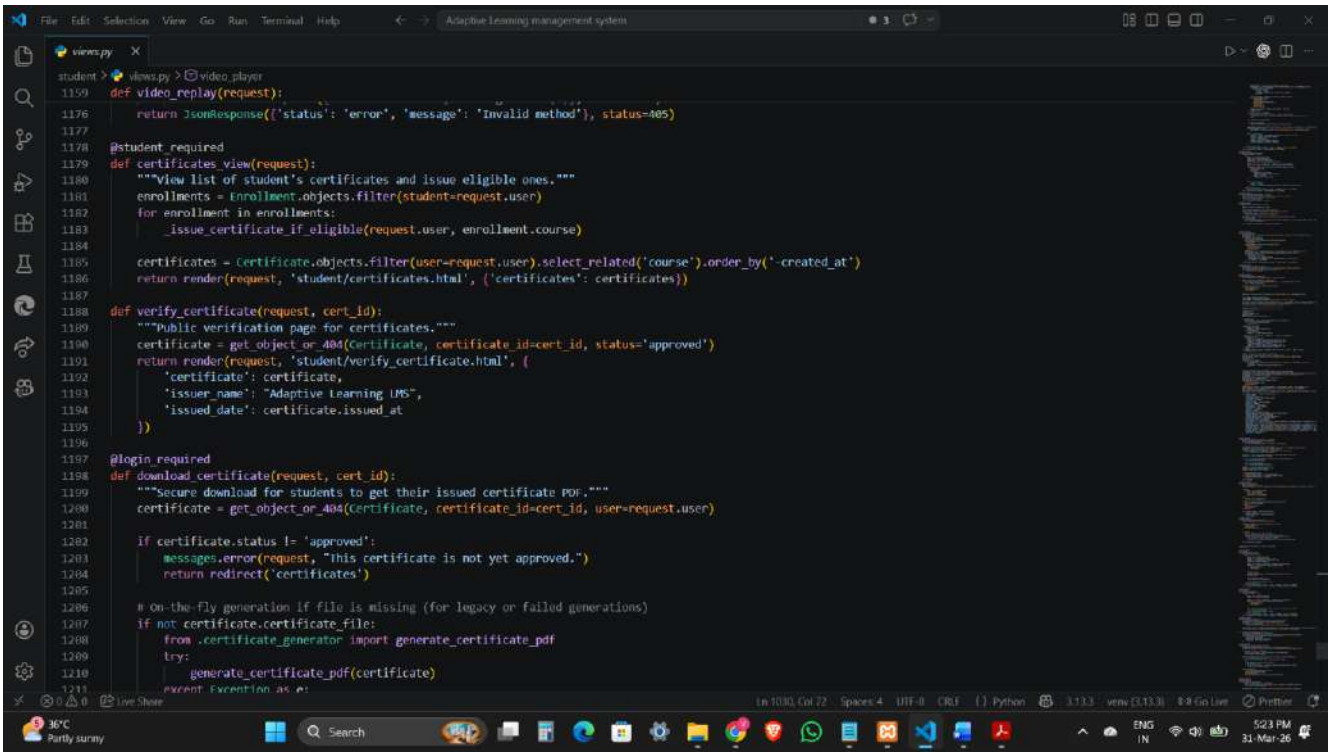
Day 1 :- Design professional certificate UI (frontend template)



```html
<html lang="en">
<body>
<div class="wrapper">
    <div class="center" style="font-size: 13pt; color: #4b5563;">
    </div>

    <div class="gap-sm"></div>

    <!-- NAME -->
    <div class="center serif" style="
        font-size: 30pt;
        font-weight: 700;
        color: #111827;
    ">
        {{ user.get_full_name|default:user.username }}
    </div>

    <div class="gap-sm"></div>

    <div class="center" style="font-size: 13pt; color: #4b5563;">
        has successfully completed the course
    </div>

    <div class="gap-sm"></div>

    <!-- COURSE -->
    <div class="center" style="
        font-size: 20pt;
        font-weight: 700;
        color: #b08d2a;
    ">
        {{ course.title }}
    </div>

    <div class="gap-lg"></div>

    <!-- DESCRIPTION -->
```

```html
<html lang="en">
<body>
</div>

<!-- FOOTER -->
<table width="100%" style="margin-top: 50pt;">
<tr>

<td width="33%" align="left" style="font-size: 11pt; line-height: 1.8;">
    <b>Date:</b> {{ certificate.issued_at|date:"F d, Y" }}<br>
    <b>Certificate ID:</b> {{ certificate.certificate_id }}
</td>

<td width="34%" align="center">
    <img src="{{ qr_code_base64 }}" width="90"><br>
    <span style="font-size:10pt;color:#6b7280;">Scan to verify</span>
</td>

<td width="33%" align="right" style="font-size: 11pt;">
    <div style="margin-bottom: 25pt;">
        _____
    </div>
    <b>Authorized Signature</b><br>
    Academic Director
</td>

</tr>
</table>

</body>
</html>
```

Day 2 :- Display certificate with student details and course information



```python
def video_replay(request):
    return JsonResponse({'status': 'error', 'message': 'Invalid method'}, status=405)

@student_required
def certificates_view(request):
    """View list of student's certificates and issue eligible ones."""
    enrollments = Enrollment.objects.filter(student=request.user)
    for enrollment in enrollments:
        _issue_certificate_if_eligible(request.user, enrollment.course)

    certificates = Certificate.objects.filter(user=request.user).select_related('course').order_by('-created_at')
    return render(request, 'student/certificates.html', {'certificates': certificates})

def verify_certificate(request, cert_id):
    """Public verification page for certificates."""
    certificate = get_object_or_404(Certificate, certificate_id=cert_id, status='approved')
    return render(request, 'student/verify_certificate.html', {
        'certificate': certificate,
        'issuer_name': "Adaptive Learning LMS",
        'issued_date': certificate.issued_at
    })

@login_required
def download_certificate(request, cert_id):
    """Secure download for students to get their issued certificate PDF."""
    certificate = get_object_or_404(Certificate, certificate_id=cert_id, user=request.user)

    if certificate.status != 'approved':
        messages.error(request, "This certificate is not yet approved.")
        return redirect('certificates')

    # On-the-fly generation if file is missing (for legacy or failed generations)
    if not certificate.certificate_file:
        from .certificate_generator import generate_certificate_pdf
        try:
            generate_certificate_pdf(certificate)
        except Exception as e:
```

```python
def verify_certificate(request, cert_id):
        'certificate': certificate,
        'issuer_name': "Adaptive Learning LMS",
        'issued_date': certificate.issued_at
    })

@login_required
def download_certificate(request, cert_id):
    """Secure download for students to get their issued certificate PDF."""
    certificate = get_object_or_404(certificate, certificate_id=cert_id, user=request.user)

    if certificate.status != 'approved':
        messages.error(request, "This certificate is not yet approved.")
        return redirect('certificates')

    # On-the-fly generation if file is missing (for legacy or failed generations)
    if not certificate.certificate_file:
        from .certificate_generator import generate_certificate_pdf
        try:
            generate_certificate_pdf(certificate)
        except Exception as e:
            messages.error(request, f"Unable to generate certificate PDF: {str(e)}")
            return redirect('certificates')

    return FileResponse(
        certificate.certificate_file.open('rb'),
        content_type='application/pdf',
        as_attachment=True,
        filename=f"Certificate-{certificate.certificate_id}.pdf"
    )

@login_required
@student_required
def notifications_view(request):
    if request.GET.get('action') == 'mark_read':
        Notification.objects.filter(user=request.user, is_read=False).update(is_read=True)
```

Day 3 :- Add unique certificate ID for verification

```python
class QuizAttempt(models.Model):
    def __str__(self):
        return f"{self.student.username} - {self.module.title} - Attempt {self.attempt_number}"


class CourseCertificateTemplate(models.Model):
    course = models.OneToOneField(Course, on_delete=models.CASCADE)
    template_name = models.CharField(max_length=120, default='Default Certificate')
    signer_name = models.CharField(max_length=120, blank=True)
    signer_title = models.CharField(max_length=120, blank=True)
    qr_verify_base_url = models.URLField(blank=True)
    auto_issue = models.BooleanField(default=True)

    def __str__(self):
        return f"Certificate Template - {self.course.title}"


class IssuedCertificate(models.Model):
    student = models.ForeignKey(User, on_delete=models.CASCADE)
    course = models.ForeignKey(Course, on_delete=models.CASCADE)
    certificate_number = models.CharField(max_length=64, unique=True)
    verification_hash = models.CharField(max_length=64, unique=True)
    issued_at = models.DateTimeField(auto_now_add=True)
    is_active = models.BooleanField(default=True)
    revoked_at = models.DateTimeField(null=True, blank=True)
    revoked_by = models.ForeignKey(
        User,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name='revoked_certificates',
    )
    revoke_reason = models.CharField(max_length=255, blank=True)

    class Meta:
        ordering = ['-issued_at']
        unique_together = ('student', 'course')
```

models.py

```python
class IssuedCertificate(models.Model):
    class Meta:
        unique_together = ('student', 'course')

    def __str__(self):
        return f"{self.certificate_number} - {self.student.username}"

    def revoke(self, revoked_by=None, reason=''):
        self.is_active = False
        self.revoked_at = timezone.now()
        self.revoked_by = revoked_by
        self.revoke_reason = reason
        self.save(update_fields=['is_active', 'revoked_at', 'revoked_by', 'revoke_reason'])

    @staticmethod
    def build_verification_hash(student_id, course_id, certificate_number):
        raw = f"{student_id}:{course_id}:{certificate_number}"
        return hashlib.sha256(raw.encode()).hexdigest()


class Certificate(models.Model):
    STATUS_CHOICES = [
        ('pending_teacher', 'Pending Teacher Approval'),
        ('pending_admin', 'Pending Admin Approval'),
        ('approved', 'Approved'),
        ('rejected', 'Rejected'),
    ]

    user = models.ForeignKey(User, on_delete=models.CASCADE)
    course = models.ForeignKey(Course, on_delete=models.CASCADE)
    certificate_id = models.CharField(max_length=50, unique=True)
    verification_token = models.UUIDField(default=uuid.uuid4, unique=True)
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='pending_teacher')
    issued_at = models.DateTimeField(null=True, blank=True)
    certificate_file = models.FileField(upload_to='certificates/', null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

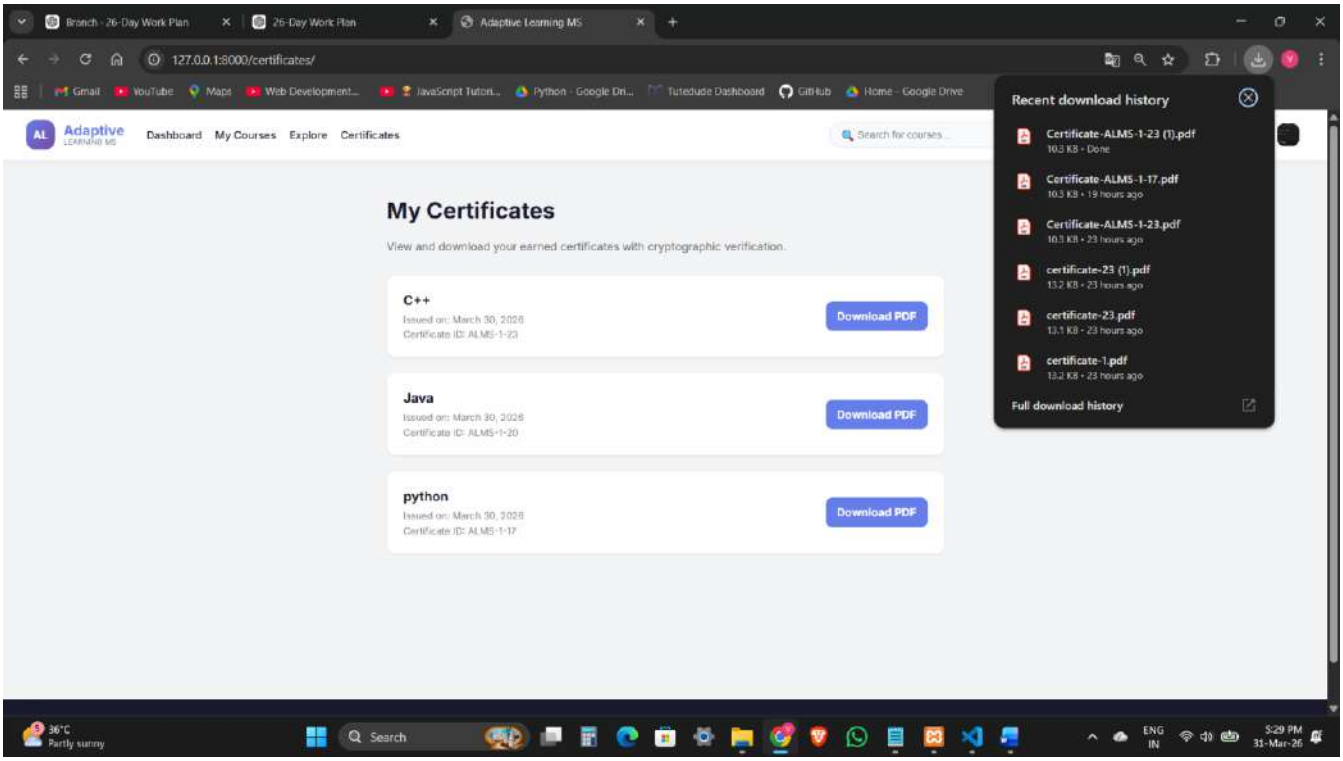
models.py

```python
class Certificate(models.Model):

    @staticmethod
    def generate_unique_id():
        import random
        import string
        year = timezone.now().year
        # Generate 4 random uppercase chars/digits
        rand = ''.join(random.choices(string.ascii_uppercase + string.digits, k=4))
        cert_id = f"ALMS-{year}-{rand}"
        # Ensure uniqueness
        while Certificate.objects.filter(certificate_id=cert_id).exists():
            rand = ''.join(random.choices(string.ascii_uppercase + string.digits, k=4))
            cert_id = f"ALMS-{year}-{rand}"
        return cert_id


class PlatformSetting(models.Model):
    platform_name = models.CharField(max_length=255, default='Adaptive Learning LMS')
    maintenance_mode = models.BooleanField(default=False)
    video_host = models.CharField(max_length=255, blank=True)
    signed_url_secret = models.CharField(max_length=255, blank=True)
    certificate_signer = models.CharField(max_length=255, blank=True)
    token_ttl_seconds = models.PositiveIntegerField(default=300)
    attention_monitoring_enabled = models.BooleanField(default=False)
    siem_endpoint = models.URLField(blank=True)
    blockchain_anchor_enabled = models.BooleanField(default=False)
    email_from_address = models.EmailField(blank=True)
    smtp_host = models.CharField(max_length=255, blank=True)
    smtp_port = models.PositiveIntegerField(default=587)
    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return "Platform Settings"
```

Day 4 :- Implement certificate download (PDF generation for student)

```python
def verify_certificate(request, cert_id):
    })

@login_required
def download_certificate(request, cert_id):
    """Secure download for students to get their issued certificate PDF."""
    certificate = get_object_or_404(Certificate, certificate_id=cert_id, user=request.user)

    if certificate.status != 'approved':
        messages.error(request, "This certificate is not yet approved.")
        return redirect('certificates')

    # On-the-fly generation if file is missing (for legacy or failed generations)
    if not certificate.certificate_file:
        from .certificate_generator import generate_certificate_pdf
        try:
            generate_certificate_pdf(certificate)
        except Exception as e:
            messages.error(request, f"Unable to generate certificate PDF: {str(e)}")
            return redirect('certificates')

    return FileResponse(
        certificate.certificate_file.open('rb'),
        content_type='application/pdf',
        as_attachment=True,
        filename=f"Certificate-{certificate.certificate_id}.pdf"
    )

@login_required
@student_required
def notifications_view(request):
    if request.GET.get('action') == 'mark_read':
        Notification.objects.filter(user=request.user, is_read=False).update(is_read=True)
        return redirect('notifications')

    notifications = Notification.objects.filter(user=request.user).order_by('-created_at')
```

```python
def video_replay(request):
            )

            # For replay tracking, we could add a field to ModuleProgress if needed
            # For now, just acknowledge
            return JsonResponse({'status': 'ok'})
        except Exception as e:
            return JsonResponse({'status': 'error', 'message': str(e)}, status=400)
    return JsonResponse({'status': 'error', 'message': 'Invalid method'}, status=405)

@student_required
def certificates_view(request):
    """View list of student's certificates and issue eligible ones."""
    enrollments = Enrollment.objects.filter(student=request.user)
    for enrollment in enrollments:
        _issue_certificate_if_eligible(request.user, enrollment.course)

    certificates = Certificate.objects.filter(user=request.user).select_related('course').order_by('-created_at')
    return render(request, 'student/certificates.html', {'certificates': certificates})

def verify_certificate(request, cert_id):
    """Public verification page for certificates."""
    certificate = get_object_or_404(Certificate, certificate_id=cert_id, status='approved')
    return render(request, 'student/verify_certificate.html', {
        'certificate': certificate,
        'issuer_name': "Adaptive Learning LMS",
        'issued_date': certificate.issued_at
    })

@login_required
def download_certificate(request, cert_id):
    """Secure download for students to get their issued certificate PDF."""
    certificate = get_object_or_404(Certificate, certificate_id=cert_id, user=request.user)

    if certificate.status != 'approved':
        messages.error(request, "This certificate is not yet approved.")
```



- **Week 12 Plan :-** Improve UI/UX and test complete student learning workflow.

Total Working Hrs: 28 hours **Signature of Student:**

The above entries are correct and the grading of work done by Trainee is

Excellent	Very Good	Good	Fair	Below Average	Poor

Signature of External Guide with Company Seal: **Signature of Internal Guide**

Date: **Date:**

SAL EDUCATION

SAL ENGINEERING AND TECHNICAL INSTITUTE

Opp. Science city, Sola-Bhadaj Road, Ahmedabad, Gujarat 380060
Ph:- 079 67129000 Website: www.sal.edu.in

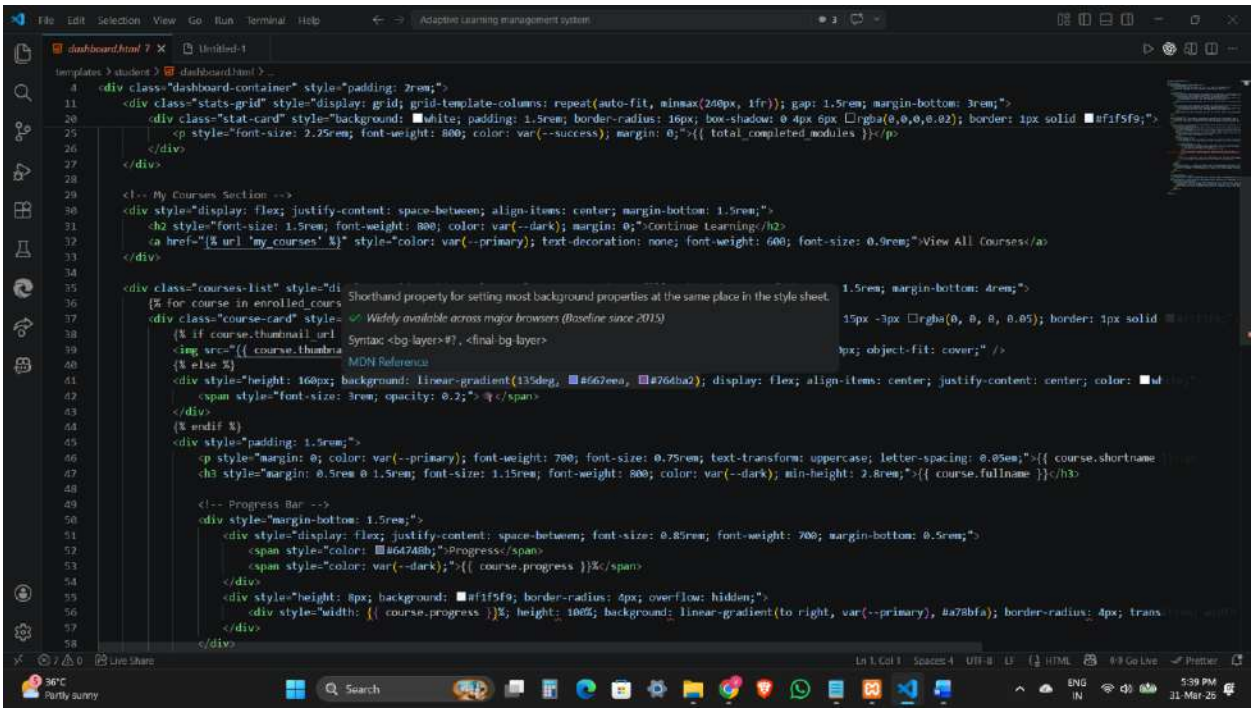
Student's Weekly Report of Internship

Student Name:	Vekariya Vraj Dipakbhai			
Enrollment Number:	221260116056			
Name Of Organization:	Bisag N			
External Guide Name:	Parth Sutariya			
External Guide Contact details:	**Email:**	Parthsutariya12@gmail.com	**Mob No:**	81558 51038
Internal Faculty Guide Name:	Prof. Amitkumar J. Patel			

Week 12 (23/03/2026 – 29/03/2026)

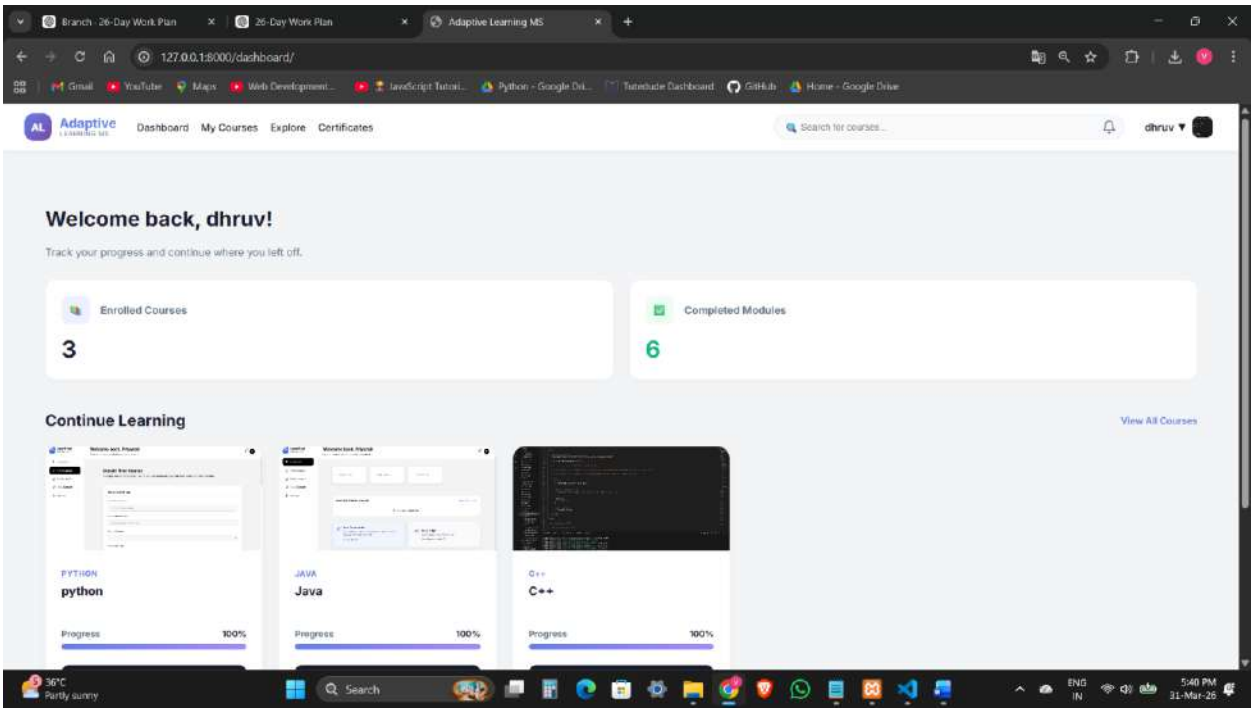
1. Improve student dashboard UI (layout, buttons, usability enhancements)
2. Add notification system (course completion, quiz updates, assignment status)
3. Implement Assignment Module (Student Side):
   - View assignment PDF/task
   - Submit GitHub & Google Drive links
   - Display submission status (Pending / Approved / Rejected)
   - View feedback after teacher review
4. Perform end-to-end testing (enroll → video → quiz → assignment → certificate)

Day 1 :- Improve student dashboard UI (layout, buttons, usability enhancements)

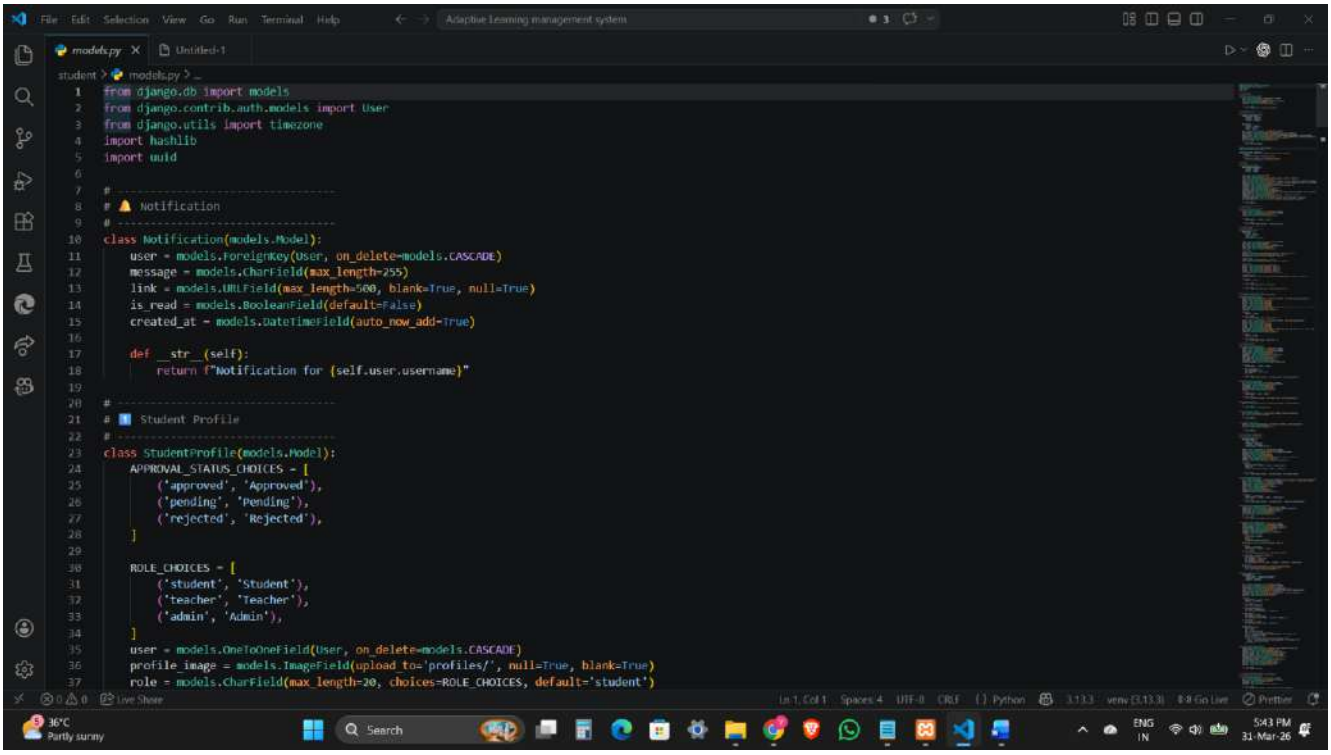


```html
{% extends 'student/base.html' %}

{% block content %}
<style>
    .courses-grid {
        display: grid;
        grid-template-columns: repeat(auto-fill, minmax(340px, 1fr));
        gap: 2rem;
        margin-top: 2rem;
    }

    .modern-course-card {
        background: white;
        border-radius: 16px;
        overflow: hidden;
        border: 1px solid #f1f5f9;
        box-shadow: 0 4px 6px rgba(0, 0, 0, 0.02);
        display: flex;
        flex-direction: column;
        transition: transform 0.2s;
    }

    .modern-course-card:hover {
        transform: translateY(-4px);
        box-shadow: 0 10px 15px rgba(0, 0, 0, 0.05);
    }

    .modern-course-img {
        height: 180px;
        width: 100%;
        background: #f8fafc;
        display: flex;
        align-items: center;
        justify-content: center;
        position: relative;
    }
```

```html
<div class="explore-container">
    <div class="courses-grid">
        <div class="modern-course-card">
            <div class="modern-course-content">

                <div
                    style="display: flex; align-items: center; justify-content: space-between; margin-top: auto; border-top: 1px solid #f1f5f9; padding-top: 1.5rem;">
                    <div style="color: #94a3b8; font-size: 0.85rem; font-weight: 600;">
                        <span> {{ course.enrollment_set.count }} Students</span>
                    </div>

                    {% if course.id in enrolled_course_ids %}
                    <a href="{% url 'my_courses' %}" class="btn-action btn-green">
                        Enrolled
                    </a>
                    {% else %}
                    <a href="{% url 'enroll_course' course.id %}" class="btn-action btn-blue">
                        Enroll Now
                    </a>
                    {% endif %}
                </div>
            </div>
        </div>
        {% empty %}
        <div
            style="grid-column: 1 / -1; text-align: center; padding: 4rem; background: white; border-radius: 16px; border: 1px dashed #cbd5e1;">
            <p style="font-size: 1.1rem; color: #64748b; margin-top: 1rem;">No courses found matching your criteria.</p>
            <a href="{% url 'explore' %}" style="color: var(--primary); font-weight: 600; text-decoration: none;">View
                all courses</a>
        </div>
        {% endfor %}
    </div>
</div>
{% endblock %}
```

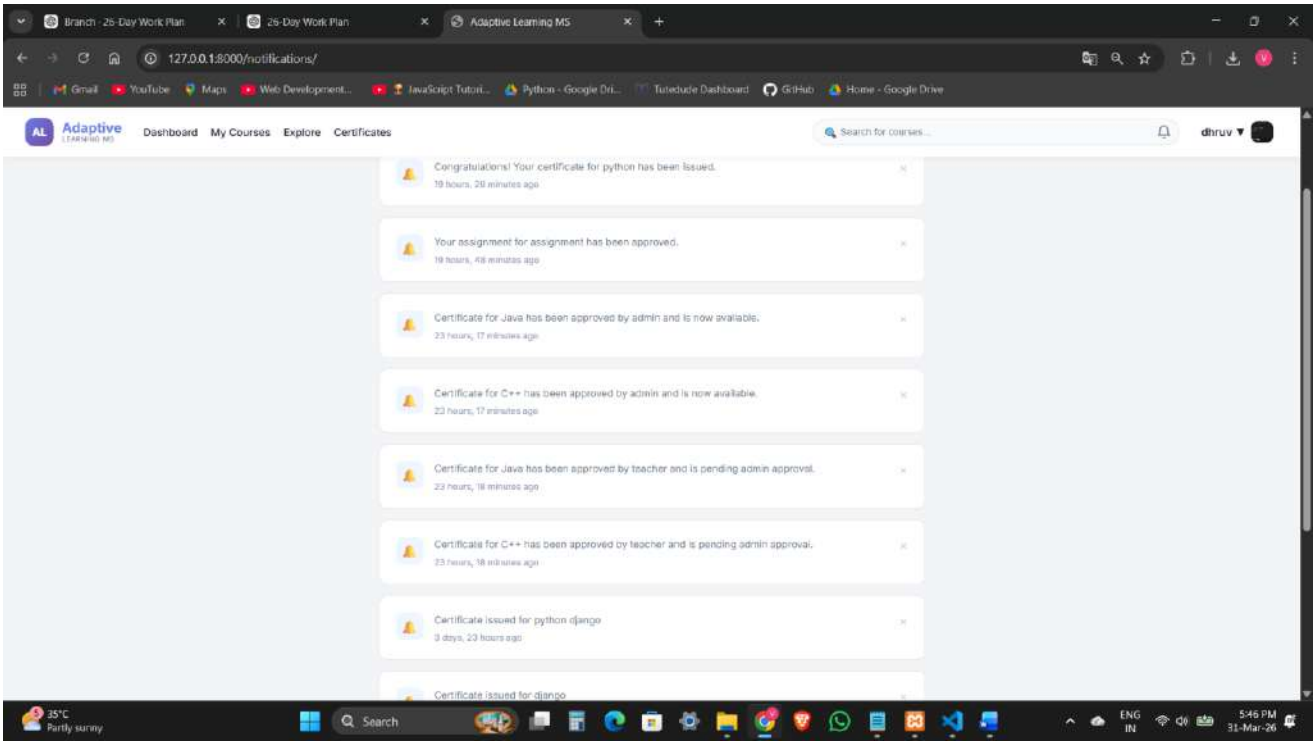


Day 2 :- Add notification system (course completion, quiz updates, assignment status)



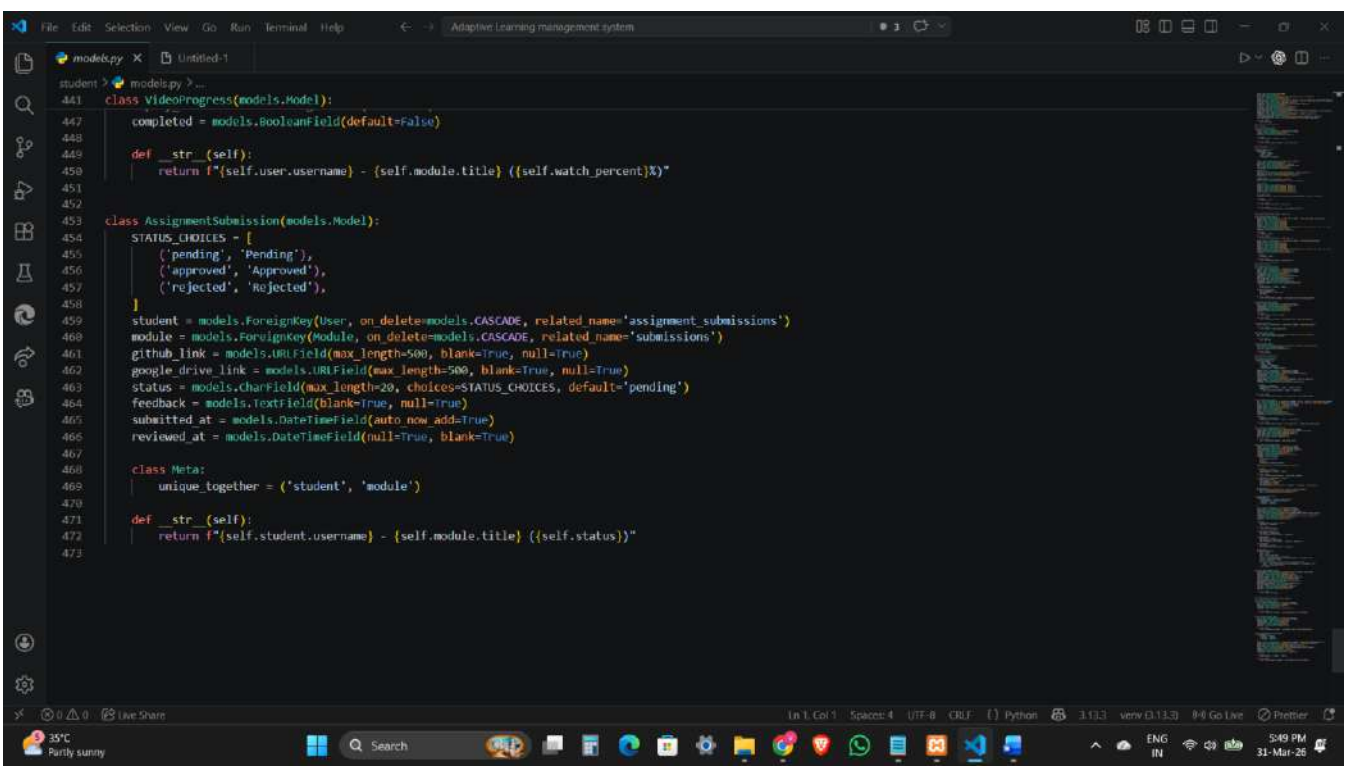
```python
@login_required
@csrf_exempt
def video_replay(request):
    if request.method == 'POST':
        try:
            data = json.loads(request.body)
            module_id = data.get('module_id')

            module = get_object_or_404(Module, id=module_id, type='video')
            progress, _ = ModuleProgress.objects.get_or_create(
                user=request.user,
                module=module
            )

            # For replay tracking, we could add a field to ModuleProgress if needed
            # For now, just acknowledge
            return JsonResponse({'status': 'ok'})
        except Exception as e:
            return JsonResponse({'status': 'error', 'message': str(e)}, status=400)
    return JsonResponse({'status': 'error', 'message': 'Invalid method'}, status=405)

@student_required
def certificates_view(request):
    """View list of student's certificates and issue eligible ones."""
    enrollments = Enrollment.objects.filter(student=request.user)
    for enrollment in enrollments:
        _issue_certificate_if_eligible(request.user, enrollment.course)

    certificates = Certificate.objects.filter(user=request.user).select_related('course').order_by('-created_at')
    return render(request, 'student/certificates.html', {'certificates': certificates})

def verify_certificate(request, cert_id):
    """Public verification page for certificates."""
    certificate = get_object_or_404(Certificate, certificate_id=cert_id, status='approved')
    return render(request, 'student/verify_certificate.html', {
```

```python
def download_certificate(request, cert_id):

    return FileResponse(
        certificate.certificate_file.open('rb'),
        content_type='application/pdf',
        as_attachment=True,
        filename=f"Certificate-{certificate.certificate_id}.pdf"
    )

@login_required
@student_required
def notifications_view(request):
    if request.GET.get('action') == 'mark_read':
        Notification.objects.filter(user=request.user, is_read=False).update(is_read=True)
        return redirect('notifications')

    notifications = Notification.objects.filter(user=request.user).order_by('-created_at')

    # Optionally mark all as read automatically on visit
    # Notification.objects.filter(user=request.user, is_read=False).update(is_read=True)

    return render(request, 'student/notifications.html', {'notifications': notifications})

@login_required
@student_required
def delete_notification(request, notification_id):
    notification = get_object_or_404(Notification, id=notification_id, user=request.user)
    notification.delete()
    return redirect('notifications')
```

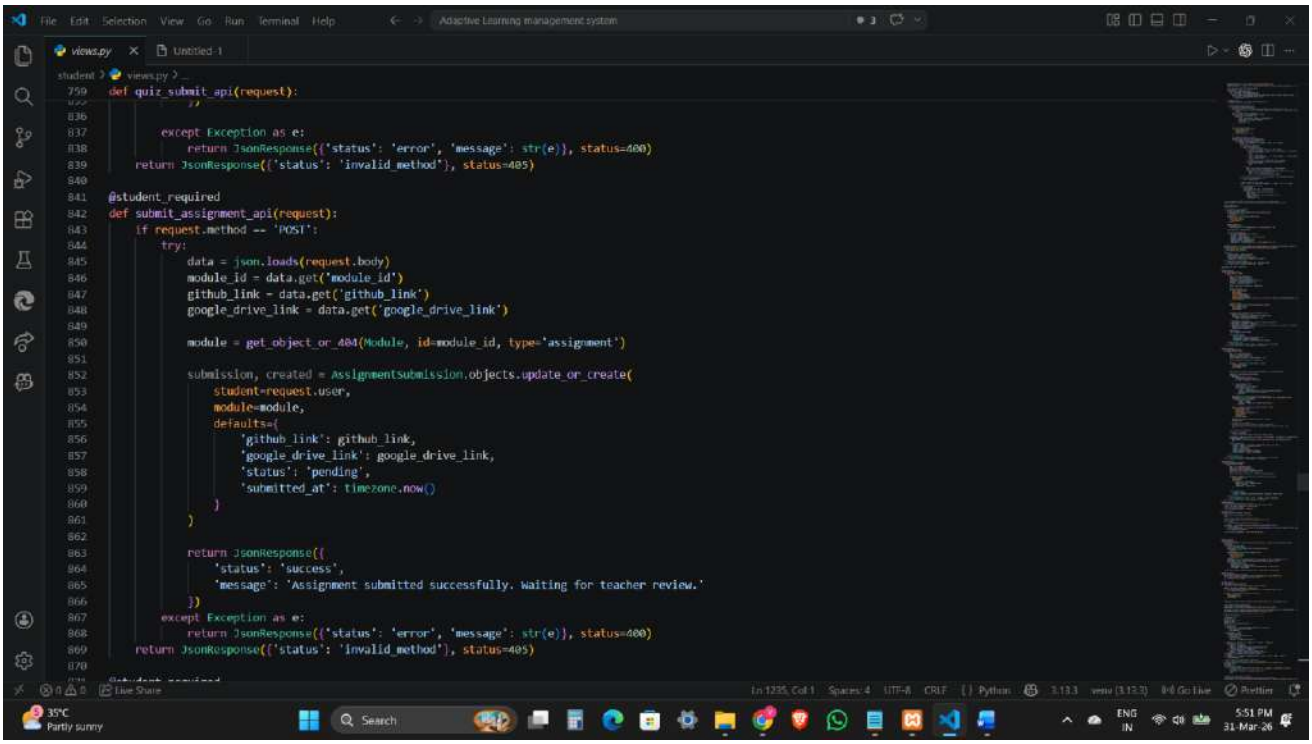


Day 3 :- Implement Assignment Module (Student Side):

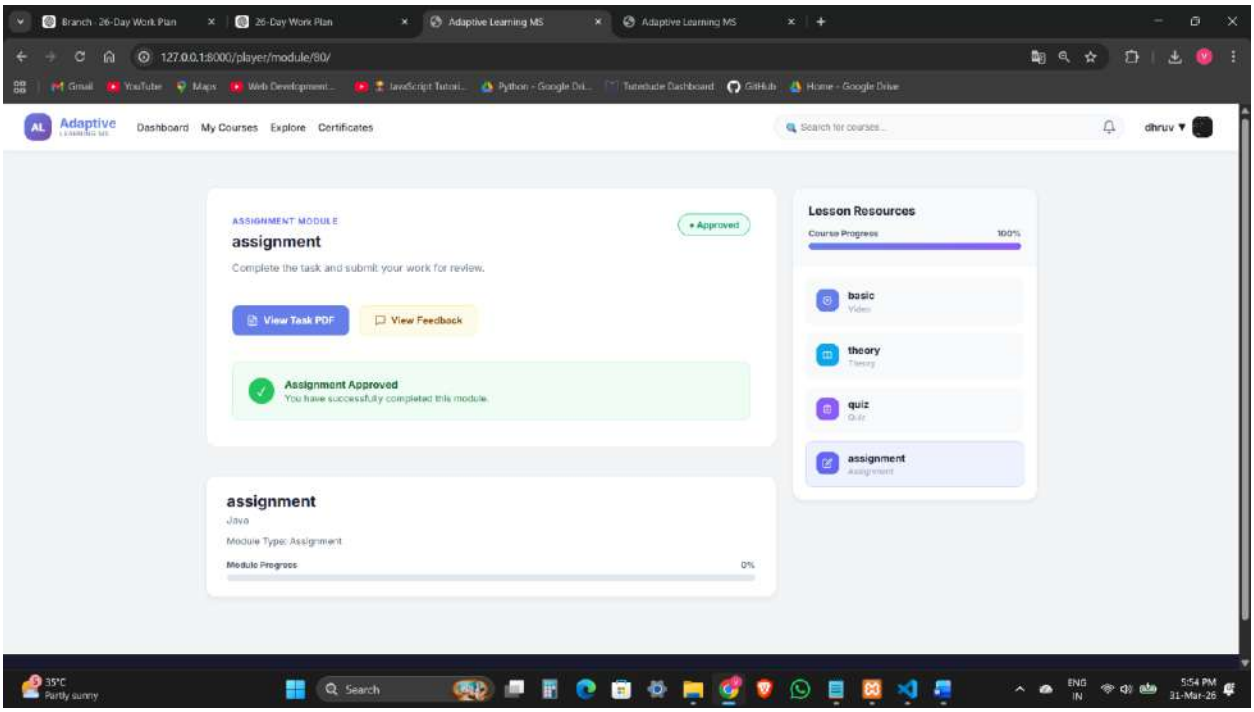
- View assignment PDF/task
- Submit GitHub & Google Drive links
- Display submission status (Pending / Approved / Rejected)
- View feedback after teacher review

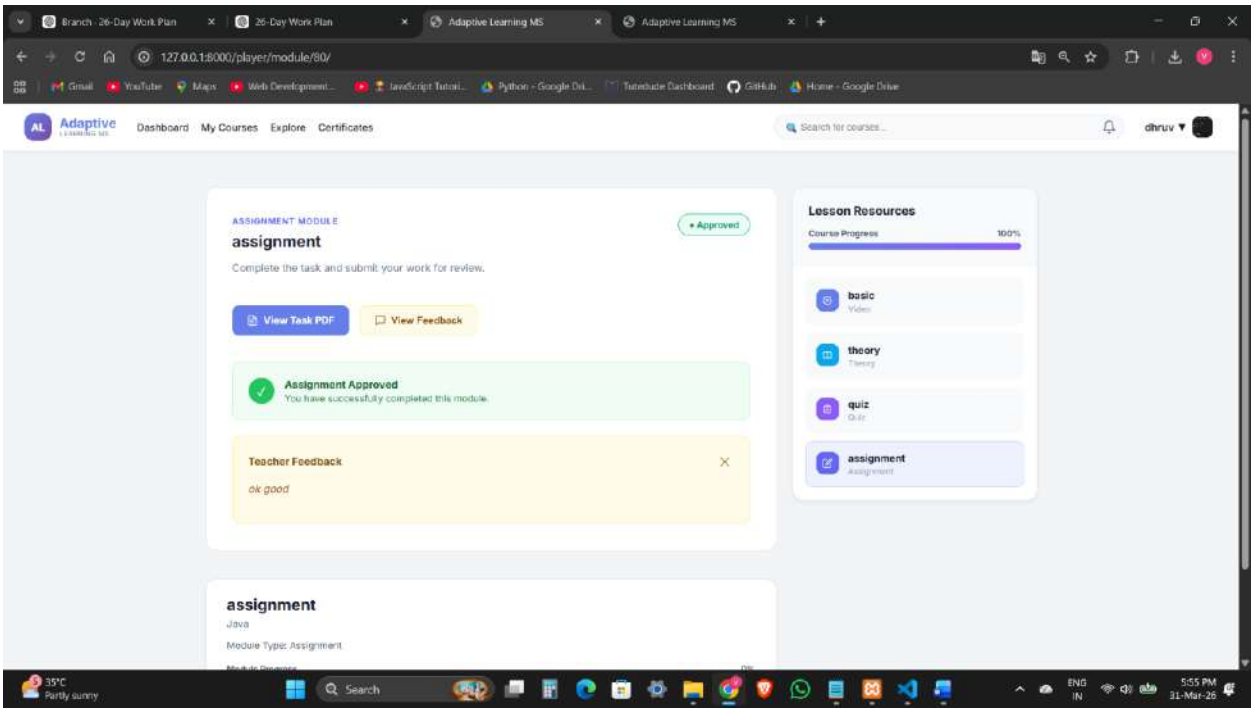


```python
class VideoProgress(models.Model):
    completed = models.BooleanField(default=False)

    def __str__(self):
        return f"{self.user.username} - {self.module.title} ({self.watch_percent}%)"


class AssignmentSubmission(models.Model):
    STATUS_CHOICES = [
        ('pending', 'Pending'),
        ('approved', 'Approved'),
        ('rejected', 'Rejected'),
    ]
    student = models.ForeignKey(User, on_delete=models.CASCADE, related_name='assignment_submissions')
    module = models.ForeignKey(Module, on_delete=models.CASCADE, related_name='submissions')
    github_link = models.URLField(max_length=500, blank=True, null=True)
    google_drive_link = models.URLField(max_length=500, blank=True, null=True)
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='pending')
    feedback = models.TextField(blank=True, null=True)
    submitted_at = models.DateTimeField(auto_now_add=True)
    reviewed_at = models.DateTimeField(null=True, blank=True)

    class Meta:
        unique_together = ('student', 'module')

    def __str__(self):
        return f"{self.student.username} - {self.module.title} ({self.status})"
```



```python
def quiz_submit_api(request):
        )

        except Exception as e:
            return JsonResponse({'status': 'error', 'message': str(e)}, status=400)
    return JsonResponse({'status': 'invalid_method'}, status=405)

@student_required
def submit_assignment_api(request):
    if request.method == 'POST':
        try:
            data = json.loads(request.body)
            module_id = data.get('module_id')
            github_link = data.get('github_link')
            google_drive_link = data.get('google_drive_link')

            module = get_object_or_404(Module, id=module_id, type='assignment')

            submission, created = AssignmentSubmission.objects.update_or_create(
                student=request.user,
                module=module,
                defaults={
                    'github_link': github_link,
                    'google_drive_link': google_drive_link,
                    'status': 'pending',
                    'submitted_at': timezone.now()
                }
            )

            return JsonResponse({
                'status': 'success',
                'message': 'Assignment submitted successfully. Waiting for teacher review.'
            })
        except Exception as e:
            return JsonResponse({'status': 'error', 'message': str(e)}, status=400)
    return JsonResponse({'status': 'invalid_method'}, status=405)
```





Day 4 :- Perform end-to-end testing (enroll → video → quiz → assignment → certificate)

- Data Model
  - Add a submission model like:
- AssignmentSubmission
  - student_id
  - assignment_id
  - github_link
  - drive_link
  - status (Pending / Approved / Rejected)
  - feedback
  - submitted_at

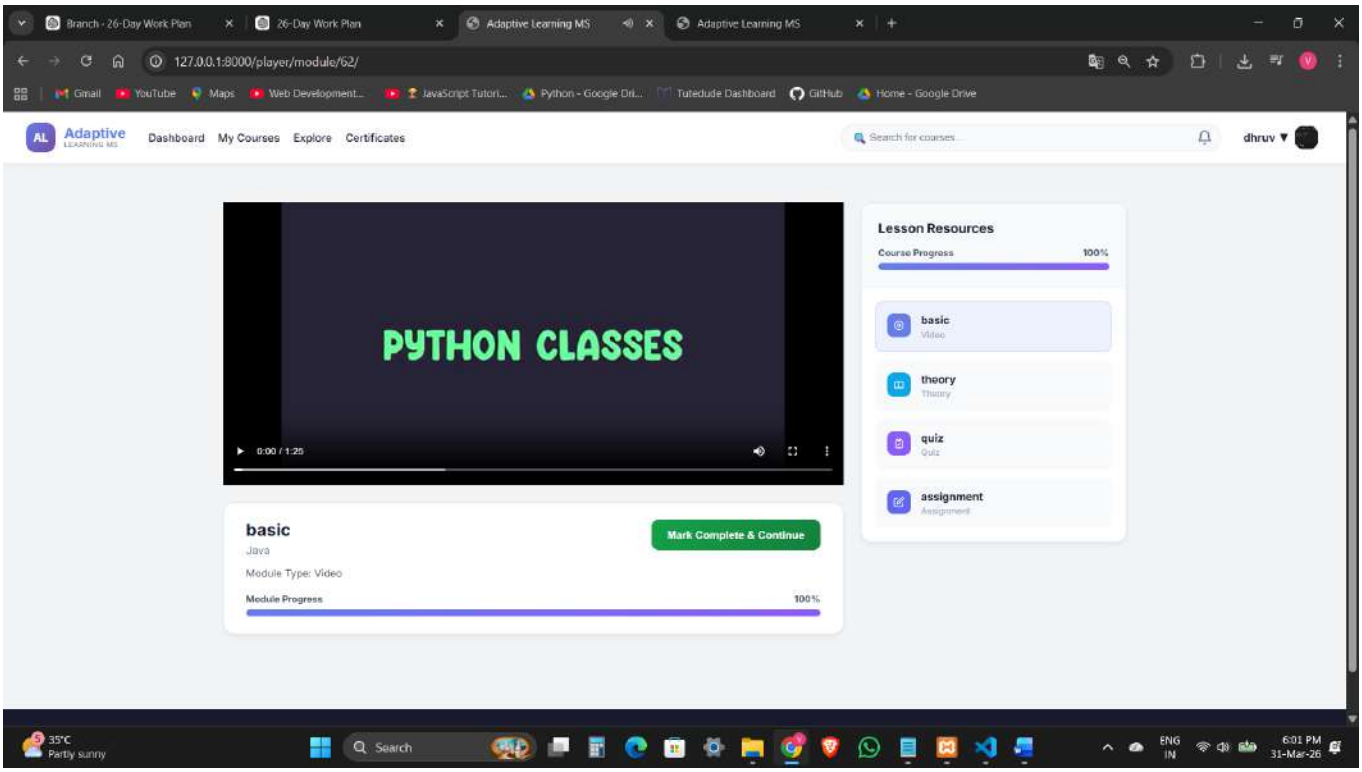
- Frontend Flow (Student Panel)

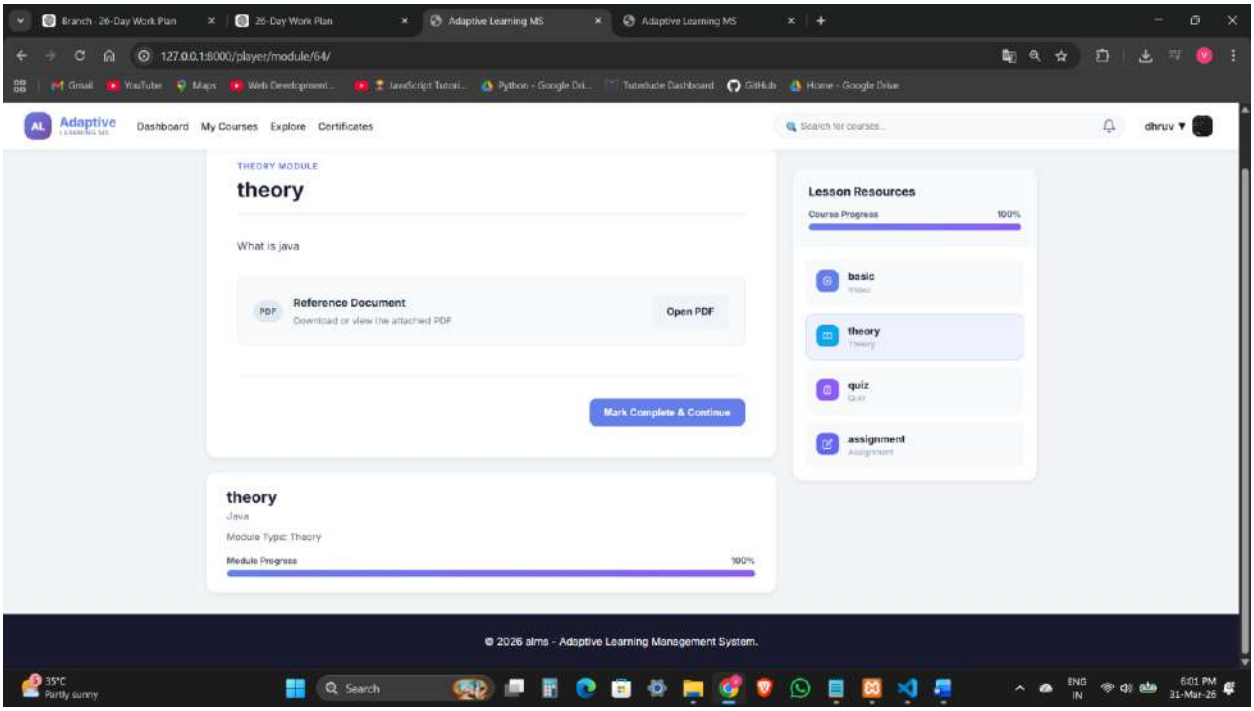
  Inside course player:
  - Add new module type → Assignment
  - UI States:
    - Not Submitted → Show Submit Form
    - Submitted → Show "Pending Review"
    - Approved → Show "Completed ✅"
    - Rejected → Show feedback + resubmit option

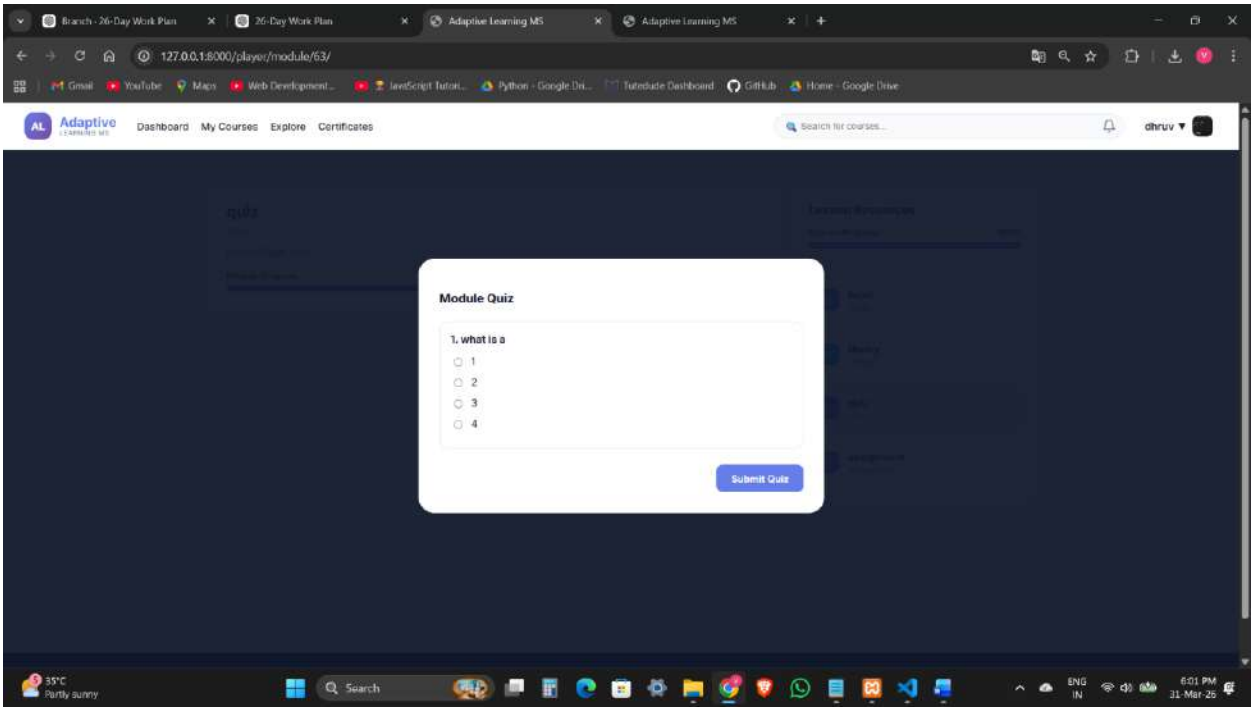
- 3. Validation Rules

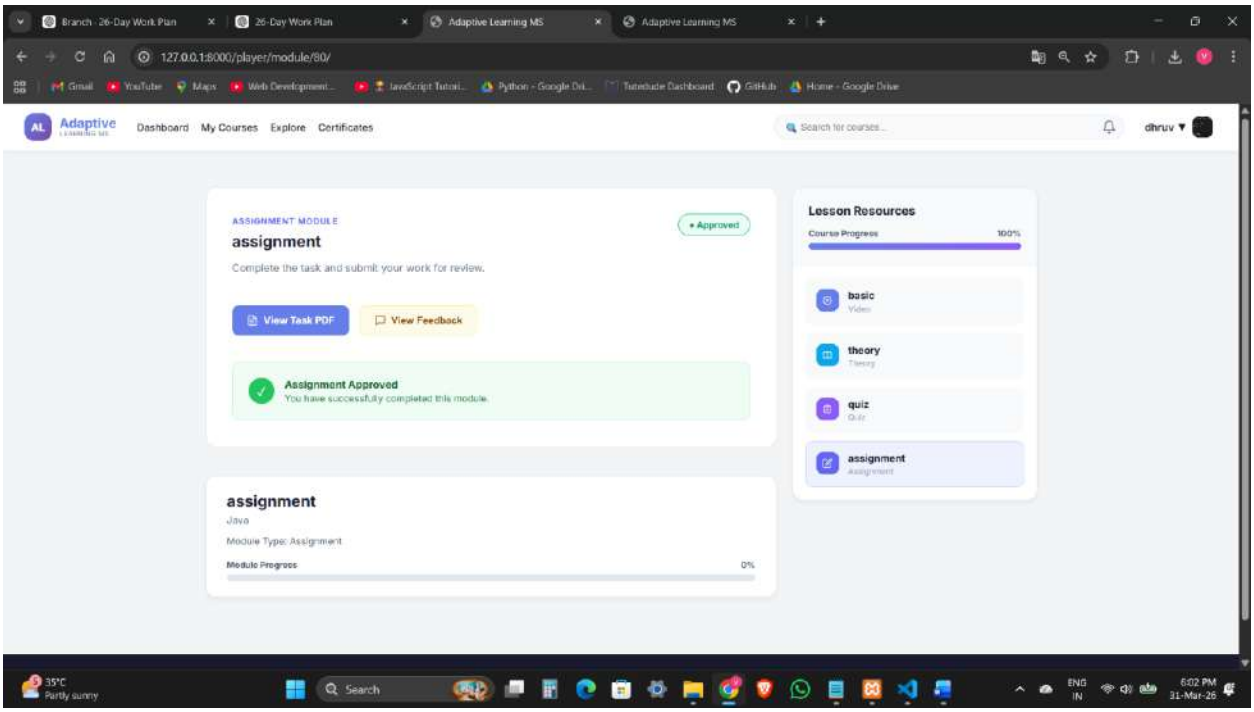
    - Prevent empty link submission
    - Ensure valid URL format
    - Allow resubmission only if rejected

- Integration with Completion Logic
    - Very important:
    - Assignment should be treated like:
        - module_completed = (status == "Approved")
    - Only then allow:
        - Next module unlock
        - Certificate eligibility




Branch - 26-Day Work Plan
26-Day Work Plan
Adaptive Learning MS
Adaptive Learning MS
127.0.0.1:8000/player/module/64/
Adaptive
Dashboard
My Courses
Explore
Certificates
Search for courses...
dhruv
THEORY MODULE
theory
What is java
Reference Document
Download or view the attached PDF
Open PDF
Mark Complete & Continue
Lesson Resources
Course Progress
100%
basic
theory
quiz
assignment
theory
Java
Module Type: Theory
Module Progress
100%
© 2026 alms - Adaptive Learning Management System.
35°C
Partly sunny
Search
ENG
IN
6:01 PM
31-Mar-26


Branch - 26-Day Work Plan
26-Day Work Plan
Adaptive Learning MS
Adaptive Learning MS
127.0.0.1:8000/player/module/63/
Adaptive
Dashboard
My Courses
Explore
Certificates
Search for courses...
dhruv
Module Quiz
1. what is a
1
2
3
4
Submit Quiz
35°C
Partly sunny
Search
ENG
IN
6:01 PM
31-Mar-26



- **Week 13 Plan :-** Enhance student experience with assignment tracking analytics, resume learning, and performance optimization.

Total Working Hrs: 28 hours **Signature of Student:**

The above entries are correct and the grading of work done by Trainee is

Excellent	Very Good	Good	Fair	Below Average	Poor

Signature of External Guide with Company Seal: **Signature of Internal Guide**

Date: **Date:**